



First Level Students

FOUNDATIONS OF DIGITAL CIRCUITS

Dr. Gehad Taher
Dr. Amira Eleff

Contents

Contents	2
1 Introduction to Digital Systems	6
1.1 The Digital System Hierarchy	6
1.2 Overview of Digital vs. Analog Systems	10
1.3 Understanding Binary Digital Signals in Digital Systems	17
1.4 Digital Waveforms	19
1.5 Waveform Characteristics	20
1.6 Timing Diagrams	21
1.7 Introduction to Binary, Octal, Decimal, and Hexadecimal Systems	26
1.8 Exercises	29
2 Number System Conversion and Arithmetic Operations	32
2.1 Conversion Methods	32
2.2 Complements	43
2.3 Signed and Complements of Binary Numbers	49
2.4 Arithmetic operations	53
2.5 Codes in Digital Systems	61

2.6	Binary Storage and Register:	65
2.7	Exercises	69
3	Logic Gates	74
3.1	What are Logic Gates?	74
3.2	The NOT Gate (Inverter)	75
3.3	The OR Gate	76
3.4	The AND Gate	76
3.5	The XOR Gate	77
3.6	The NOR Gate	78
3.7	The NAND Gate	79
3.8	The XNOR Gate	79
3.9	Universal Gates	80
3.10	Exercises	84
4	Boolean Algebra and Simplification	87
4.1	Boolean Expressions	87
4.2	Logic Gates for Boolean Operations	88
4.3	Truth Tables for Multiple Variables	89
4.4	Laws and Rules of Boolean Algebra	90
4.5	DeMorgan's Theorem	95
4.6	Logic Simplification Using Boolean Algebra	98
4.7	Sum of Products (SOP) in Digital Logic	101
4.8	Product of Sums (POS) in Digital	106
4.9	Simplifying Circuits Using K-Maps	110
4.10	Don't Care in K-Maps:	117
4.11	Exercises: Circuit Simplification	118

5	Combinational Logic Gates	122
5.1	Structure of Combinational Circuits	124
5.2	Arithmetic Circuits & Data Circuits	130
5.3	Decoder and Encoder	151
5.4	Multiplexer And De- Multiplexer	170
5.5	Exercises	188
6	Sequential Circuits	192
6.1	Sequential Circuits	193
6.2	Flip-Flops: SR, JK, D, T	198
6.3	Latches in Digital Electronics	215
	Bibliography	226

Introduction

This ebook, Foundations of Digital Circuits, is designed as an introduction to the foundational concepts of digital electronics and systems aiming to understand the basics of digital design, binary arithmetic, and logic circuits.

This ebook begins with fundamental topics then delves into practical applications and logic circuits. Each chapter integrates theoretical explanations, examples, and hands-on lab activities to enhance understanding.

Chapter 1

Introduction to Digital Systems

Digital systems are foundational to modern technology, including computers, smartphones, and embedded devices. These systems operate by organizing their components into a well-defined hierarchy, each level building upon the previous. Understanding this hierarchy is essential for designing, analyzing, and troubleshooting digital systems.

1.1 The Digital System Hierarchy

Digital systems are organized in a hierarchy that consists of:

1. **Systems:** Examples include computers and digital cameras.
2. **Functional Units:** Registers, memory, and arithmetic units.
3. **Gates and Flip-Flops:** Basic building blocks of logic circuits.
4. **Circuits:** Combinations of gates and flip-flops designed to perform specific tasks.

Each layer plays a specific role in processing digital data, starting from the basic logic gates to complete systems like computers or digital cameras.

Systems

Definition: A system is the highest level in the hierarchy and represents the complete device or application, such as a computer or digital television. Systems perform complex tasks like computation, communication, or multimedia playback.

Examples of Systems:

- **Computers:** Handle data processing and storage.
- **Digital Cameras:** Capture and process images.
- **Electronic Calculators:** Perform arithmetic operations.
- **Digital Televisions:** Decode and display video signals.

Functional Units

Definition: Functional units are subsystems within a system that perform specific tasks. They consist of multiple circuits and components working together to achieve a particular functionality.

Examples of Functional Units:

- **Arithmetic Logic Unit (ALU):** Performs arithmetic and logical operations in a computer.
- **Registers:** Temporarily store data during processing.
- **Memory Units:** Store data and instructions for execution.

- **Control Units:** Manage the flow of data and instructions within a system.

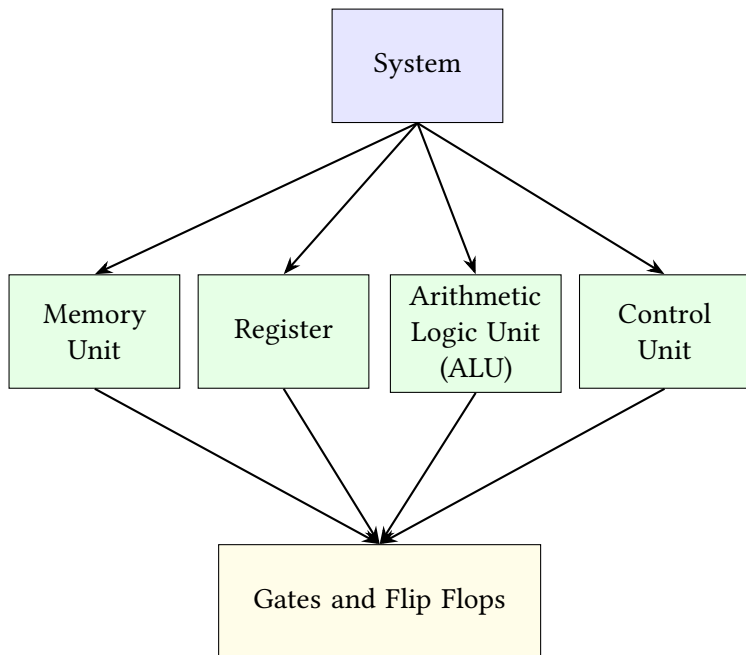


Figure 1.1: Functional unit hierarchy in a digital system.

Gates and Flip-Flops

Definition: Gates and flip-flops form the basic building blocks of digital circuits. They perform simple logical operations and store data temporarily.

Logic Gates:

- **AND, OR, NOT gates:** Perform basic Boolean operations.
- **XOR and NAND gates:** Implement more complex logic functions.

Flip-Flops: Flip-flops are memory elements that store one bit of data. They are essential for building sequential circuits like counters and registers.

4. Circuits

Definition: Circuits combine gates and flip-flops to create functional blocks. These blocks implement specific logic or storage functions.

Types of Circuits:

- **Combinational Circuits:** Output depends only on the current inputs. Examples: Adders, multiplexers.
- **Sequential Circuits:** Output depends on current inputs and past states. Examples: Counters, registers.

Summary of the Hierarchy

- **Circuits** are the foundation, combining gates and flip-flops.
- **Gates and Flip-Flops** perform basic operations and store data.

- **Functional Units** group circuits to perform specific tasks.
- **Systems** integrate functional units into complete devices.

This layered organization allows digital systems to be modular, scalable, and easier to design and debug.

1.2 Overview of Digital vs. Analog Systems

What Are Digital Systems?

A **digital system** is a collection of components that process discrete data, typically in **binary form** (0s and 1s). These systems are widely used in electronics, computers, and communication devices because of their reliability, efficiency, and precision.

What Are Analog Systems?

An **analog system** processes continuous signals that vary over a range of values. These systems are commonly used in audio, video, and sensor applications where precision in signal amplitude is essential.

Key Differences Between Digital and Analog Systems

Table 1.1: Key Differences Between Digital and Analog Systems

Feature	Digital Systems	Analog Systems
Data Representation	Discrete (0s and 1s)	Continuous signals
Noise Sensitivity	Highly resistant to noise	Prone to distortion
Storage	Easy (binary format in memory)	Difficult (requires physical media)
Applications	Computers, smartphones, digital cameras	Amplifiers, radio, temperature sensors

Amplitude

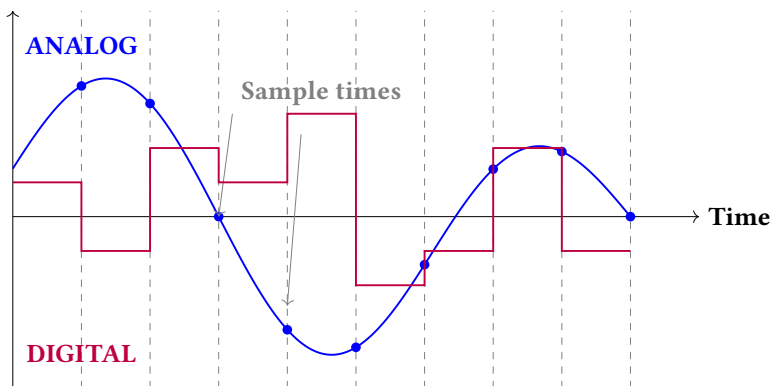


Figure 1.2: Visual Difference between Analog and Digital Signals

Analog-to-Digital Conversion (ADC)

An **Analog-to-Digital Converter (ADC)** is used to transform continuous analog signals into discrete digital signals. This process is crucial for interfacing analog inputs with digital systems.

Steps in ADC:

1. **Sampling:** The analog signal is measured at regular intervals (sampling rate).
2. **Quantization:** The sampled values are rounded to the nearest discrete value.
3. **Encoding:** The quantized values are converted into binary code.

Example: Analog-to-Digital Conversion

Consider an analog signal with voltage levels ranging from 0 V to 5 V. If a 3-bit ADC (Analog-to-Digital Converter) is used:

1. Step 1: Determine the number of levels

The number of levels for an n -bit ADC is given by:

$$\text{Number of Levels} = 2^n$$

Since $n = 3$, we have:

$$2^3 = 8 \text{ levels.}$$

2. Step 2: Calculate the step size

The step size (or resolution) is the voltage difference between consecutive levels, calculated as:

$$\text{Step Size} = \frac{\text{Voltage Range}}{\text{Number of Levels}}$$

Substituting the given values:

$$\text{Step Size} = \frac{5 \text{ V}}{8} = 0.625 \text{ V.}$$

Each level corresponds to an increment of 0.625 V.

3. Step 3: Find the level corresponding to 2.5 V

The level corresponding to a specific voltage is calculated as:

$$\text{Level} = \frac{\text{Voltage}}{\text{Step Size}}$$

12227

For 2.5 V:

$$\text{Level} = \frac{2.5}{0.625} = 4.$$

4. **Step 4: Convert the level to binary code**

Each level corresponds to a unique binary code. The binary equivalent of level 4 is:

$$\text{Binary Code} = 100.$$

Thus, for a 3-bit ADC with a voltage range of 0 V to 5 V, the voltage 2.5 V corresponds to the binary code 100.

Digital-to-Analog Converters (DACs)

A **Digital-to-Analog Converter (DAC)** transforms discrete digital signals into continuous analog signals. DACs are widely used in applications where digital data must be interfaced with the analog world. For example:

- In audio systems, DACs convert digital sound files into analog signals for speakers.
- In display systems, DACs generate analog video signals from digital data.

How DACs Work

A DAC takes a digital input (binary) and outputs a proportional analog voltage or current. The conversion is based on the following principles:

1. The DAC resolution, defined by the number of input bits (n), determines the number of discrete levels it can output:

$$\text{Number of Levels} = 2^n.$$

2. The analog output voltage (V_{out}) is proportional to the binary input value (D) and the reference voltage (V_{ref}):

$$V_{\text{out}} = D \cdot \frac{V_{\text{ref}}}{2^n - 1},$$

where D is the decimal equivalent of the binary input.

Example: 3-bit DAC Conversion

Problem: A 3-bit DAC has a reference voltage of 5 V. Find the analog output for the binary input 101_2 .

Solution:

1. **Convert Binary to Decimal:** $101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 5_{10}$.
2. **Determine the Resolution:** For a 3-bit DAC:

$$\text{Number of Levels} = 2^3 = 8.$$

3. **Calculate the Output Voltage:** The output voltage for a given digital input is:

$$V_{\text{out}} = D \cdot \frac{V_{\text{ref}}}{2^n - 1}.$$

Substituting the values:

$$V_{\text{out}} = 5 \cdot \frac{5}{2^3 - 1} = 5 \cdot \frac{5}{7}.$$

Simplify:

$$V_{\text{out}} \approx 3.57 \text{ V}.$$

Result: For the binary input 101_2 , the DAC outputs approximately 3.57 V.

Key Parameters of DACs

- **Resolution:** The number of bits (n) determines the precision of the output signal. Higher resolution provides finer granularity.
- **Reference Voltage (V_{ref}):** The maximum output voltage is directly proportional to the reference voltage.
- **Linearity:** Measures how accurately the analog output matches the expected values.
- **Speed:** Determines how quickly the DAC can convert digital signals to analog.

Why Digital Systems Are Preferred

Digital systems dominate modern technology due to their numerous advantages:

1. **High Noise Immunity:** Digital signals can withstand significant noise without losing data integrity.

2. **Ease of Storage:** Digital data can be stored compactly and retrieved without degradation.
3. **Flexibility:** Digital circuits can be easily reprogrammed for various applications.
4. **Integration:** Modern integrated circuits can house billions of transistors, enabling highly complex digital designs.

Mixed Systems

Many systems today combine digital and analog technologies to leverage the benefits of both. Examples include:

- **CD Players:** Digital data from CDs is converted into analog signals for playback.
- **Smartphones:** Digital processing of signals received from analog microphones and speakers.
- **Automotive Systems:** Sensors provide analog data, processed digitally for engine control.

Real-World Example: Digital Audio Systems

- **Input:** Sound waves are captured by a microphone and converted to an analog voltage.
- **ADC:** The analog signal is sampled, quantized, and encoded into binary.
- **Processing:** Digital processors modify or compress the signal.

- DAC: The digital signal is converted back to analog for playback through speakers.

1.3 Understanding Binary Digital Signals in Digital Systems

Digital systems play a critical role in modern technology, from computing devices to communication systems. At the heart of these systems lies the concept of **binary digital signals**, which serve as the foundation for processing and transmitting information.

What is a Binary Digital Signal?

A binary digital signal is an *information variable* represented by a *physical quantity*. In the realm of digital systems, these variables are constrained to specific, **discrete values**, making them distinctly different from the continuous values used in analog systems.

For instance, in a digital circuit, these discrete values often correspond to physical states such as voltage levels. These states enable the representation of information in a precise and reliable manner.

The Role of Discrete Values in Digital Systems

Digital systems rely on discrete states because they are inherently more resistant to noise and variations than continuous signals. Among these discrete states, **binary values**—which consist of just two levels—are the most commonly used. These two levels are:

- **Low** or **Off** (commonly represented as 0)
- **High** or **On** (commonly represented as 1)

This binary nature simplifies the design and operation of digital systems, enabling efficient data processing and transmission.

Representations of Binary Values

Binary values can be abstracted and represented in different forms depending on the context. Some common representations include:

- **Digits:** 0 and 1
- **Logical States:** False (F) and True (T)
- **Electrical States:** Low (L) and High (H)
- **Physical States:** Off and On

These representations provide flexibility and clarity when interpreting binary signals in various applications. For example:

- In a **logical operation**, True and False might be used.
- In an **electrical circuit**, Low and High might refer to voltage levels (e.g., 0 volts for Low and 5 volts for High).

Why Binary Signals?

The simplicity of a two-level system makes binary signals highly effective for:

1. **Reliability:** Clear distinction between 0 and 1 minimizes errors.
2. **Ease of Implementation:** Binary states are straightforward to design and manage using switches, transistors, and other hardware components.

3. **Compatibility:** Binary systems seamlessly integrate with modern computing and communication technologies.

Real-World Example

Consider a basic digital circuit that uses voltage levels to represent binary values:

- A voltage close to 0 **volts** represents a binary 0 (Low, False, Off).
- A voltage near 5 **volts** represents a binary 1 (High, True, On).

This clear distinction allows digital devices to interpret, process, and transmit information efficiently, forming the backbone of nearly all modern electronic systems.

1.4 Digital Waveforms

Digital waveforms are fundamental in digital systems and represent alternating **HIGH** and **LOW** voltage levels. These waveforms are essential for transmitting and processing information in digital circuits.

Types of Pulses

Digital waveforms are composed of a series of pulses. Pulses are transitions between **LOW** and **HIGH** voltage states, and they can be categorized into two types:

1. **Positive-going pulse:** A transition that begins at the **LOW** voltage level, rises to the **HIGH** voltage level, and then returns to the **LOW** level.

2. **Negative-going pulse:** A transition that starts at the **HIGH** voltage level, drops to the **LOW** voltage level, and then returns to the **HIGH** level.

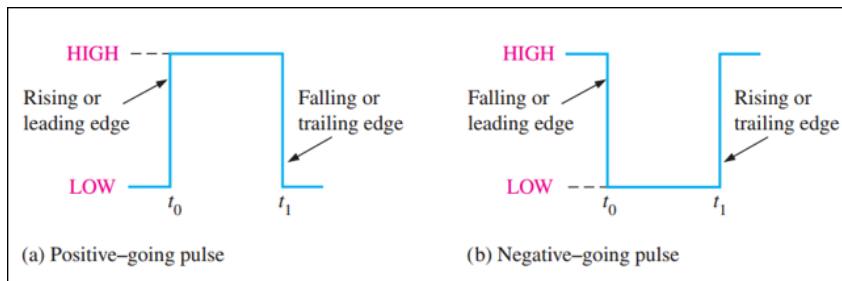


Figure 1.3: Illustration of (a) Positive-going pulse and (b) Negative-going pulse

1.5 Waveform Characteristics

Periodic Pulse Waveforms

Periodic pulse waveforms are composed of pulses that repeat at fixed intervals, called the **period**. The **frequency** of the waveform is the rate at which it repeats and is measured in **hertz (Hz)**.

The relationship between frequency (f) and period (T) is expressed mathematically as:

$$f = \frac{1}{T} \quad \text{or equivalently} \quad T = \frac{1}{f}$$

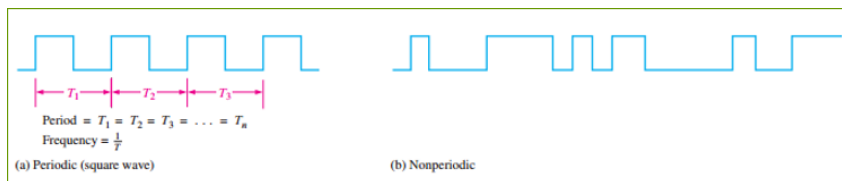


Figure 1.4: (a) Periodic (square wave) (b) Non periodic

For example, consider a repetitive wave with a frequency of $f = 3.2 \text{ GHz}$. The period (T) of this wave can be calculated as:

$$T = \frac{1}{f} = \frac{1}{3.2 \text{ GHz}} = 313 \text{ s}$$

This demonstrates that the period is inversely proportional to the frequency. A higher frequency corresponds to a shorter period and vice versa.

1.6 Timing Diagrams

A **Timing Diagram** is a graphical representation of digital signals as they change states over time. It is commonly used to analyze and debug the behavior of digital circuits.

Components of a Timing Diagram

- **Signals or Waveforms:** Represent digital signals that transition between HIGH and LOW states.
- **Time Axis:** The horizontal axis that shows the time progression.
- **Transitions:** Changes in signal states, including:

- **Rising Edge:** LOW to HIGH transition.
- **Falling Edge:** HIGH to LOW transition.
- **Bit Time:** The duration of one complete HIGH and LOW cycle for a digital signal.

Key Characteristics of Digital Waveforms

- **Rise Time (t_r):** The time it takes for a signal to transition from 10% to 90% of its maximum amplitude.
- **Fall Time (t_f):** The time it takes for a signal to transition from 90% to 10% of its maximum amplitude.
- **Pulse Width (t_W):** The duration for which the signal remains HIGH in a single pulse.
- **Period (T):** The time taken to complete one full cycle of the waveform.
- **Frequency (f):** The number of cycles per second, given by:

$$f = \frac{1}{T}.$$

- **Duty Cycle (D):** The ratio of the pulse width to the period, expressed as a percentage:

$$D = \frac{t_W}{T} \times 100\%.$$

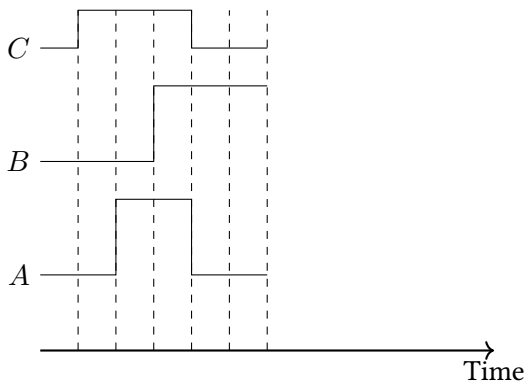


Figure 1.5: Simple Timing Diagram using Three Signals A,B,C

Example Timing Diagram

Below is a simple timing diagram for three signals (A , B , and C): In the diagram:

- A transitions to HIGH at $t = 1$ and returns to LOW at $t = 2$.
- B transitions to HIGH at $t = 1.5$ and remains HIGH until $t = 3$.
- C exhibits multiple transitions, highlighting a more complex waveform.

Applications of Timing Diagrams

Timing diagrams are essential in:

- **Analyzing Signal Behavior:** Understanding transitions and synchronization in digital systems.
- **Debugging Digital Circuits:** Identifying issues such as glitches or delays.
- **Designing Timing for Data Transfer:** Ensuring data is transmitted correctly between devices.

Practical Example: Clock Timing Diagram

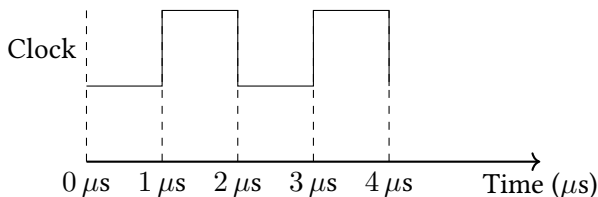
Consider a clock signal with a frequency of 1 MHz. The period is:

$$T = \frac{1}{f} = \frac{1}{1 \text{ MHz}} = 1 \mu\text{s}.$$

If the duty cycle is 50%, the pulse width (t_W) is:

$$t_W = D \times T = 0.5 \times 1 \mu\text{s} = 0.5 \mu\text{s}.$$

The timing diagram is shown below:



Digital Systems

The modern world is often described as the "digital age" because of the pervasive use of digital systems. These systems are designed to process discrete (digital) information, making them versatile, efficient, and reliable. Some examples of digital systems include:

- **Digital Computers:** Used for general purposes like scientific, industrial, and commercial applications.
- **Telecommunication Systems:** Digital switching in telephone exchanges.
- **Consumer Electronics:** Digital cameras, calculators, TVs, and PDAs.
- **Control Systems:** Robots, automated machinery, and embedded systems.

Advantages of Digital Systems:

1. **Accuracy and Reliability:** Digital data can be reproduced with high fidelity, unlike analog systems prone to degradation.
2. **Efficiency:** Digital systems can store and process large amounts of data compactly.
3. **Scalability:** Easy to integrate with advanced technologies like machine learning, IoT, and cloud computing.

1.7 Introduction to Binary, Octal, Decimal, and Hexadecimal Systems

Digital systems operate using various number systems to represent and process information. The four primary number systems commonly used in digital electronics are **Binary**, **Octal**, **Decimal**, and **Hexadecimal**. This section provides an introduction to these systems and their applications.

1. Binary System (Base 2)

The **binary number system** is the foundation of digital systems. It uses only two digits: 0 and 1. Each digit, known as a *bit*, represents a power of 2.

Key Features:

- Base (Radix): Binary is a base-2 system.
- Digits: Two possible digits: 0 (LOW) and 1 (HIGH).
- Applications: Used to represent all forms of data (numbers, characters, and instructions) in digital systems.

Positional Notation

Binary numbers use positional notation, where the value of each digit depends on its position and the base of the system. The general formula is:

$$N = \sum_i (\text{Digit} \times 2^i),$$

where i is the position of the digit (starting from 0 for the rightmost bit).

- Binary numbers are represented as sequences of bits, such as 1011_2 .

Binary numbers are often grouped for efficiency:

- **Nibble:** 4 bits.
- **Byte:** 8 bits.
- **Word:** Depends on the system, typically 16, 32, or 64 bits.

Example: Convert 1011_2 to Decimal

$$1011_2 = (1 \cdot 2^3) + (0 \cdot 2^2) + (1 \cdot 2^1) + (1 \cdot 2^0) = 11_{10}.$$

Example: Convert 101.01_2 to Decimal

$$\begin{aligned}(101.01)_2 &= 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} \\ &= 4 + 0 + 1 + 0 + 0.25 \\ &= 5.25_{10}.\end{aligned}$$

Binary is used in computers and digital circuits due to its simplicity and direct representation of *on* and *off* states in electronic systems.

2. Octal System (Base 8)

The **octal number system** uses eight digits: $0, 1, 2, \dots, 7$. It is often used as a shorthand representation for binary numbers, as each octal digit corresponds to a group of three binary digits.

- Octal numbers simplify the representation of long binary sequences.

- Each position in an octal number corresponds to a power of 8.

Example:

$$127_8 = (1 \cdot 8^2) + (2 \cdot 8^1) + (7 \cdot 8^0) = 87_{10}.$$

Binary to Octal Conversion: Group binary digits in sets of three (from right to left):

$$101110_2 = 101\ 110 = 56_8.$$

3. Decimal System (Base 10)

The **decimal number system** is the most familiar and widely used system in daily life. It uses ten digits: 0, 1, 2, ..., 9. Each position corresponds to a power of 10.

- Decimal numbers are easy for humans to read and understand.
- Digital systems often convert data between binary and decimal for user interaction.

Example:

$$573_{10} = (5 \cdot 10^2) + (7 \cdot 10^1) + (3 \cdot 10^0).$$

4. Hexadecimal system (Base 16)

The **hexadecimal number system** uses 16 symbols: 0, 1, 2, ..., 9, A, B, C, D, E, F, where A = 10, B = 11, and so on. It is commonly used in programming and memory addressing because each hexadecimal digit corresponds to a group of four binary digits.

- Hexadecimal numbers provide a compact representation of binary data.
- Each position in a hexadecimal number corresponds to a power of 16.

Example:

$$1A_{16} = (1 \cdot 16^1) + (10 \cdot 16^0) = 26_{10}.$$

Applications

- **Binary:** Used in computers, digital circuits, and communication systems.
- **Octal:** Simplifies binary representation in microcontrollers and file permissions.
- **Decimal:** The primary system for human-readable data.
- **Hexadecimal:** Used in programming, memory addressing, and debugging tools.

1.8 Exercises

1. Explain the main differences between digital and analog systems in terms of data representation, precision, and noise resistance. Provide one example for each.
2. Compare digital media (e.g., MP3, MP4) with analog media (e.g., cassette tapes, VHS). Highlight the role of digital systems in improving media quality and accessibility.

3. List three applications of digital systems in daily life and explain how digital systems enhance their functionality.
4. How do digital systems store large amounts of data efficiently? Discuss the role of binary representation in storage systems like hard drives and SSDs.
5. Why is hexadecimal used in digital systems, especially in programming and memory addressing? Provide one practical example.
6. How does the discrete nature of digital signals make them more immune to noise compared to analog signals? Use a communication system as an example.
7. What makes binary representation particularly well-suited for digital systems? Discuss its relationship with transistor-based circuits.
8. Calculate the duty cycle of a waveform with $t_W = 5 \text{ ms}$ and $T = 20 \text{ ms}$.
9. Draw a timing diagram for the following binary sequence: 110101.

Calculate these numbers in decimal

1. $(1101)_2$
2. $(10101)_2$
3. $(111000)_2$
4. $(1010111)_2$

5. $(116)_8$

6. $(54)_8$

7. $(25)_8$

8. $(2F)_{16}$

9. $(4B)_{16}$

10. $(9D)_{16}$

Number System Conversion and Arithmetic Operations

2.1 Conversion Methods

Number systems are the foundation of digital systems, allowing us to represent and manipulate data efficiently. The four primary number systems used in digital design are:

- **Decimal (Base-10):** Commonly used by humans, uses digits 0 – 9.
- **Binary (Base-2):** Used by computers, uses digits 0 and 1.
- **Octal (Base-8):** Compact representation of binary, uses digits 0 – 7.
- **Hexadecimal (Base-16):** Compact representation of binary, uses digits 0 – 9 and *A – F*.

This section details how to convert numbers between these systems.

Decimal to Binary Conversion

To convert decimal numbers to binary, repeatedly divide the decimal number by 2 and record the remainders.

Steps:

1. Divide the decimal number by 2.
2. Record the remainder (either 0 or 1).
3. Continue dividing the quotient by 2 until the quotient is 0.
4. Write the binary digits (remainders) in reverse order.

Example 1: Convert 25_{10} to binary.

1. $25 \div 2 = 12$ remainder 1
2. $12 \div 2 = 6$ remainder 0
3. $6 \div 2 = 3$ remainder 0
4. $3 \div 2 = 1$ remainder 1
5. $1 \div 2 = 0$ remainder 1

Binary equivalent: $25_{10} = 11001_2$.

Example 2: Convert 45_{10} to binary.

1. $45 \div 2 = 22$ remainder 1
2. $22 \div 2 = 11$ remainder 0
3. $11 \div 2 = 5$ remainder 1
4. $5 \div 2 = 2$ remainder 1
5. $2 \div 2 = 1$ remainder 0
6. $1 \div 2 = 0$ remainder 1

Binary equivalent: $45_{10} = 101101_2$.

Converting Fractional Decimal to Binary

Let us convert the fractional decimal number 10.625 into binary.

Step 1: Separate the Integer and Fractional Parts

The given number is 10.625.

- The **integer part** is 10.
- The **fractional part** is 0.625.

Step 2: Convert the Integer Part to Binary

To convert the integer 10 to binary, we repeatedly divide by 2 and record the remainders:

$$10 \div 2 = 5 \quad \text{remainder } 0$$

$$5 \div 2 = 2 \quad \text{remainder } 1$$

$$2 \div 2 = 1 \quad \text{remainder } 0$$

$$1 \div 2 = 0 \quad \text{remainder } 1$$

Reading the remainders from bottom to top, we get $10_{10} = 1010_2$.

Step 3: Convert the Fractional Part to Binary

To convert 0.625 to binary, we repeatedly multiply by 2 and record the integer parts:

$$0.625 \times 2 = 1.25 \quad \text{integer part: } 1 \quad \text{fractional part: } 0.25$$

$$0.25 \times 2 = 0.5 \quad \text{integer part: } 0 \quad \text{fractional part: } 0.5$$

$$0.5 \times 2 = 1.0 \quad \text{integer part: } 1 \quad \text{fractional part: } 0$$

The binary equivalent of 0.625 is 0.101_2 .

Step 4: Combine the Integer and Fractional Parts

Now, combine the integer part 1010_2 and the fractional part 0.101_2 using a binary point:

$$10.625_{10} = 1010.101_2$$

Step 5: Verify the Result

To verify, convert 1010.101_2 back to decimal:

$$\begin{aligned} &= 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} \\ &= 8 + 0 + 2 + 0 + 0.5 + 0 + 0.125 \\ &= 10.625 \end{aligned}$$

Thus, the binary equivalent of 10.625_{10} is verified to be 1010.101_2 .

Converting Fractional Decimal to Octal

Octal numbers (base-8) are compact representations of binary. Decimal numbers are converted to octal by dividing repeatedly by 8.

Steps:

1. Divide the decimal number by 8.
2. Record the remainder (digits 0 – 7).
3. Continue dividing the quotient by 8 until the quotient is 0.
4. Write the octal digits (remainders) in reverse order.

Example 1: Convert 456_{10} to octal.

1. $456 \div 8 = 57$ remainder 0
2. $57 \div 8 = 7$ remainder 1

3. $7 \div 8 = 0$ remainder 7

Octal equivalent: $456_{10} = 710_8$.

Example 2: Convert 125_{10} to octal.

1. $125 \div 8 = 15$ remainder 5

2. $15 \div 8 = 1$ remainder 7

3. $1 \div 8 = 0$ remainder 1

Octal equivalent: $125_{10} = 175_8$.

Converting Fractional Decimal to Octal

Let us convert the fractional decimal number 23.40625 into octal.

Step 1: Separate the Integer and Fractional Parts

The given number is 23.40625:

- The **integer part** is 23.
- The **fractional part** is 0.40625.

Step 2: Convert the Integer Part to Octal

To convert 23 into octal, use repeated division by 8:

$$23 \div 8 = 2 \quad \text{remainder } 7$$

$$2 \div 8 = 0 \quad \text{remainder } 2$$

Reading the remainders from bottom to top, we get:

$$\begin{array}{r|l} 23_{10} & = 27_8 \\ 37 & \quad 227 \end{array}$$

Step 3: Convert the Fractional Part to Octal

To convert 0.40625 into octal, use repeated multiplication by 8 and record the integer parts:

$$0.40625 \times 8 = 3.25 \quad \text{integer part: 3} \quad \text{fractional part: 0.25}$$

$$0.25 \times 8 = 2.0 \quad \text{integer part: 2} \quad \text{fractional part: 0.0}$$

Since the fractional part becomes 0, the conversion stops. The octal equivalent of 0.40625 is:

$$0.40625_{10} = 0.32_8$$

Step 4: Combine the Integer and Fractional Parts

Now, combine the integer part 27_8 and the fractional part 0.32_8 :

$$23.40625_{10} = 27.32_8$$

Step 5: Verify the Result

To verify, convert 27.32_8 back to decimal:

$$\begin{aligned} 27.32_8 &= (2 \cdot 8^1) + (7 \cdot 8^0) + (3 \cdot 8^{-1}) + (2 \cdot 8^{-2}) \\ &= 16 + 7 + 0.375 + 0.03125 \\ &= 23.40625_{10} \end{aligned}$$

Decimal to Hexadecimal Conversion

Hexadecimal numbers (base-16) are compact representations of binary and widely used in computing. Decimal numbers are converted to hexadecimal by dividing repeatedly by 16.

Steps:

1. Divide the decimal number by 16.
2. Record the remainder (digits 0 – 9 or $A - F$).
3. Continue dividing the quotient by 16 until the quotient is 0.
4. Write the hexadecimal digits (remainders) in reverse order.

Example 1: Convert 754_{10} to hexadecimal.

1. $754 \div 16 = 47$ remainder 2
2. $47 \div 16 = 2$ remainder 15 (F)
3. $2 \div 16 = 0$ remainder 2

Hexadecimal equivalent: $754_{10} = 2F2_{16}$.

Example 2: Convert 1984_{10} to hexadecimal.

1. $1984 \div 16 = 124$ remainder 0
2. $124 \div 16 = 7$ remainder 12 (C)
3. $7 \div 16 = 0$ remainder 7

Hexadecimal equivalent: $1984_{10} = 7C0_{16}$.

Converting Fractional Decimal to Hexadecimal

Let us convert the fractional decimal number 45.6875 into hexadecimal.

Step 1: Separate the Integer and Fractional Parts

The given number is 45.6875:

- The **integer part** is 45.
- The **fractional part** is 0.6875.

Step 2: Convert the Integer Part to Hexadecimal

To convert 45 into hexadecimal, use repeated division by 16:

$$45 \div 16 = 2 \quad \text{remainder } 13 \quad (\text{which is } D \text{ in hexadecimal})$$

$$2 \div 16 = 0 \quad \text{remainder } 2$$

Reading the remainders from bottom to top, we get:

$$45_{10} = 2D_{16}$$

Step 3: Convert the Fractional Part to Hexadecimal

To convert 0.6875 into hexadecimal, use repeated multiplication by 16 and record the integer parts:

$$0.6875 \times 16 = 11.0 \quad \text{integer part: } 11 \quad (\text{which is } B \text{ in hexadecimal})$$

Since the fractional part becomes 0, the conversion stops. The hexadecimal equivalent of 0.6875 is:

$$0.6875_{10} = 0.B_{16}$$

Step 4: Combine the Integer and Fractional Parts

Now, combine the integer part $2D_{16}$ and the fractional part $0.B_{16}$:

$$45.6875_{10} = 2D.B_{16}$$

$40 \quad | \quad 227$

Step 5: Verify the Result

To verify, convert $2D.B_{16}$ back to decimal:

$$\begin{aligned}2D.B_{16} &= (2 \cdot 16^1) + (13 \cdot 16^0) + (11 \cdot 16^{-1}) \\&= 32 + 13 + 0.6875 \\&= 45.6875_{10}\end{aligned}$$

Binary to Octal Conversion

Binary numbers are converted to octal by grouping binary digits in sets of 3 (starting from the right) and converting each group into its octal equivalent.

Steps:

1. Group binary digits in sets of 3 (from right to left).
2. Convert each group to octal.

Example 1: Convert 101110_2 to octal.

$$101110_2 = (101)(110) = 56 = 56_8.$$

Example 2: Convert 110101011_2 to octal.

$$110101011_2 = (110)(101)(011) = 653 = 653_8.$$

Binary to Hexadecimal Conversion

Binary numbers are converted to hexadecimal by grouping binary digits in sets of 4 (starting from the right) and converting each group into hexadecimal.

Steps:

1. Group binary digits in sets of 4 (from right to left).
2. Convert each group to hexadecimal.

Example 1: Convert 11011101_2 to hexadecimal.

$$11011101_2 = (1101)(1101) = D D = DD_{16}.$$

Example 2: Convert 1011110001_2 to hexadecimal.

$$1011110001_2 = (0010)(1111)(0001) = 2 f 1 = 2f1_{16}.$$

Octal to Hexadecimal Conversion

Octal numbers are converted to hexadecimal by first converting to binary and then grouping binary digits in sets of 4.

Steps:

1. Convert octal digits to binary (3 bits per digit).
2. Group binary digits in sets of 4 (from right to left).
3. Convert each group to hexadecimal.

Example 1: Convert 765_8 to hexadecimal.

1. Convert 765_8 to binary:

$$7_8 = 111_2, \quad 6_8 = 110_2, \quad 5_8 = 101_2.$$

$$\text{Binary: } 765_8 = 111110101_2.$$

2. Group binary digits in sets of 4:

$$111110101_2 = (0001)(1111)(0101) = 1F5_{16}.$$

Example 2: Convert 123_8 to hexadecimal.

1. Convert 123_8 to binary:

$$1_8 = 001_2, \quad 2_8 = 010_2, \quad 3_8 = 011_2.$$

$$\text{Binary: } 123_8 = 001010011_2.$$

2. Group binary digits in sets of 4:

$$001010011_2 = (0000)(0101)(0011) = 53_{16}.$$

2.2 Complements

Complements are essential in digital systems for simplifying arithmetic operations, particularly subtraction. They are used in various base- r systems, such as decimal and binary. There are two types of complements:

1. **Radix Complement (r 's Complement).**
2. **Diminished Radix Complement ($(r - 1)$'s Complement).**

Diminished Radix Complement ($(r - 1)$'s Complement)

The diminished radix complement of a number N in base r with n digits is defined as:

$$(r^n - 1) - N$$

This complement can be calculated digit-by-digit.

Example (Decimal System): For a 6-digit decimal number:

$$9\text{'s complement of } 546700 = (10^6 - 1) - 546700 = 999999 - 546700 = 453299$$

Example (Binary System): For a 7-bit binary number:

$$1\text{'s complement of } 1011000 = (2^7 - 1) - 1011000 = 1111111 - 1011000 = 0100111$$

Observation:

- Subtraction from $(r^n - 1)$ never requires a borrow.
- The complement can be computed by inverting all the bits:

$$- 1 - 0 = 1$$

$$- 1 - 1 = 0$$

Properties of $(r - 1)$'s Complement

- All 0s in the number become 1s.
- All 1s in the number become 0s.

Example:

Given $(11001010)_2$, the 1's complement is $(00110101)_2$.

Radix Complement (*r*'s Complement)

The radix complement is obtained by adding 1 to the diminished radix complement:

$$r\text{'s complement of } N = [(r^n - 1) - N] + 1 = r^n - N.$$

Example (Decimal System): For a 6-digit decimal number:

$$10\text{'s complement of } 012398 = (10^6 - 012398) = 987602.$$

Example (Binary System): For a 7-bit binary number:

$$2\text{'s complement of } 1101100 = 0010100.$$

Steps to Find the r 's Complement:

1. Find the $(r - 1)$'s complement.
2. Add 1 to the result.

Example:

Given 11001010_2 :

- 1's complement: 00110101_2 .
- Add 1: $00110101_2 + 1 = 00110110_2$.

Subtraction Using Complements

Subtraction of two n -digit unsigned numbers $M - N$ in base r can be performed as:

1. Add M to the r 's complement of N .
2. If there is an end carry, discard it. The result is positive.
3. If there is no end carry, take the r 's complement of the result and add a negative sign.

Example 1: Subtraction Using 10's Complement Perform $72532 - 3250$:

$$10\text{'s complement of } 3250 = 99999 - 3250 = 96750.$$

$$72532 + 96750 = 169282.$$

Discard the end carry (100000):

$$\text{Result: } 69282.$$

Example 2: Subtraction Without End Carry Perform $3250 - 72532$:

$$10\text{'s complement of } 72532 = 99999 - 72532 = 27468.$$

$$3250 + 27468 = 30718.$$

No end carry:

$$\text{Result: } -(10\text{'s complement of } 30718) = -69282.$$

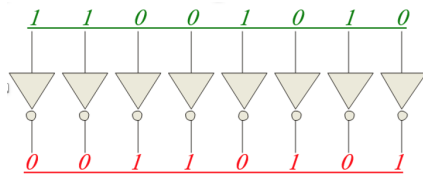


Figure 2.1: 1's Complement Conversion

One's Complement Representation One's complement of a binary number is obtained by flipping all the bits (changing 0 to 1 and 1 to 0).

Steps to Compute One's Complement:

1. Write the binary number.
2. Invert each bit.

Example 1: Find the one's complement of 1010_2 .

Original: 1010_2 One's Complement: 0101_2 .

Example 2: Find the one's complement of 110011_2 .

Original: 110011_2 One's Complement: 001100_2 .

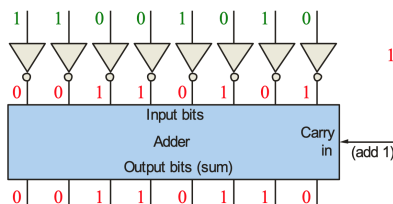


Figure 2.2: 2's Complement Conversion

Two's Complement Representation

Two's complement is widely used in digital systems for representing signed numbers because it simplifies arithmetic operations. It is computed by adding 1 to the one's complement of the number.

Steps to Compute Two's Complement:

1. Compute the one's complement of the binary number.
2. Add 1 to the result.

Example 1: Find the two's complement of 1010_2 .

1. Compute the one's complement:

$$1010_2 \rightarrow 0101_2.$$

2. Add 1 to the one's complement:

$$\begin{array}{r} 1 \\ 0101 \\ + 1 \\ \hline 0110 \end{array}$$

Two's Complement: 0110_2 .

Example 2: Find the two's complement of 110011_2 .

1. Compute the one's complement:

$$110011_2 \rightarrow 001100_2.$$

2. Add 1 to the one's complement:

$$\begin{array}{r} 001100 \\ + 1 \\ \hline 001101 \end{array}$$

Two's Complement: 001101_2 .

2.3 Signed and Complements of Binary Numbers

Binary numbers can represent both positive and negative values using different techniques. These methods include:

- **Sign-Magnitude Representation**
- **One's Complement Representation**
- **Two's Complement Representation**

Sign-Magnitude Representation

In sign-magnitude representation:

- The Most Significant Bit (MSB) represents the sign:
 - 0: Positive number
 - 1: Negative number
- The remaining bits represent the magnitude of the number.

Example 1: Represent $+5$ and -5 in a 4-bit system.

- $+5_{10}$: $0\ 101_2$ (Sign bit 0, magnitude 101_2).
- -5_{10} : $1\ 101_2$ (Sign bit 1, magnitude 101_2).

Example 2: Represent $+7$ and -7 in a 5-bit system.

- $+7_{10}$: $0\ 0111_2$ (Sign bit 0, magnitude 0111_2).
- -7_{10} : $1\ 0111_2$ (Sign bit 1, magnitude 0111_2).

Range of Signed Binary Numbers

The range of signed binary numbers in a system depends on the number of bits (n):

Sign-Magnitude and One's Complement:

- In the **1's complement form**, a negative number is the **1's complement** of the corresponding positive number.
- For example, using **eight bits**, the decimal number -25 is expressed as the **1's complement** of $+25$ (00011001) as:

11100110

Sign-Magnitude and Two's Complement:

- In the **2's complement form**, a negative number is the **2's complement** of the corresponding positive number.
- Again, using **eight bits**, let's take the decimal number $+25$ and express it as the **2's complement** of $+25$ (00011001). Inverting each bit and adding 1, you get:

11100111

Example: For a 4-bit system

- **Sign-Magnitude and One's Complement:**

-7 to $+7$

- **Two's Complement:**

-8 to $+7$

51 | 227

Example: Representing -39 in Various 8-Bit Forms

Express the decimal number -39 as an 8-bit number in the sign-magnitude, 1's complement, and 2's complement forms.

Solution

Step 1: Write the 8-bit number for $+39$:

00100111

Step 2: Sign-Magnitude Form

In the **sign-magnitude form**, -39 is produced by changing the sign bit to 1 and leaving the magnitude bits as they are. The number is:

10100111

Step 3: 1's Complement Form

In the **1's complement form**, -39 is produced by taking the 1's complement of $+39$ (00100111). This is done by inverting all the bits:

11011000

Step 4: 2's Complement Form

In the **2's complement form**, -39 is produced by taking the 2's complement of $+39$ (00100111) as follows:

$$\begin{array}{r}
 11011000 \text{ 1's complement} \\
 +1 \\
 \hline
 11011001 \text{ 2's complement}
 \end{array}$$

2.4 Arithmetic operations

Arithmetic operations in binary systems include addition, subtraction, multiplication, and division. These operations are performed using rules similar to decimal arithmetic but are adapted for the binary number system.

Binary Addition

Binary addition is performed using the following rules:

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10$ (carry 1)

1. Adding Two Binary Positive Numbers:

Example 1: Add $1011_2 + 1101_2$.

a) Align the numbers:

$$1011_2 + 1101_2.$$

b) Perform addition column by column:

$$\begin{array}{r} 111 \\ 1011 \\ +1101 \\ \hline 11000 \end{array}$$

c) The result is 11000_2 .

Example 2: Add $1110_2 + 1011_2$.

a) Align the numbers:

$$1110_2 + 1011_2.$$

b) Perform addition column by column:

$$\begin{array}{r} 11 \\ 1110 \\ + 1011 \\ \hline 11001 \end{array}$$

c) The result is 11001_2 .

2. Adding Positive Number with Magnitude Larger than Negative Number

$$\begin{array}{r} 00001111 \quad 15 \\ + 11111010 \quad -6 \\ \hline 100001001 \quad 9 \end{array}$$

Discard carry $\rightarrow 1$.

The final carry bit is discarded. The sum is positive and therefore in true (uncomplemented) binary.

Addition of a positive number and a smaller negative number yields a positive number.

3. Adding Negative Number with Magnitude Larger than Positive Number

$$\begin{array}{r}
 00010000 \quad 16 \\
 + 11101000 \quad -24 \\
 \hline
 11111000 \quad -8
 \end{array}$$

The sum is negative and therefore in 2's complement form.

Addition of a positive number and a larger negative number yields a negative number in 2's complement.

4. Adding Both Numbers Negative

$$\begin{array}{r}
 11111011 \quad -5 \\
 + 11110111 \quad -9 \\
 \hline
 111110010 \quad -14
 \end{array}$$

Discard carry $\rightarrow 1$.

The final carry bit is discarded. The sum is negative and therefore in 2's complement form.

Addition of two negative numbers yields a negative number in 2's complement.

Binary Subtraction

Binary subtraction uses the following rules:

- $0 - 0 = 0$

- $1 - 0 = 1$
- $1 - 1 = 0$
- $0 - 1 = 1$ (borrow 1 from the next column)

Example 1: Subtract $1010_2 - 1001_2$.

1. Align the numbers:

$$1010_2 - 1001_2.$$

2. Perform subtraction column by column:

$$\begin{array}{r} 0 \\ 1010 \\ -1001 \\ \hline 0001 \end{array}$$

3. The result is 0001_2 or 1_{10} .

Example 2: Subtract $1100_2 - 1011_2$.

1. Align the numbers:

$$1100_2 - 1011_2.$$

2. Perform subtraction column by column:

$$\begin{array}{r} 01 \\ 1100 \\ -1011 \\ \hline 0001 \end{array}$$

3. The result is 0001_2 or 1_{10} .

Subtraction Using Two's Complement

To subtract two signed numbers, take the 2's complement of the subtrahend and add. Discard any final carry bit.

Steps for Subtraction:

1. Take the two's complement of the subtrahend (number to subtract).
2. Add it to the minuend (number to subtract from).

Example: Compute $5_{10} - 3_{10}$ in 4-bit binary.

1. Represent 5_{10} and 3_{10} in binary:

$$5_{10} = 0101_2, \quad 3_{10} = 0011_2.$$

2. Compute the two's complement of 3_{10} :

- One's complement of 3_{10} : $0011_2 \rightarrow 1100_2$.
- Add 1: $1100_2 + 1 = 1101_2$.

3. Add the result to 5_{10} :

$$\begin{array}{r} \textcolor{blue}{1} \textcolor{red}{1} \textcolor{red}{1} \\ 0101 \\ +1101 \\ \hline 0010 \end{array}$$

$$0101_2 + 1101_2 = \textcolor{blue}{1}0010_2.$$

4. Discard the carry bit $\textcolor{blue}{1}$:

$$\text{Result: } 0010_2 = 2_{10}.$$

Binary Multiplication

Binary multiplication is similar to decimal multiplication and uses the following rules:

- $0 \cdot 0 = 0$
- $1 \cdot 0 = 0$
- $0 \cdot 1 = 0$
- $1 \cdot 1 = 1$

Steps for Multiplication:

1. Multiply each bit of the second number with the entire first number.
2. Shift the result one position to the left for each successive bit.
3. Add the partial products.

Example: Multiply $101_2 \cdot 11_2$.

1. Multiply 101_2 by the rightmost bit of 11_2 (1):

$$101_2 \cdot 1$$

2. Multiply 101_2 by the next bit of 11_2 (1) and shift left by one position:

$$1010_2 \cdot 1$$

3. Add the results:

$$\begin{array}{r}
 101 \\
 * 11 \\
 \hline
 101 \\
 +1010 \\
 \hline
 1111_2
 \end{array}$$

Result: 1111_2 or 15_{10} .

Binary Division

Binary division follows a process similar to long division in decimal arithmetic. The steps include:

1. Compare the divisor with the current portion of the dividend.
2. Write 1 in the quotient if the divisor fits, otherwise write 0.
3. Subtract the divisor and bring down the next bit of the dividend.
4. Repeat until all bits are processed.

Rules for Binary division:

1. $0 \div 0 = \infty$
2. $0 \div 1 = 0$
3. $1 \div 1 = 0$
4. $1 \div 1 = \infty$

Example: Divide $1011_2 \div 10_2$.

1. Divide the first two bits of 1011_2 by 10_2 : Quotient: 1, Remainder: 01_2 .
2. Bring down the next bit of the dividend: 011_2 .
3. Divide $011_2 \div 10_2$: Quotient: 1, Remainder: 01_2 .
4. Bring down the final bit of the dividend: 011_2 .
5. Divide $011_2 \div 10_2$: Quotient: 1, Remainder: 01_2 .

Result: Quotient 101_2 , Remainder 1_2 .

Example : Perform the following binary divisions

1. $110_2 \div 11_2$

2. $110_2 \div 10_2$

$ \begin{array}{r} \textcolor{red}{10} \\ \hline 11 \overline{) 110} \\ \underline{- 11} \\ 000 \end{array} $	Quotient Remainder
---	---

$ \begin{array}{r} \textcolor{red}{11} \\ \hline 10 \overline{) 111} \\ \underline{- 10} \\ 10 \\ \underline{- 10} \\ 000 \end{array} $	Quotient Remainder
---	---

Overflow in Binary Arithmetic

Overflow occurs in binary addition or subtraction when the result exceeds the range representable by the given number of bits.

Example: Compute $10_{10} + 13_{10}$ in 4-bit binary.

1. Represent 10_{10} and 13_{10} in binary:

$$10_{10} = 1010_2, \quad 13_{10} = 1101_2.$$

2. Add the two numbers:

$$\begin{array}{r} 1010 \\ +1101 \\ \hline 10111 \end{array}$$

3. The result (10111_2) requires 5 bits, but only 4 bits are available. Overflow has occurred.

2.5 Codes in Digital Systems

Codes in digital systems are methods to represent data, such as numbers, alphabets, or special characters, in binary form for efficient processing, storage, and communication. Below are the most common codes used in digital systems.

1. Binary-Coded Decimal (BCD)

Definition: Binary-Coded Decimal (BCD) is a binary representation of decimal numbers where each decimal digit is represented by its 4-bit binary equivalent.

Advantages:

- Easy to convert between decimal and BCD.
- Simplifies hardware design for digital displays like calculators.

Example: Convert 59_{10} to BCD:

1. Separate the digits: 5 and 9.
2. Represent each digit in binary:

$$5_{10} = 0101_2, \quad 9_{10} = 1001_2.$$

3. Combine the binary equivalents:

$$59_{10} = 0101\ 1001 \text{ (BCD)}.$$

2. Gray Code

Definition: Gray Code is a binary coding system where only one bit changes between consecutive numbers. This property minimizes errors in digital communication systems.

Advantages:

- Reduce errors in analog-to-digital conversions.
- Useful in applications like rotary encoders.

Steps to Convert Binary to Gray Code:

1. The most significant bit (MSB) of the Gray Code is the same as the MSB of the binary number.

2. Each subsequent Gray Code bit is obtained by XORing the previous binary bit with the current binary bit.

Example: Convert 1101_2 to Gray Code:

Write the MSB: 1.

XOR each binary bit with the previous bit:

$$1 \oplus 1 = 0, \quad 1 \oplus 0 = 1, \quad 0 \oplus 1 = 1.$$

Gray Code: $1101_2 = 1011_2$.

3. ASCII (American Standard Code for Information Interchange)

Definition: ASCII is a 7-bit (or extended 8-bit) code used to represent characters, including letters, numbers, and symbols.

Applications:

- Text representation in computers and communication systems.

Examples:

Character A: ASCII $65_{10} = 1000001_2$.

Character Z: ASCII $90_{10} = 1011010_2$.

Character a: ASCII $97_{10} = 1100001_2$.

4. Parity Codes

Definition: A parity bit is an additional bit added to binary data to detect errors during transmission.

Types:

- **Even Parity:** The total number of 1s (including the parity bit) is even.
- **Odd Parity:** The total number of 1s (including the parity bit) is odd.

Example:

- Data: 1010_2
- Even Parity: Add 0 (total 1s remain even): 10100_2 .
- Odd Parity: Add 1 (total 1s become odd): 10101_2 .

6. Excess-3 Code

Definition: Excess-3 is a self-complementary binary-coded decimal system where each decimal digit is represented by its 4-bit binary value plus 3.

Applications:

- Error detection and simplification of complement operations.

Example: Convert 4_{10} to Excess-3:

1. Convert 4_{10} to binary:

$$4_{10} = 0100_2.$$

2. Add $3_{10} = 0011_2$:

$$0100_2 + 0011_2 = 0111_2.$$

3. Excess-3 Code:

$$\begin{array}{c|c} & 0111_2. \\ 64 & 227 \end{array}$$

Comparison of Codes

Code Type	Purpose	Application
Binary-Coded Decimal (BCD)	Represent decimal numbers in binary	Digital displays, calculators
Gray Code	Reduce errors in transitions	Rotary encoders, ADCs
ASCII	Represent characters	Text processing, communication systems
Parity Codes	Error detection	Data transmission
Excess-3 Code	Simplify arithmetic operations	Digital arithmetic

2.6 Binary Storage and Register:

What is Binary Storage?

Binary storage refers to a storage technique in which information is represented and stored in binary form, i.e., using 0s and 1s. Binary storage is a crucial concept in digital electronics, as all digital systems operate using binary logic.

Binary storage can be implemented using various types of storage devices such as semiconductor circuits, magnetic tapes, optical disks, etc. Overall, binary storage forms the foundation for storing and manipulating digital information.

Binary Storage Technologies

Depending on the characteristics and applications, there are various types of binary storage technologies. The most commonly used binary storage technologies are explained below:

Semiconductor Electronics Circuits

Semiconductor storage devices are made up of electronic components like transistors. These storage devices store binary information. A transistor or any other electronic component has two states, namely ON and OFF, that represent binary 1 and 0, respectively. Hence, semiconductor memory devices are widely used for binary storage.

Magnetic Media

Magnetic media such as magnetic tapes, hard disks, etc., can also be used for binary storage. This technology utilizes magnetic materials to store binary information. In magnetic storage devices, the binary state (either 0 or 1) is represented by magnetic field polarity. One magnetic polarity represents binary 1, while the opposite polarity represents binary 0. Magnetic memory devices are widely used for the permanent storage of digital information.

Optical Disks

Optical storage devices are made up of light-sensitive materials and plastic. These materials store information in binary form. A laser is used to read and write data on optical storage devices. Variations in the reflected laser light represent binary values 0 and 1.

Overall, binary storage is the most fundamental process in any digital electronic system, allowing for efficient and reliable storage of data and

information.

Binary Storage Operations

Binary storage involves two main operations:

- **Write (Input):** The process of inputting binary data into a storage device.
- **Read (Output):** The process of retrieving binary data from a storage device.

Thus, in a binary storage system, the **read** and **write** operations are essential for data transfer in digital systems, enabling the exchange of binary information among different digital devices.

What is a Binary Storage Register?

In a digital electronic system, a **binary storage register** is a fundamental storage unit used to store and process binary data. A binary storage register consists of a collection of flip-flops, where each flip-flop stores one bit of information in binary form.

A binary storage register is the most elementary block of a digital system's memory unit and plays a vital role in data storage and processing operations.

The size or storage capacity of a binary storage register is determined by the number of flip-flops used in the logic circuit. For example:

- A register consisting of 8 flip-flops has a storage capacity of 8 bits.
- A register consisting of 16 flip-flops has a storage capacity of 16 bits, and so on.

The block diagram of a 4-bit binary storage register is illustrated in the following figure:

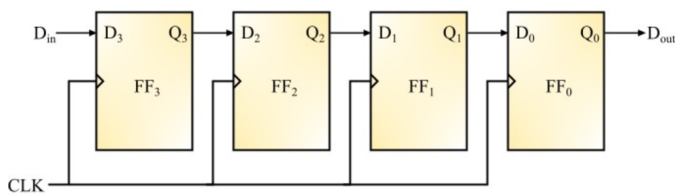


Figure 2.3: 4-Bit Binary Storage Register

The most important point to note about the binary storage register is that it stores data temporarily. Registers are mainly used to store small piece of binary data which are required during processing and executing of instructions.

In a binary storage register, the read and write operations are executed through input and output data lines, where the binary data is written into the register through data input lines and the stored data is retrieved from the register through the data output lines.

Binary storage registers are very important components of a digital system, as they play a crucial roles in execution of various operations. They provide a temporary space to store binary data and instructions required for processing and other computation activities.

Types of Binary Storage Registers

Some common types of binary storage registers used in various digital systems are listed below:

- **Serial-In Serial-Out (SISO) Register:**

A type of binary storage register that inputs and outputs binary data in a serial fashion, i.e., one bit of binary data at a time is termed as a serial-in serial-out binary register. This type of binary storage register consists of a single data input line and a single data output line.

- **Serial-In Parallel-Out (SIPO) Register:**

This type of binary storage register accepts data in a serial manner (one bit at a time) and outputs data in a parallel manner (all bits at a time). It has only one data input line but output lines equal to the number of bits.

- **Parallel-In Serial-Out (PISO) Register:**

In this binary register, the binary data is input in a parallel manner and output in a serial manner. Hence, this binary storage register consists of multiple input lines but only one output line.

- **Parallel-In Parallel-Out (PIPO) Register:**

A parallel-in parallel-out register consists of multiple input and output lines for storage and retrieval of all bits of binary data at the same time.

2.7 Exercises

Do the following conversion problems

1. $(25)_{10}$, $(63)_{10}$, $(125)_{10}$, $(50)_{10}$, $(255)_{10}$ convert these values into Binary, Octal and Hexadecimal.
2. Converting Fractional Decimal Numbers

Part A: Convert to Binary

Convert the following fractional decimal numbers into binary:

- a) 5.25
- b) 12.625
- c) 0.40625
- d) 3.1416 (to 4 binary places)
- e) 18.375

Part B: Convert to Octal

Convert the following fractional decimal numbers into octal:

- a) 10.125
- b) 45.6875
- c) 0.15625
- d) 7.8125
- e) 32.9375

Part C: Convert to Hexadecimal

Convert the following fractional decimal numbers into hexadecimal:

- a) 14.375
- b) 0.0625
- c) 25.8125
- d) 50.5
- e) 100.125

Part D: Mixed Conversions

For the following fractional decimal numbers, convert them into binary, octal, and hexadecimal:

a) 8.5

b) 19.75

c) 0.3125

d) 65.5625

e) 123.125

3. Add the following binary numbers:

(a) $10111 + 01101$

(b) $1101 + 1010$

(c) $110 + 100$

(d) $111 + 11$

(e) $111 + 101$

4. Perform the following binary subtractions:

(a) $1101 - 0100$

(b) $1001 - 0111$

(c) $10011 - 10010$

(d) $110000 - 1111$

5. Perform the following binary multiplications:

- (a) $11 * 10$
(b) $101 * 11$
(c) $111 * 110$
(d) $1100 * 101$
(e) $1110 * 1110$
6. Divide the binary numbers as indicated:
- (a) $110 \div 11$
(b) $1010 \div 10$
(c) $1111 \div 101$
7. Obtain the 1's and 2's complements of the following binary numbers:
- (a) 00010000
(b) 11011010
(c) 10000101
(d) 10101010
(e) 11111111
8. Express each decimal number in binary as an 8-bit sign-magnitude number:
- (a) +29 (b) -85 (c) +100 (d) -123
9. Convert each hexadecimal number to binary
- (a) 10A4 (b) CF8E (c) 9742
(d) ABCD (e) B4 (f) 54

10. Convert each binary number to hexadecimal:
(a) 1111 (b) 1011 (c) 11111
(d) 10101010 (e) 10101100 (f) 10111011
11. Convert each octal number into binary:
(a) 17 (b) 26 (c) 145

(d) 456 (e) 653 (f) 777
12. Convert the following decimal numbers to BCD:
(a) 104 (b) 128 (c) 132

(d) 150 (e) 547 (f) 359
13. Convert each binary number to Gray code:
(a) 11011 (b) 1001010 (c) 1111011101110
14. Determine which of the following even parity codes are in error:
(a) 100110010 (b) 011101010 (c) 10111111010001010
15. Determine which of the following odd parity codes are in error:
(a) 11110110 (b) 00110001 (c) 01010101010101010

Chapter 3

Logic Gates

3.1 What are Logic Gates?

A logic gate is a digital component designed to either permit or block an electrical signal based on certain conditions. When these logical conditions are satisfied, the signal will pass; otherwise, it is halted. Each type of gate enforces its own specific logic rules for allowing or preventing the flow of current. In computing systems, the symbols 0 and 1 indicate two possible states of a circuit or device.

Truth Table

A truth table lays out every possible set of inputs to a logic gate along with the corresponding output. This allows us to see exactly how the gate will behave for all input combinations.

The following logic gates will be treated:

- OR
- AND

- NOT
- XOR
- NOR
- NAND
- XNOR

Logic gates operate based on a logical algebraic system known as **Boolean algebra**, where variables assume only one of two values: 0 or 1.

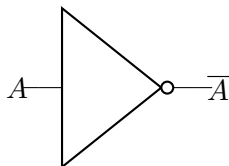
3.2 The NOT Gate (Inverter)

Definition: The NOT gate has one input and one output. The output is the logical complement of the input. It is also called an *inverter*.

The logical symbol for inversion or complementation is $\overline{A}$.

The logical operation of a NOT gate is summarized below:

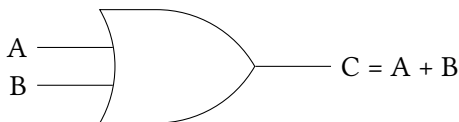
A	$C = \overline{A}$
0	1
1	0



3.3 The OR Gate

Definition: Produces an output of 1 when any or all inputs are 1. It is also called an *any-or-all gate*.

Diagram



Truth Table

A	B	C = A + B
0	0	0
0	1	1
1	0	1
1	1	1

Example

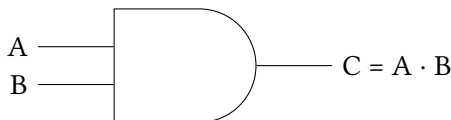
For the lamp to glow, either switch *A*, *B*, or both must be closed.

Boolean Equation: $C = A + B$

3.4 The AND Gate

Definition: Produces an output of 1 only when all inputs are present 1. Also known as an *all-or-nothing gate*.

Diagram



Truth Table

A	B	$C = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Example

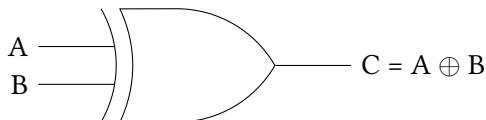
The lamp will glow only when both switches A and B are closed.

Boolean Equation: $C = A \cdot B$

3.5 The XOR Gate

Definition: Outputs 1 only if both inputs are different. This excludes cases where the inputs are the same. It is also referred to as an *inequality comparator*.

Diagram



Truth Table

A	B	$C = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

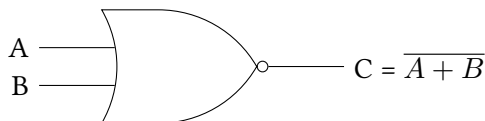
Example

XOR gates are used in digital circuits to compare two binary numbers and detect differences.

3.6 The NOR Gate

Definition: The NOR gate is a combination of an OR gate followed by a NOT gate. It produces an output of 1 only when all inputs are 0.

Diagram



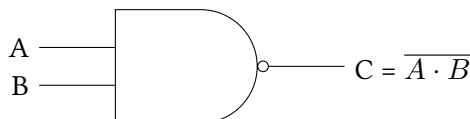
Truth Table

A	B	$C = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

3.7 The NAND Gate

Definition: The NAND gate is a combination of an AND gate followed by a NOT gate. It produces an output of 1 unless all inputs are 1.

Diagram



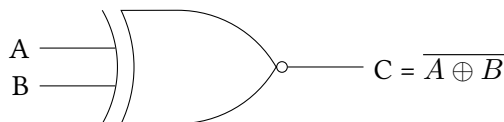
Truth Table

A	B	$C = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

3.8 The XNOR Gate

Definition: The XNOR gate is a combination of an XOR gate followed by a NOT gate. It produces an output of 1 when both inputs are the same.

Diagram



Truth Table

A	B	$C = \overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1

3.9 Universal Gates

Universal gates are a class of logic gates from which any other logic gate can be implemented. The two primary universal gates are the NAND gate and the NOR gate. These gates are significant in digital logic design because of their versatility.

Why Universal Gates?

Using only NAND or NOR gates, it is possible to construct any other basic logic gate (AND, OR, NOT, XOR, etc.). This makes them fundamental in digital circuit design.

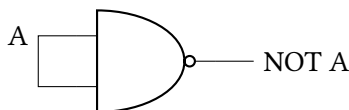


Figure 3.1: NOT Gate Using a NAND Gate

As shown in Figures below that the versatility of NAND gates makes them a powerful tool for digital circuit design.

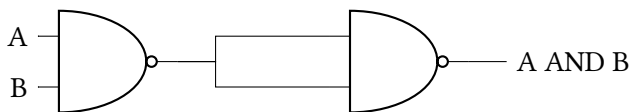


Figure 3.2: AND Gate Using NAND Gates

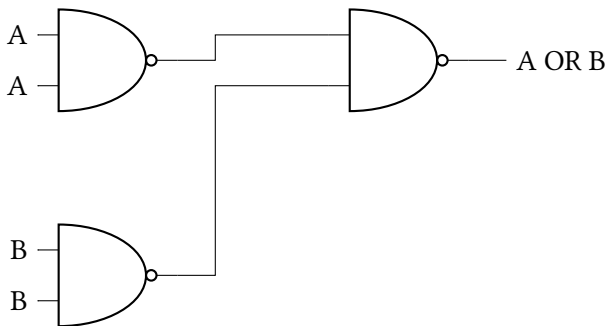


Figure 3.3: OR Gate Using NAND Gates

Explanation

1. NOT Gate:

- Uses one NAND gate where both inputs are tied together, producing $\overline{A}$.

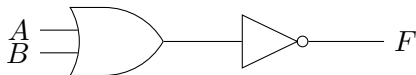
2. AND Gate:

- Two NAND gates: The first computes $\overline{A \cdot B}$, and the second inverts it to get $A \cdot B$.

3. OR Gate:

- Three NAND gates: The first two invert A and B , and the third computes $\overline{\overline{A} \cdot \overline{B}}$, which is $A + B$.

Circuit Diagram for NOR Gate

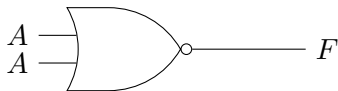


Implementing Gates Using NOR Gate

1. NOT Gate:

$$F = \overline{A}$$

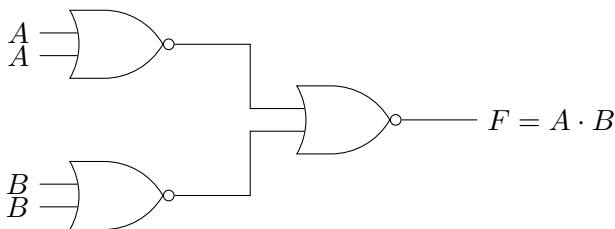
Circuit:



2. AND Gate:

$$F = A \cdot B$$

Circuit:

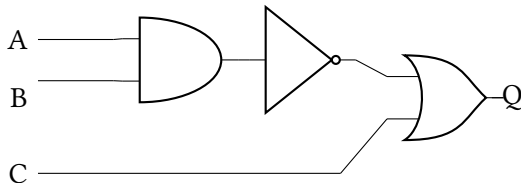


Usage of Universal Gates:

1. Microprocessors and ALUs: Universal gates are used in arithmetic logic units and CPUs.
2. Standardized Circuit Design: Using only NAND or NOR gates simplifies manufacturing.
3. Error Detection: Logic gates designed using universal gates are used in parity checkers and error detection circuits.

3.10 Exercises

- Derive the logical output X for the following circuit when:
 - $A = 1, B = 0, C = 1$
 - $A = 0, B = 1, C = 1$
- Design a circuit using an AND gate and an OR gate such that the output is 1 only if either:
 - Both inputs of the AND gate are 1, or
 - At least one input of the OR gate is 1.
- Two signals, 1100111 and 1010101, are applied to a 2-input AND gate. Sketch the resulting output signal.
- Given three signals $A = 11011$, $B = 01101$, and $C = 10100$, combine the signals using an XOR gate, followed by a NOR gate, and determine the final output.
- Look at the Figure below then answer the questions:

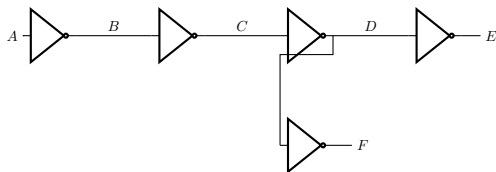


- (a) Complete the truth table below for the logic circuit shown in Figure above. Write the correct value of the output Q for each of the listed sets of inputs.

Input A	Input B	Input C	Output Q
1	0	1	
0	1	0	
0	1	1	

- (b) Two of the gates in the circuit shown in above Figure could be replaced by a single gate.
- Which **two gates** could be replaced?
 - What **single gate** would be used instead?
- (c) Why is it an advantage to use as few gates as possible in a logic circuit?

6. A combination of inverters is shown in Figure, If a LOW(0) is applied to point A, determine the net output at points E and F.



7. Examine the conditions indicated in Figure, and identify the faulty gates.



8. Complete the truth table for $F = \overline{A} \cdot B + A \cdot \overline{B}$.

A	B	$F = \overline{A} \cdot B + A \cdot \overline{B}$
0	0	
0	1	
1	0	
1	1	

9. Create a custom gate $G = \overline{A \cdot B} + C$. Draw the gate and derive the truth table.
10. Draw the circuit for $F = (A + B) \cdot (\overline{C} + D)$.
11. Design a logic circuit for a vending machine that dispenses a product when either Coin A and Coin B are inserted or Coin C alone is inserted.

Chapter 4

Boolean Algebra and Simplification

Boolean algebra is a mathematical system used to represent logical operations in digital systems. It forms the backbone of digital logic design, where binary variables represent the logical states of a circuit: 1 (TRUE or HIGH) and 0 (FALSE or LOW).

4.1 Boolean Expressions

Boolean expressions represent logical relationships between variables and operations in Boolean algebra. These expressions are fundamental in digital logic design and analysis. They consist of binary variables ($A, B, C, \dots$), constants (0, 1), and logical operators ($\cdot, +, \overline{}$).

Example 1: Simple Expression

Given:

$$Y = A \cdot (B + \overline{C})$$

- $B + \overline{C}$: OR operation between B and NOT C .

- $A \cdot (B + \overline{C})$: AND operation between A and the result.

Example 2: Complex Expression

Simplify the expression:

$$Y = A \cdot B + A \cdot \overline{B} + \overline{A} \cdot B$$

Steps:

1. Factor A :

$$Y = A \cdot (B + \overline{B}) + \overline{A} \cdot B$$

2. Apply the Complement Law ($B + \overline{B} = 1$):

$$Y = A + \overline{A} \cdot B$$

3. Apply the Complement Law again ($A + \overline{A} = 1$):

$$Y = A + B$$

4.2 Logic Gates for Boolean Operations

- **AND Gate:** Implements $A \cdot B$.
- **OR Gate:** Implements $A + B$.
- **NOT Gate:** Implements $\overline{A}$.
- **NAND Gate:** Implements $\overline{A \cdot B}$.
- **NOR Gate:** Implements $\overline{A + B}$.
- **XOR Gate:** Implements $A \oplus B$.
- **XNOR Gate:** Implements $\overline{A \oplus B}$.

4.3 Truth Tables for Multiple Variables

The total number of possible combinations of binary inputs to a gate is determined by the following formula:

$$N = 2^n \quad ()$$

where N is the number of possible input combinations and n is the number of input variables. To illustrate,

- For two input variables: $N = 2^2 = 4$ combinations
- For three input variables: $N = 2^3 = 8$ combinations
- For four input variables: $N = 2^4 = 16$ combinations

For a 3-variable expression (A, B, C), the truth table has $2^3 = 8$ rows. Example for $Y = A \cdot \overline{B} + C$:

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Simplification Example

Simplify:

$$Y = A \cdot B + A \cdot \overline{B} + \overline{A} \cdot B$$

Steps:

1. Factor A :

$$Y = A \cdot (B + \overline{B}) + \overline{A} \cdot B$$

2. Apply the Complement Law ($B + \overline{B} = 1$):

$$Y = A + \overline{A} \cdot B$$

3. Apply the Complement Law again ($A + \overline{A} = 1$):

$$Y = A + B$$

Applications of Boolean Expressions

- **Digital Circuit Design:** Boolean expressions represent the logic of digital circuits like multiplexers, decoders, and adders.
- **Programming Logic:** Boolean algebra is used in programming for decision-making and conditional statements.
- **Error Checking:** Boolean expressions verify the correctness of digital systems.

4.4 Laws and Rules of Boolean Algebra

Boolean algebra follows specific laws and rules that simplify logical expressions and aid in the design of digital circuits. Below are detailed examples for each law and rule.

1. Identity Law

- AND: $A \cdot 1 = A$
- OR: $A + 0 = A$

Examples:

1. $Y = A \cdot 1 + 0$

Using Identity Law: $Y = A + 0 = A$

2. $Y = (A + 0) \cdot 1$

Using Identity Law: $Y = A \cdot 1 = A$

3. $Y = B \cdot (C + 0)$

Using Identity Law: $Y = B \cdot C$

2. Null Law

- AND: $A \cdot 0 = 0$
- OR: $A + 1 = 1$

Examples:

1. $Y = A \cdot 0 + 1$

Using Null Law: $Y = 0 + 1 = 1$

2. $Y = (B + 1) \cdot C$

Using Null Law: $Y = 1 \cdot C = C$

3. $Y = A + 1 \cdot B$

Using Null Law: $Y = A + 1 = 1$

3. Idempotent Law

- AND: $A \cdot A = A$
- OR: $A + A = A$

Examples:

1. $Y = A \cdot A + A$

Using Idempotent Law: $Y = A + A = A$

2. $Y = (B \cdot B) + C$

Using Idempotent Law: $Y = B + C$

3. $Y = A \cdot A \cdot A$

Using Idempotent Law: $Y = A \cdot A = A$

4. Complement Law

- AND: $A \cdot \overline{A} = 0$
- OR: $A + \overline{A} = 1$

Examples:

1. $Y = A \cdot \overline{A} + B$

Using Complement Law: $Y = 0 + B = B$

2. $Y = (A + \overline{A}) \cdot C$

Using Complement Law: $Y = 1 \cdot C = C$

3. $Y = B \cdot \overline{B} + A \cdot \overline{A}$

Using Complement Law: $Y = 0 + 0 = 0$

5. Commutative Law

- AND: $A \cdot B = B \cdot A$
- OR: $A + B = B + A$

Examples:

1. $Y = A \cdot B + C$

Using Commutative Law: $Y = B \cdot A + C$

2. $Y = (B + A) + D$

Using Commutative Law: $Y = (A + B) + D$

3. $Y = C \cdot (A \cdot B)$

Using Commutative Law: $Y = C \cdot (B \cdot A)$

6. Associative Law

- AND: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
- OR: $(A + B) + C = A + (B + C)$

Examples:

1. $Y = (A \cdot B) \cdot C$

Using Associative Law: $Y = A \cdot (B \cdot C)$

2. $Y = ((A + B) + C) + D$

Using Associative Law: $Y = (A + (B + C)) + D$

3. $Y = A \cdot (B \cdot (C \cdot D))$

Using Associative Law: $Y = ((A \cdot B) \cdot C) \cdot D$

7. Distributive Law

- $A \cdot (B + C) = A \cdot B + A \cdot C$
- $A + (B \cdot C) = (A + B) \cdot (A + C)$

Examples:

1. $Y = A \cdot (B + C)$

Using Distributive Law: $Y = A \cdot B + A \cdot C$

2. $Y = (A + (B \cdot C)) + D$

Using Distributive Law: $Y = ((A + B) \cdot (A + C)) + D$

3. $Y = A \cdot (B + \overline{B})$

Using Complement and Distributive Laws: $Y = A \cdot 1 = A$

8. Absorption Law

- $A + (AB) = A(1 + B) = A \cdot 1 = A$
- $A \cdot (A + B) = A$

Examples:

1. $Y = A + (AB)$

Using Absorption Law: $Y = A$

2. $Y = A \cdot (A + C)$

Using Absorption Law: $Y = A$

3. $Y = (AC + B) \cdot (AC)$

Simplify using Absorption Law: $Y = A \cdot C$

4.5 DeMorgan's Theorem

DeMorgan's Theorems are fundamental in Boolean algebra and digital logic design. They provide a method to simplify the negation of AND and OR operations, making them essential for circuit design and simplification.

1. First Theorem:

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

The negation of an OR operation is equivalent to the AND operation of the negated inputs.

2. Second Theorem:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

The negation of an AND operation is equivalent to the OR operation of the negated inputs.

Proofs Using Truth Tables

First Theorem: $\overline{A + B} = \overline{A} \cdot \overline{B}$

A	B	$A + B$	$\overline{A + B}$	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Second Theorem: $\overline{A \cdot B} = \overline{A} + \overline{B}$

A	B	$A \cdot B$	$\overline{A \cdot B}$	$\overline{A}$	$\overline{B}$	$\overline{A + B}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Example 1: Simplify $\overline{X + Y} + Z$

1. Apply the first theorem:

$$\overline{X + Y} = \overline{X} \cdot \overline{Y}$$

2. Substitute into the expression:

$$\overline{X + Y} + Z = (\overline{X} \cdot \overline{Y}) + Z$$

- This is the simplified form.

Example 2: Simplify $\overline{(P \cdot Q)} \cdot R$

1. Apply the second theorem:

$$\overline{P \cdot Q} = \overline{P} + \overline{Q}$$

2. Substitute into the expression:

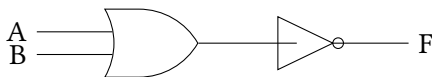
$$\overline{(P \cdot Q)} \cdot R = (\overline{P} + \overline{Q}) \cdot R$$

- This is the simplified form.

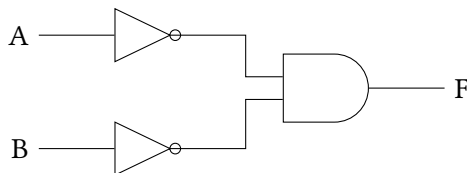
Circuit Representations

First Theorem: $\overline{A + B} = \overline{A} \cdot \overline{B}$

Original Circuit:

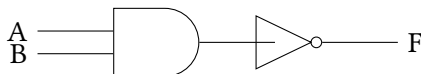


Equivalent Circuit:

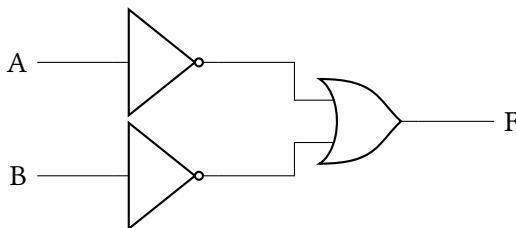


Second Theorem: $\overline{A \cdot B} = \overline{A} + \overline{B}$

Original Circuit:



Equivalent Circuit:



Applications of DeMorgan's Theorems

1. **Simplifying Boolean Expressions:** Complex logical expressions can be simplified for easier circuit implementation.
2. **Designing Circuits with Universal Gates:** DeMorgan's theorems allow the exclusive use of NAND or NOR gates to implement any logical function.
3. **Debugging Circuits:** Alternative representations of logic help identify faults or inefficiencies.

4.6 Logic Simplification Using Boolean Algebra

Now you can use all the previous rules for simplification to reduce the number of gates used, below are some examples to show you how to simplify the Boolean expression and how to use the rules mentioned before.

Example 1: Simplify $A \cdot (A + B \cdot C)$

$$\begin{aligned} F &= A \cdot (A + B \cdot C) \\ &= A \cdot A + A \cdot (B \cdot C) \quad (\text{Distributive Law}) \\ &= A + A \cdot (B \cdot C) \quad (\text{Idempotent Law}) \\ &= A \quad (\text{Absorption Law}) \end{aligned}$$

Final Simplified Expression: $F = A$

Example 2: Simplify $A + A \cdot \overline{B} + B$

$$\begin{aligned}F &= A + A \cdot \overline{B} + B \\&= A(1 + \overline{B}) + B \\&= A + B \quad (\text{Absorption Law})\end{aligned}$$

Final Simplified Expression: $F = A + B$

Example 3: Simplify $(A + B) \cdot (\overline{A} + C)$

$$\begin{aligned}F &= (A + B) \cdot (\overline{A} + C) \\&= (A \cdot \overline{A}) + (A \cdot C) + (B \cdot \overline{A}) + (B \cdot C) \quad (\text{Distributive Law}) \\&= 0 + (A \cdot C) + (B \cdot \overline{A}) + (B \cdot C) \quad (\text{Complement Law}) \\&= A \cdot C + B \cdot \overline{A} + B \cdot C \\&= A \cdot C + B \quad (\text{Factoring } B \cdot (\overline{A} + C))\end{aligned}$$

Final Simplified Expression: $F = A \cdot C + B$

Example 4: Simplify $\overline{A \cdot B} + \overline{A} \cdot \overline{B}$

$$\begin{aligned}F &= \overline{A \cdot B} + \overline{A} \cdot \overline{B} \\&= (\overline{A} + \overline{B}) + (\overline{A} \cdot \overline{B}) \quad (\text{DeMorgan's Theorem}) \\&= \overline{A} + \overline{B} \quad (\text{Absorption Law})\end{aligned}$$

Final Simplified Expression: $F = \overline{A} + \overline{B}$

Example 5: Simplify $A \cdot B + \overline{A} \cdot B + A \cdot \overline{B}$

$$\begin{aligned}F &= A \cdot B + \overline{A} \cdot B + A \cdot \overline{B} \\&= B \cdot (A + \overline{A}) + A \cdot \overline{B} \quad (\text{Factoring}) \\&= B + A \cdot \overline{B} \quad (\text{Complement Law } A + \overline{A} = 1)\end{aligned}$$

Final Simplified Expression: $F = B + A \cdot \overline{B}$

Example 6: Simplify $\overline{(A + \overline{B})} \cdot (A + B)$

$$\begin{aligned}F &= \overline{(A + \overline{B})} \cdot (A + B) \\&= (\overline{A} \cdot B) \cdot (A + B) \quad (\text{DeMorgan's Theorem}) \\&= (\overline{A} \cdot B \cdot A) + (\overline{A} \cdot B \cdot B) \quad (\text{Distributive Law}) \\&= 0 + \overline{A} \cdot B \quad (\text{Complement Law } \overline{A} \cdot A = 0) \\&= \overline{A} \cdot B\end{aligned}$$

Final Simplified Expression: $F = \overline{A} \cdot B$

Example 7: Simplify $A \cdot B + A \cdot \overline{B} + \overline{A} \cdot B$

$$\begin{aligned}F &= A \cdot (B + \overline{B}) + \overline{A} \cdot B \quad (\text{Factoring}) \\&= A \cdot 1 + \overline{A} \cdot B \quad (\text{Complement Law } B + \overline{B} = 1) \\&= A + \overline{A} \cdot B\end{aligned}$$

Final Simplified Expression: $F = A + \overline{A} \cdot B$

Example 8: Simplify $A + \bar{A} \cdot (B + C)$

$$\begin{aligned} F &= A + (\bar{A} \cdot B) + (\bar{A} \cdot C) \quad (\text{Distributive Law}) \\ &= A + B + C \quad (\text{Absorption Law } A + \bar{A} \cdot X = A + X) \end{aligned}$$

Final Simplified Expression: $F = A + B + C$

Example 9: Simplify $\overline{A \cdot B} \cdot (A + B)$

$$\begin{aligned} F &= (\bar{A} + \bar{B}) \cdot (A + B) \quad (\text{DeMorgan's Theorem}) \\ &= (\bar{A} \cdot A) + (\bar{A} \cdot B) + (\bar{B} \cdot A) + (\bar{B} \cdot B) \quad (\text{Distributive Law}) \\ &= 0 + (\bar{A} \cdot B) + (\bar{B} \cdot A) + 0 \quad (\text{Complement Law}) \\ &= \bar{A} \cdot B + \bar{B} \cdot A \end{aligned}$$

Final Simplified Expression: $F = \bar{A} \cdot B + \bar{B} \cdot A$

4.7 Sum of Products (SOP) in Digital Logic

The **Sum of Products (SOP)** is a standard form of representing Boolean functions. It is a logical expression where multiple product terms are summed (**ORed**) together. Each product term is a combination of literals (variables and their complements) connected by **AND** operations. SOP is widely used in digital circuit design because it is straightforward to implement using **AND**, **OR**, and **NOT** gates.

Structure of SOP Expression

An SOP expression has the following general form:

$$F(A, B, C, \dots) = m_1 + m_2 + m_3 + \dots + m_n$$

$\begin{array}{c|c} 101 & 227 \end{array}$

where:

- m_i are **minterms** (product terms).
- Each minterm contains all input variables in either true (A) or complemented (A') form.

Steps to Derive SOP Form

1. **Truth Table Construction:** Define the truth table for the given Boolean function.
2. **Identify Minterms:** Identify rows where the output is 1. These rows correspond to the minterms.
3. **Write Minterms:** For each 1 in the output, write a product term with:
 - The variable in true form (A) if it is 1 in that row.
 - The variable in complemented form (A') if it is 0 in that row.
4. **Combine Using OR:** Sum all the minterms using the OR operator.

Example 1: SOP Expression from a Truth Table

Given Truth Table:

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

Step-by-Step Solution:

1. Identify rows where $F = 1$: Rows 2, 3, and 4.
2. Write the minterms:

$$\text{Row 2: } A'B, \quad \text{Row 3: } AB', \quad \text{Row 4: } AB$$

3. Combine the minterms:

$$F(A, B) = A'B + AB' + AB$$

Example 2: SOP Expression for Three Variables**Given Truth Table:**

<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i>
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Step-by-Step Solution:

1. Identify rows where $F = 1$: Rows 2, 4, 5, 6, and 8.

2. Write the minterms:

Row 2: $A'B'C$, Row 4: $A'BC$, Row 5: $AB'C'$, Row 6 : $AB'C$,

3. Combine the minterms:

$$F(A, B, C) = A'B'C + A'BC + AB'C' + AB'C + ABC$$

Canonical SOP Form

In the **canonical SOP form**, each minterm includes **all variables** in either their true or complemented form. For example:

- If A, B, C are variables, each minterm will have terms like $A'B'C$, $AB'C'$, etc.

Example: Given the function $F = AC + BC$, express it in canonical SOP form:

1. Expand missing variables using the distributive property:

$$F = AC + BC = (AC)(B + B') + (BC)(A + A')$$

$$F = ABC + AB'C + ABC + A'BC$$

2. Simplify:

$$F = ABC + AB'C + A'BC$$

Implementing SOP in Digital Circuits

To implement an SOP expression:

1. Use **AND gates** to generate each product term.
2. Use **OR gates** to sum the product terms.
3. Use **NOT gates** to complement variables as needed.

Example 3: SOP Circuit Design

Given $F(A, B, C) = A'B'C + A'BC + ABC$, design the circuit:

1. Write minterms:
 - $A'B'C$: Requires **NOT gates** for A and B , then AND with C .
 - $A'BC$: Requires a **NOT gate** for A , then AND with B and C .
 - ABC : All variables in true form, AND them.
2. Use **OR gates** to sum the terms.

Advantages of SOP Form

- Easy to derive from truth tables.
- Directly implementable in digital circuits using basic gates.
- Suitable for minimization techniques like Karnaugh maps.

Additional Examples for Practice

1. Construct SOP for:

$$A + B'C, \quad AB' + BC + A'C'$$

2. Convert $F(A, B, C) = AB + AC$ into canonical SOP form.

4.8 Product of Sums (POS) in Digital

The **Product of Sums (POS)** is a standard form of representing Boolean functions. In the POS form, the Boolean expression is written as a product (AND) of multiple sum terms (OR). Each sum term is a combination of variables (literals) in either their true or complemented form.

This representation is particularly useful in digital circuit design as it directly maps to NOR or OR-AND gate implementations.

Structure of POS Expression

A POS expression can be written as:

$$F(A, B, C, \dots) = (S_1)(S_2)(S_3) \dots (S_n)$$

Where:

- S_i represents **sum terms (maxterms)**.
- Each sum term contains all variables in either true (A) or complemented (A') form.

Steps to Derive POS Form

1. **Truth Table Construction:** Create the truth table for the given Boolean function.
2. **Identify Maxterms:** Identify rows in the truth table where the output (F) is 0. These rows represent the maxterms.
3. **Write Maxterms:** For each row where $F = 0$:
 - Use the variable in complemented form (A') if it is 1 in that row.
 - Use the variable in true form (A) if it is 0 in that row.
4. **Combine Using AND:** Multiply all the sum terms using the AND operator to form the final POS expression.

Example 1: POS Expression from a Truth Table

Given Truth Table:

A	B	F
0	0	1
0	1	0
1	0	1
1	1	0

Step-by-Step Solution:

1. Identify rows where $F = 0$: Rows 2 and 4.

2. Write the maxterms:

$$\text{Row 2: } (A + B'), \quad \text{Row 4: } (A' + B')$$

3. Combine the maxterms:

$$F(A, B) = (A + B')(A' + B')$$

Example 2: POS Expression for Three Variables

Given Truth Table:

<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i>
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Step-by-Step Solution:

1. Identify rows where $F = 0$: Rows 2, 4, 6, and 8.

2. Write the maxterms:

$$\text{Row 2: } (A+B+C'), \quad \text{Row 4: } (A+B'+C'), \quad \text{Row 6: } (A'+B+C'),$$

3. Combine the maxterms:

$$F(A, B, C) = (A+B+C')(A+B'+C')(A'+B+C')(A'+B'+C')$$

Canonical POS Form

In the **canonical POS form**, each maxterm includes **all variables** in either their true or complemented form.

Example: Given $F(A, B) = A + B'C$, express it in canonical POS form.

1. Construct the truth table for $F(A, B, C)$.
2. Identify rows where $F = 0$, and write the maxterms:

$$F(A, B, C) = (A + B + C)(A + B' + C')(A' + B + C)$$

Logic Circuit Implementation

The POS form can be directly implemented using logic gates. Each sum term corresponds to an OR gate, and the product of these terms corresponds to an AND gate.

Example: Circuit for $F(A, B, C) = (A + B + C)(A' + B' + C)(A' + B + C')$:

Advantages of POS Form

- Useful for implementing functions with NOR gates.
- Allows easier simplification for certain Boolean functions.
- Complements SOP for a complete understanding of Boolean logic.

4.9 Simplifying Circuits Using K-Maps

Karnaugh Maps (K-Maps) are a powerful tool for simplifying Boolean expressions. This section explains K-Maps step-by-step with examples, including clear diagrams to visualize the simplification process.

Steps to Simplify Using K-Maps

1. **Write the Boolean Function:** Represent the function in Sum of Minterms (SOP) or Product of Maxterms (POS) .
2. **Fill the K-Map:** Use the truth table to plot 1s (for SOP) or 0s (for POS) in the K-Map. This is an example for a 2-Variable K-Map where m_0, m_1, m_2, m_3 are called cells of the K-Map and contains all the combinations for the AB or 2-Variable truth table which contains 4 combinations.

$$B = 0 \quad B = 1$$

$A = 0$	m_0	m_1
$A = 1$	m_3	m_2

or 3-Variable truth table:

$$\overline{AB} = 00\overline{AB} = 01AB = 11A\overline{B} = 10$$

$\overline{C} = 0$	m ₀	m ₁	m ₃	m ₂
$C = 1$	m ₄	m ₅	m ₇	m ₆

or 4-Variable truth table:

$$\overline{AB} = 00\overline{AB} = 01AB = 11A\overline{B} = 10$$

$\overline{CD} = 00$	m ₀	m ₁	m ₃	m ₂
$\overline{C}D = 01$	m ₄	m ₅	m ₇	m ₆
$\overline{C}D = 11$	m ₈	m ₉	m ₁₁	m ₁₀
$C\overline{D} = 10$	m ₁₂	m ₁₃	m ₁₅	m ₁₄

- Group Adjacent Cells:** Combine adjacent 1s (or 0s) into groups of size 2^n (1, 2, 4, 8, etc.).
- Derive the Expression:** Each group corresponds to a simplified term.
- Verify the Expression:** Use a truth table to confirm equivalence.

Example 1: Simplify a 2-Variable Function

Given the truth table:

A	B	$F(A, B)$
0	0	1
0	1	1
1	0	0
1	1	1

Step 1: Write the Boolean Function

From the truth table:

$$F(A, B) = m(0) + m(1) + m(3) = \overline{A} \cdot \overline{B} + \overline{A} \cdot B + A \cdot B$$

$F(A, B)$ is the combination for all the 1's element in the truth table which are m_0 , m_1 , and m_3 .

Since the default of the value of the letter is 1 so now reading m_0 cell gives us $A=0$ which is complement of A and denoted as $\overline{A}$ in the derives expression and $B=0$ also so it will be denoted in the expression as $\overline{B}$. Now the m_0 will be $\overline{A} \cdot \overline{B}$, now apply this to all the cells that contain 1.

Step 2: Fill the K-Map

The K-Map for $F(A, B)$ is shown below:

$$B = 0 \quad B = 1$$

$A = 0$	1	1
$A = 1$	0	1

112
227

Step 3: Group Adjacent 1s

$$B = 0B = 1$$

$A = 0$	1	1
$A = 1$	0	1

Group 1 (cyan): Top row ($A = 0$).

Group 2 (red): Bottom-right ($A = 1, B = 1$).

Step 4: Simplified Expression

K-Map Simplify the 2 groups, start with the first group looking to the headers for the columns of the group 1 it contains both the letter B and its complement $\bar{B}$ so they are cancelled and the result equals the remaining part which is $\bar{A}$.

Group 1: $\bar{A}$

Group 2: $A \cdot B$

Simplified function:

$$F(A, B) = \bar{A} + A \cdot B$$

Example 2: Simplify a 3-Variable Function

Given:

$$F(A, B, C) = \sum m(1, 3, 5, 7)$$

Step 1: Truth Table

A	B	C	$F(A, B, C)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

From the Truth Table:

$$F = \overline{A}BC + \overline{A}BC + A\overline{B}C + ABC$$

Step 2: K-Map Simplification

$$\overline{A}B = 00 \quad \overline{A}B = 01 \quad AB = 11 \quad A\overline{B} = 10$$

$C = 0$	0	0	0	0
$C = 1$	1	1	1	1

From the previous K-Map there is only one group. Resulting in B and $\overline{B}$ cancel each other
 $\overline{A}$ cancel with A
 The result will be C .

Example 3: Simplify a 4-Variable Function

We are given the truth table for $F(A, B, C, D)$. Let's simplify this Boolean function step by step.

We are given the truth table for $F(A, B, C, D)$. Let's simplify this Boolean function step by step.

A	B	C	D	$F(A, B, C, D)$
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Step 1: Write the K-Map

$$\overline{AB} = 00\overline{AB} = 01AB = 11A\overline{B} = 10$$

$\overline{C}\overline{D} = 00$	1	1	0	1
$\overline{C}D = 01$	1	1	1	0
$CD = 11$	1	0	1	1
$C\overline{D} = 10$	0	0	0	0

Step 2: Group the 1s in the K-Map

The goal is to form groups of adjacent 1s in sizes of powers of 2 (1, 2, 4, 8, etc.). Groups can wrap around the edges of the K-map.

- **Group 1 (4 Red cells):** A group of four 1s in the top-left $\overline{ABCD}, \overline{A}\overline{B}\overline{C}\overline{D}, \overline{A}\overline{B}CD, \overline{A}\overline{B}\overline{C}D$:

This simplifies to: $\overline{A}\overline{C}$.

- **Group 2 (2 green cells):** A group of two 1s in the right ($AB\overline{C}D, ABCD$):

This simplifies to: ABD .

- **Group 4 (blue cells):** A group of four 1s in corners ($\overline{A}\overline{B}\overline{C}\overline{D}, \overline{A}\overline{B}CD, \overline{A}\overline{B}\overline{C}D, \overline{A}\overline{B}CD$): This simplifies to: $\overline{B}$.

Step 3: Simplified Expression

Combining all groups, the simplified Boolean function is:

$$F(A, B, C, D) = \overline{A}\overline{C} + ABD + \overline{B}$$

4.10 Don't Care in K-Maps:

A don't care condition in a Karnaugh Map (K-map) represents a situation where the output for certain input combinations does not affect the system's functionality. These are denoted by 'X' in the K-map. You can use these don't care conditions to simplify Boolean expressions further by grouping them with 1s or ignoring them, depending on which leads to a simpler expression.

Example: Simplifying with Don't Care Conditions

Problem:

Simplify the following function using a Karnaugh Map (K-map):

$$F(A, B, C) = \Sigma m(1, 3, 7) + \Sigma d(0, 2, 5)$$

Where:

- Σm : Minterms (positions where the function F is 1).
- Σd : Don't care conditions (positions where F can be either 0 or 1).

Steps:

1. **Create the K-map:** For a 3-variable function, the K-map layout is as follows:

2. Fill in the K-map:

- Mark $F = 1$ at positions 1, 3, and 7 (minterms).
- Mark X (don't care) at positions 0, 2, and 5.
- Remaining positions are 0.

$$\overline{AB} = 00\overline{AB} = 01AB = 11A\overline{B} = 10$$

$\overline{C} = 0$	X	1	1	X
$C = 1$	0	X	1	0

3. Group the cells:

- Group 1 (covering cells 0, 1, 2, 3): This simplifies to $\overline{C}$.
- Group 2 (covering cells 1, 3, 5, 7): This simplifies to B .

4. Formulate the simplified expression:

$$F = \overline{C} + B$$

4.11 Exercises: Circuit Simplification

1. Identify the law of Boolean algebra upon which each of the following equalities is based:

(a) $A + AB + ABC + ABCD = \overline{ABCD} + ABC + AB + A$

$$(b) A + \overline{AB} + ABC + ABCD = \overline{DCBA} + \overline{CBA} + \overline{BA} + A$$
$$(c) AB(\overline{CD} + \overline{CD} + EF + \overline{EF}) = ABCD + ABC\overline{D} + ABEF + A\overline{B}\overline{E}\overline{F}$$

2. Using Boolean algebra, simplify the following expressions:

$$(a) (\overline{A} + B)(A + C)$$
$$(b) \overline{AB} + \overline{ABC} + \overline{ABCD} + \overline{ABCDE}$$
$$(c) BC + \overline{BCD} + B$$
$$(d) (B + \overline{B})(BC + \overline{BCD})$$
$$(e) BC + (B + \overline{C})D + BC$$

3. Using Boolean algebra, simplify the following expressions:

$$(a) CE + C(E + F) + \overline{E}(E + G)$$
$$(b) \overline{BCD} + (B + C + \overline{D}) + \overline{BCDE}$$
$$(c) (C + CD)(C + \overline{CD})(C + E)$$
$$(d) BCDE + BC(\overline{DE}) + (BC)DE$$
$$(e) BCD[BC + \overline{DCD} + BD]$$

4. Draw the logic circuit represented by each expression:

$$(a) AB + A\overline{B}$$
$$(b) ABCD$$
$$(c) A + BC$$

- (d) $ABC + D$
- (e) $(\overline{A + B}) \cdot (C + D)$
- (f) $(A \cdot \overline{B}) + (C \cdot B)$
- (g) $(A \cdot B) + (\overline{C} \cdot D) + E$

5. Apply DeMorgan's theorems to each expression:

- (a) $\overline{A + B}$
- (b) $\overline{AB}$
- (c) $\overline{A + B + C}$
- (d) $\overline{ABC}$
- (e) $A(\overline{B + C})$
- (f) $\overline{AB + CD}$
- (h) $AB + \overline{CD}$
- (g) $\overline{(A + B)(C + D)}$

6. Convert the following expressions to sum-of-product (SOP) forms:

- (a) $(C + D)(A + \overline{D})$
- (b) $A(A\overline{D} + C)$
- (c) $(A + C)(CD + AC)$

7. Develop a truth table for each of the following standard SOP expressions:

$$(a) \overline{A}BCD + ABC\overline{D} + \overline{A}BC\overline{D} + \overline{A}\overline{B}CD$$

$$(b) WXYZ + WX\overline{Y}Z + W\overline{X}YZ + \overline{W}XYZ + \overline{W}XY\overline{Z} + WXY\overline{Z}$$

8. Develop a truth table for each of the following standard POS expressions:

$$(a) (\overline{A} + \overline{B} + C)(A + B + C)(A + B + \overline{C})$$

$$(b) (A + \overline{B} + C + D)(\overline{A} + B + \overline{C} + D)(A + B + \overline{C} + \overline{D})(\overline{A} + \overline{B} + C + D)$$

9. Use a Karnaugh map to find the minimum SOP form for each expression:

$$(a) \overline{A}\overline{B}C + \overline{A}BC + \overline{A}B\overline{C}$$

$$(b) AC(\overline{B} + C)$$

$$(c) \overline{A}(BC + \overline{B}C) + A(\overline{B}C + \overline{B}C)$$

$$(d) \overline{A}BC + ABC\overline{C} + \overline{A}B\overline{C} + ABC\overline{C}$$

10. Use a Karnaugh map to find the minimum POS form for each expression:

$$(a) (A + B + C)(\overline{A} + \overline{B} + C)(A + \overline{B} + C)(A + B + \overline{C})$$

$$(b) (X + \overline{Y})(\overline{X} + Z)(X + Y + \overline{Z})(\overline{X} + \overline{Y} + Z)(\overline{X} + Y + Z)$$

$$(c) A(B + \overline{C})(\overline{A} + C)(A + B + C)(\overline{A} + B + C)(A + \overline{B} + \overline{C})$$

Chapter 5

Combinational Logic Gates

Combinational circuits are a fundamental type of digital logic circuit where the output depends solely on the current input values. Unlike sequential circuits, combinational circuits have no memory elements and thus do not rely on past inputs or outputs. These circuits are designed to perform specific logical functions using basic logic gates such as AND, OR, NOT, NAND, NOR, XOR, and XNOR.

From 5.1 **When input changes**, the **output changes** dynamically, which makes combinational circuits ideal for tasks like arithmetic operations, encoding, and decision-making.

Key Characteristics of Combinational Circuits

- **No Memory:** Combinational circuits do not store information. The output is purely determined by the present combination of inputs.
- **Inputs and Outputs:**
 - n binary inputs are provided to the circuit.

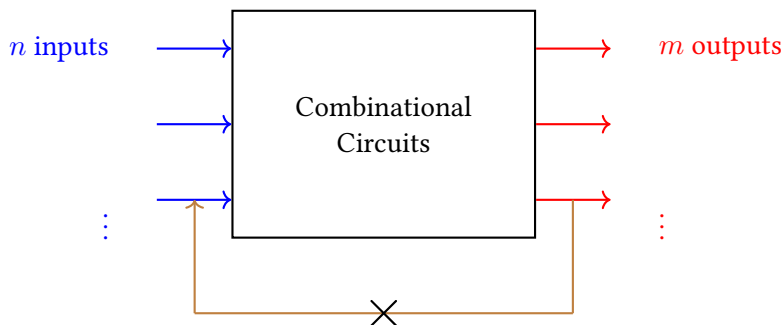


Figure 5.1: *Combinational Circuit Block Diagram*

- The circuit produces m binary outputs.
- **Deterministic Behavior:**
 - Each input combination leads to a unique output.
 - Outputs may change instantly (propagation delay) as the inputs change.
- **Boolean Representation:**
 - The operations of the circuit are defined using Boolean expressions.
- **Independent Timing:** There is no clock signal or sequential operation involved.

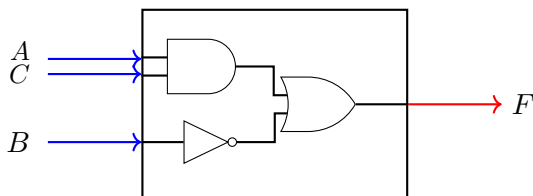


Figure 5.2: Example for combinational circuit analysis given a function

5.1 Structure of Combinational Circuits

A combinational circuit typically consists of:

1. **Input Variables:** The signals fed into the circuit.
2. **Logic Gates:** A combination of gates such as AND, OR, NOT, etc., which perform the desired logic operation.
3. **Output Variables:** The result of the logic operation.

Analysis

Objective: Given a circuit, determine its *function*.

Representation Methods:

- Boolean Function
- Truth Table

From 5.2 we can get the output boolean function and the truth table for the given circuit.

The output function (F) for the above circuit is:

$$F = (A \cdot C) + \overline{B}$$

Truth Table The truth table for the circuit is as follows:

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Analysis Procedure

To obtain the output Boolean functions from a logic diagram:

- Label all gate outputs that are functions of input variables.**
 - Determine the Boolean functions for each gate output.
- Label the gates that are a function of input variables and previously labeled gates.**
 - Find the Boolean function for these gates.
- Repeat the process outlined in step 2 until the outputs are obtained.**

4. Obtain the output Boolean functions in terms of input variables.

Explanation:

- This procedure is essential for analyzing a combinational logic circuit.
- The process begins by identifying the functionality of each gate starting from the inputs.
- Each step ensures that intermediate results are determined to compute the final output.
- The Boolean functions derived provide a mathematical representation of the circuit's behavior.

Design

Objective: Given a desired *function*, determine the required circuit.

Representation Methods:

- Boolean Function
- Truth Table

Explanation:

- The desired function is $F = A \cdot B + \overline{C}$.
- The circuit comprises:

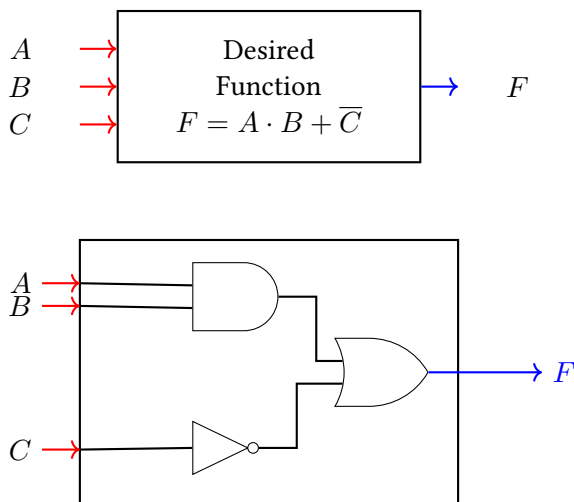


Figure 5.3: Example for combinational circuit design given a boolean function

- An AND gate to compute $A \cdot B$.
 - A NOT gate to compute $\overline{C}$.
 - An OR gate to combine $A \cdot B$ and $\overline{C}$: $F = (A \cdot B) + \overline{C}$.
- Inputs (A, B, C) are processed through the circuit to produce the output (F).

Design Procedure

For a given problem statement:

- **Determine the number of inputs and outputs.**
- **Derive the truth table.**
- **Simplify the Boolean expression for each output.**
- **Produce the required circuit.**

Explanation:

The design procedure involves the following steps:

1. **Determine Inputs and Outputs:** Identify the number of inputs and outputs required based on the problem statement. This step defines the scope of the circuit.
2. **Truth Table:** Construct the truth table that captures the relationship between inputs and outputs.
3. **Simplify Boolean Expressions:** Use techniques such as Karnaugh maps or Boolean algebra to simplify the logical expressions derived from the truth table.
4. **Circuit Implementation:** Design and produce the circuit using logic gates (AND, OR, NOT, etc.) based on the simplified Boolean expressions.

This process ensures a systematic approach to designing efficient and optimized combinational logic circuits.

Examples of Combinational Circuits

1. Basic Logic Circuits

- **AND Gate:** Output is high (1) only when all inputs are high.
- **OR Gate:** Output is high (1) if at least one input is high.
- **NOT Gate:** Inverts the input signal.

2. Arithmetic Circuits

- **Half Adder:** Adds two binary digits to produce a sum and carry output.
- **Full Adder:** Adds three binary digits (two inputs and one carry input).
- **Subtractor:** Performs binary subtraction.

3. Data Circuits

- **Multiplexer (MUX):** Selects one input from multiple inputs based on control signals and forwards it to the output.
- **Demultiplexer (DEMUX):** Takes a single input and distributes it to one of several outputs based on control signals.
- **Encoder:** Converts multiple input lines into a smaller number of output lines (binary representation).
- **Decoder:** Converts a binary input into a specific output line.

4. Comparators

Compare two binary numbers and determine their relationship (greater than, less than, or equal).

5.2 Arithmetic Circuits & Data Circuits

5.2.1 Adders (Half Adder): Explanation with Examples

A **half adder** is a fundamental digital circuit that adds two single-bit binary numbers. It has two outputs:

- **Sum (S):** The sum of the two inputs.
- **Carry (C):** The carry-out from the addition.

Truth Table for a Half Adder

A	B	$S = A \oplus B$	$C = A \cdot B$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions for Half Adder

The outputs of the half adder are derived as follows:

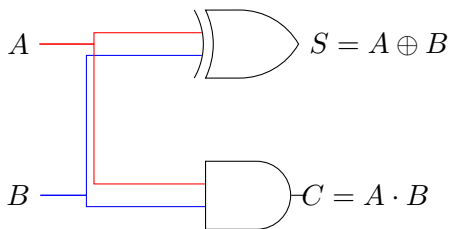
- **Sum (S):**

$$S = A \oplus B$$

- **Carry (C):**

$$C = A \cdot B$$

Circuit Diagram of a Half Adder



Examples

Example 1: Add $A = 0$, $B = 1$ Using a Half Adder

Inputs: $A = 0$, $B = 1$

Step 1: Calculate Sum (S):

$$S = A \oplus B = 0 \oplus 1 = 1$$

Step 2: Calculate Carry (C):

$$C = A \cdot B = 0 \cdot 1 = 0$$

Outputs:

$$S = 1, \quad C = 0$$

Example 2: Add $A = 1$, $B = 1$ Using a Half Adder

Inputs: $A = 1$, $B = 1$

Step 1: Calculate Sum (S):

$$S = A \oplus B = 1 \oplus 1 = 0$$

Step 2: Calculate Carry (C):

$$C = A \cdot B = 1 \cdot 1 = 1$$

Outputs:

$$S = 0, \quad C = 1$$

Example 3: Add $A = 1, B = 0$ Using a Half Adder

Inputs: $A = 1, B = 0$

Step 1: Calculate Sum (S):

$$S = A \oplus B = 1 \oplus 0 = 1$$

Step 2: Calculate Carry (C):

$$C = A \cdot B = 1 \cdot 0 = 0$$

Outputs:

$$S = 1, \quad C = 0$$

Applications of Half Adders

- **Building Full Adders:** A half adder is a fundamental component used in constructing full adders.
- **Arithmetic Units:** Used in microprocessors for single-bit addition.
- **Digital Logic Design:** Forms the basis of more complex arithmetic circuits.

Limitations of Half Adders

- It cannot handle carry-in for multi-bit addition.
- Used only for single-bit addition.

5.2.2 Adders (Full Adder): Explanation with Examples

A **full adder** is a combinational circuit that adds three single-bit binary numbers: two inputs (A and B) and a carry-in (C_{in}). It produces two outputs:

- **Sum (S):** The sum of the three inputs.
- **Carry-Out (C_{out}):** The carry resulting from the addition.

Truth Table for a Full Adder

A	B	C_{in}	$S = A \oplus B \oplus C_{in}$	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expressions for a Full Adder

The outputs of the full adder are derived as follows:

1. Sum (S):

$$S = A \oplus B \oplus C_{\text{in}}$$

This represents the XOR of the three inputs.

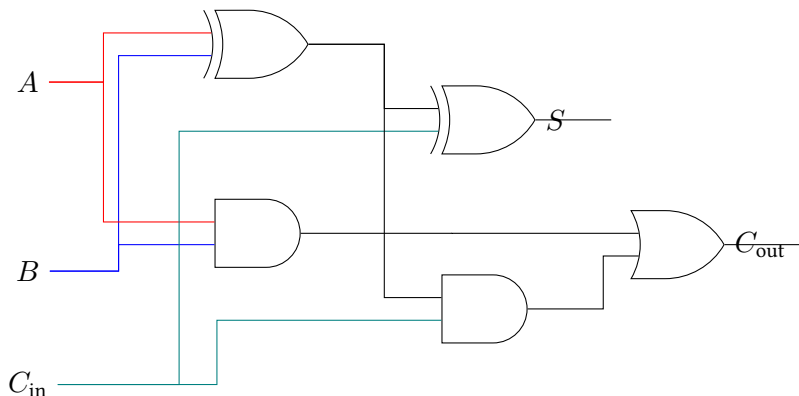
2. Carry-Out (C_{out}):

$$C_{\text{out}} = (A \cdot B) + (B \cdot C_{\text{in}}) + (A \cdot C_{\text{in}})$$

Circuit Diagram of a Full Adder

The full adder can be implemented by combining two half adders and an OR gate:

- The first half adder adds A and B to produce a partial sum (S_1) and a carry (C_1).
- The second half adder adds S_1 and C_{in} to produce the final sum (S) and a second carry (C_2).
- An OR gate combines C_1 and C_2 to produce the final carry-out (C_{out}).



Example 1: Add $A = 0, B = 1, C_{in} = 1$ Using a Full Adder

Inputs: $A = 0, B = 1, C_{in} = 1$

Step 1: Calculate Sum (S):

$$S = A \oplus B \oplus C_{in} = 0 \oplus 1 \oplus 1 = 0$$

Step 2: Calculate Carry-Out (C_{out}):

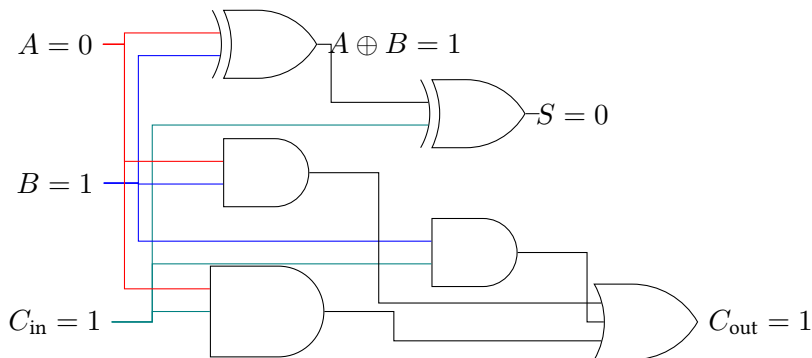
$$C_{out} = (A \cdot B) + (B \cdot C_{in}) + (A \cdot C_{in})$$

Substituting values:

$$C_{out} = (0 \cdot 1) + (1 \cdot 1) + (0 \cdot 1) = 0 + 1 + 0 = 1$$

Outputs:

$$S = 0, \quad C_{out} = 1$$



Example 2: Add $A = 1, B = 1, C_{in} = 0$ Using a Full Adder

Inputs: $A = 1, B = 1, C_{in} = 0$

Step 1: Calculate Sum (S):

$$S = A \oplus B \oplus C_{in} = 1 \oplus 1 \oplus 0 = 0$$

Step 2: Calculate Carry-Out (C_{out}):

$$C_{out} = (A \cdot B) + (B \cdot C_{in}) + (A \cdot C_{in})$$

Substituting values:

$$C_{out} = (1 \cdot 1) + (1 \cdot 0) + (1 \cdot 0) = 1 + 0 + 0 = 1$$

Outputs:

$$S = 0, \quad C_{out} = 1$$

Example 3: Add $A = 1, B = 0, C_{in} = 1$ Using a Full Adder

Inputs: $A = 1, B = 0, C_{in} = 1$

Step 1: Calculate Sum (S):

$$S = A \oplus B \oplus C_{in} = 1 \oplus 0 \oplus 1 = 0$$

Step 2: Calculate Carry-Out (C_{out}):

$$C_{out} = (A \cdot B) + (B \cdot C_{in}) + (A \cdot C_{in})$$

Substituting values:

$$C_{out} = (1 \cdot 0) + (0 \cdot 1) + (1 \cdot 1) = 0 + 0 + 1 = 1$$

Outputs:

$$S = 0, \quad C_{out} = 1$$

Applications of Full Adders

- **Multi-Bit Addition:** Full adders are cascaded to add multi-bit binary numbers.
- **Arithmetic Circuits:** Used in CPUs and ALUs for arithmetic operations.
- **Digital Logic Design:** Form the basis of advanced combinational logic circuits.

Binary Adder-Subtractor

Implementation of a Full Adder using Two Half Adders and an OR Gate

Circuit Description: The Full Adder computes the sum (S) and carry (C) of three binary inputs (X, Y, Z):

- **Step 1:** Compute the intermediate sum ($S_1 = X \oplus Y$) and carry ($C_1 = X \cdot Y$) using the first Half Adder.
- **Step 2:** Use the second Half Adder to compute the final sum ($S = S_1 \oplus Z$) and carry ($C_2 = S_1 \cdot Z$).
- **Step 3:** Combine the intermediate carries (C_1 and C_2) using an OR gate to produce the final carry (C).

Boolean Equations:

- **Sum:** $S = (X \oplus Y) \oplus Z$
- **Carry:** $C = (X \cdot Y) + ((X \oplus Y) \cdot Z)$

Circuit Diagram:

Explanation:

- **Inputs:** X, Y are the primary inputs, and Z is the carry-in.
- **First Half Adder:**
 - Intermediate sum: $S_1 = X \oplus Y$.
 - Intermediate carry: $C_1 = X \cdot Y$.

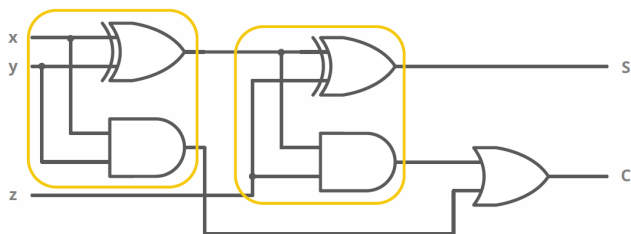


Figure 5.4: Implementation of Full Adder Using Two Half Adders and OR gate

- **Second Half Adder:**

- Final sum: $S = S_1 \oplus Z$.
- Second carry: $C_2 = S_1 \cdot Z$.

- **OR Gate:**

- Combines the intermediate carries (C_1 and C_2) to produce the final carry: $C = C_1 + C_2$.

- **Outputs:**

- Final sum: S .
- Final carry: C .

5.2.3 Subtractor (Half Subtractor)

A **Half Subtractor** is a combinational logic circuit used to perform binary subtraction of two bits. It has two outputs:

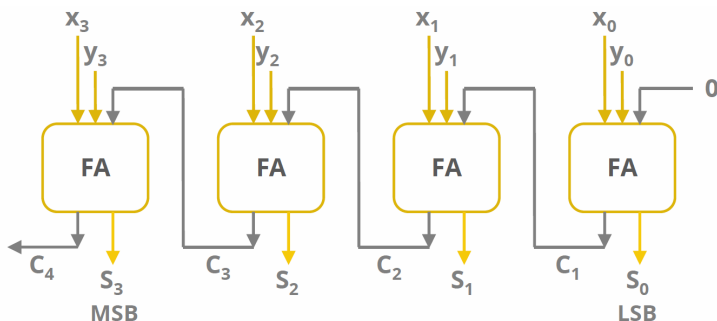


Figure 5.5: Binary Adder: 4-bit Ripple Carry Adder

- **Difference (D):** The binary difference of the inputs.
- **Borrow (B):** Indicates if a borrow is required in the subtraction.

The two inputs to the Half Subtractor are:

- **A:** Minuend (the bit to be subtracted from).
- **B:** Subtrahend (the bit to subtract).

Truth Table

The truth table for the Half Subtractor is:

A	B	Difference (D)	Borrow (B)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Logic Expressions

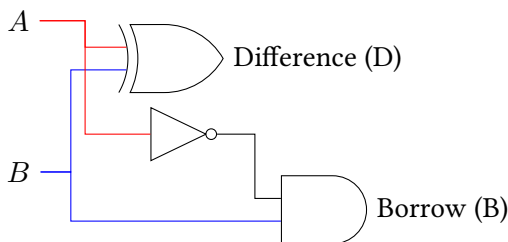
The outputs for the Half Subtractor are derived as follows:

$$\text{Difference (D)} = A \oplus B \quad (\text{XOR gate})$$

$$\text{Borrow (B)} = \overline{A} \cdot B \quad (\text{AND gate with NOT on A})$$

Circuit Diagram

The logic circuit for the Half Subtractor is shown below:



Example 1: Subtract $B = 0$ from $A = 1$.

$$A = 1, \quad B = 0$$

$$\text{Difference (D)} = A \oplus B = 1 \oplus 0 = 1$$

$$\text{Borrow (B)} = \overline{A} \cdot B = \overline{1} \cdot 0 = 0$$

Output: Difference = 1, Borrow = 0.

Example 2: Subtract $B = 1$ from $A = 0$.

$$A = 0, \quad B = 1$$

$$\text{Difference (D)} = A \oplus B = 0 \oplus 1 = 1$$

$$\text{Borrow (B)} = \bar{A} \cdot B = \bar{0} \cdot 1 = 1$$

Output: Difference = 1, Borrow = 1.

Example 3: Subtract $B = 1$ from $A = 1$.

$$A = 1, \quad B = 1$$

$$\text{Difference (D)} = A \oplus B = 1 \oplus 1 = 0$$

$$\text{Borrow (B)} = \bar{A} \cdot B = \bar{1} \cdot 1 = 0$$

Output: Difference = 0, Borrow = 0.

5.2.4 Subtractor (Full Subtractor)

The Half Subtractor is used to subtract only two numbers. To overcome this problem, a full subtractor was designed. A **Full Subtractor** is a combinational circuit that performs subtraction of three bits: **Minuend (A)**, **Subtrahend (B)**, and **Borrow In (Bin)**. It produces two outputs:

- **Difference (D):** The result of the subtraction.
- **Borrow Out (Bout):** Indicates if a borrow is needed for the next higher bit subtraction.

Truth Table

The truth table of a Full Subtractor is given below:

A	B	B_{in}	$D(Difference)$	$B_{out}(BorrowOut)$
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Boolean Expressions

The Boolean expressions for the outputs of a Full Subtractor are:

$$D = A \oplus B \oplus B_{in}$$

$$B_{out} = \overline{A}B + \overline{A}B_{in} + BB_{in}$$

Full Subtractor Circuit Diagram

The circuit diagram of a Full Subtractor can be constructed using XOR gates, AND gates, and OR gates. Below is a simple representation.

Construction of Full Subtractor Circuit:

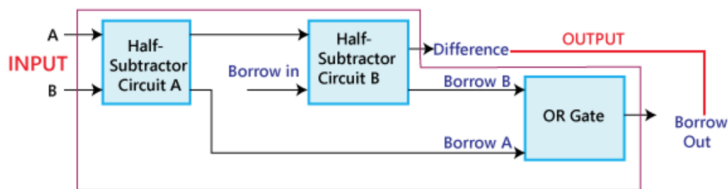


Figure 5.6: Full Subtractor Circuit Diagram

In the given circuit, two half-subtractor circuits are interconnected using an OR gate. The first half-subtractor takes two single-bit binary inputs, A and B . Since a half-subtractor generates two outputs, namely 'Difference' and 'Borrow', the 'Difference' output from the first half-subtractor serves as the first input to the second half-subtractor. Similarly, the 'Borrow' output from the first half-subtractor is passed as the second input to the second half-subtractor.

The second half-subtractor then produces its own 'Difference' and 'Borrow' outputs. The final 'Difference' bit of the full subtractor circuit is obtained from the second half-subtractor. To determine the final 'Borrow' output, the 'Borrow' outputs from both half-subtractors are fed into an OR gate. The result of this OR operation gives the final 'Borrow' output of the full subtractor circuit.

The actual logic circuit of the full subtractor is shown in the above diagram 5.7. The full subtractor circuit construction can also be represented in a Boolean expression.

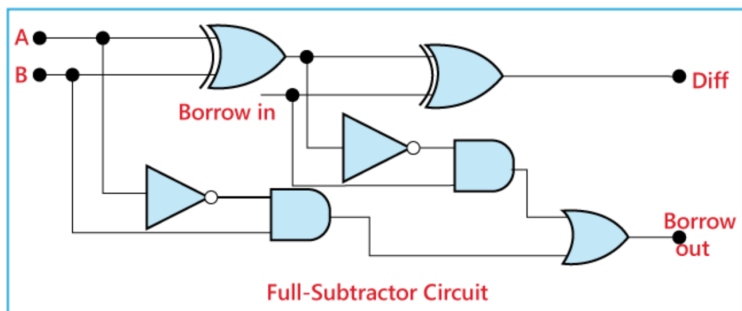


Figure 5.7: Full Subtractor Circuit Gate Design

Working Example

Consider the subtraction of three binary bits: $A = 1$, $B = 0$, $B_{in} = 1$:

- Compute the Difference:

$$D = A \oplus B \oplus B_{in} = 1 \oplus 0 \oplus 1 = 0$$

- Compute the Borrow Out:

$$B_{out} = \overline{A}B + \overline{A}B_{in} + BB_{in}$$

Substituting the values:

$$B_{out} = \overline{1} \cdot 0 + \overline{1} \cdot 1 + 0 \cdot 1 = 0$$

Hence, the result is $D = 0$, $B_{out} = 0$.

Binary Adder-Subtractor

A Binary Adder-Subtractor is a specialized circuit capable of performing both addition and subtraction operations. The specific operation executed is determined by the value of the control signal. Within an Arithmetic Logic Unit (ALU), it serves as a crucial component.

To effectively use a Binary Adder-Subtractor, it is essential to have an understanding of the XOR gate, Full Adder, Binary Addition, and Binary Subtraction.

For example, we will take two 4-bit binary numbers 'X' and 'Y' for the operation with digits.

$X_0 X_1 X_2 X_3$ for X

$Y_0 Y_1 Y_2 Y_3$ for Y

The Binary Adder-Subtractor is a combination of 4 Full-Adder, which is able to perform the addition and subtraction of 4-bit binary numbers. The control line determines whether the operation being performed is either subtraction or addition. This determination is done by the binary values 0 and 1, which is hold by K.

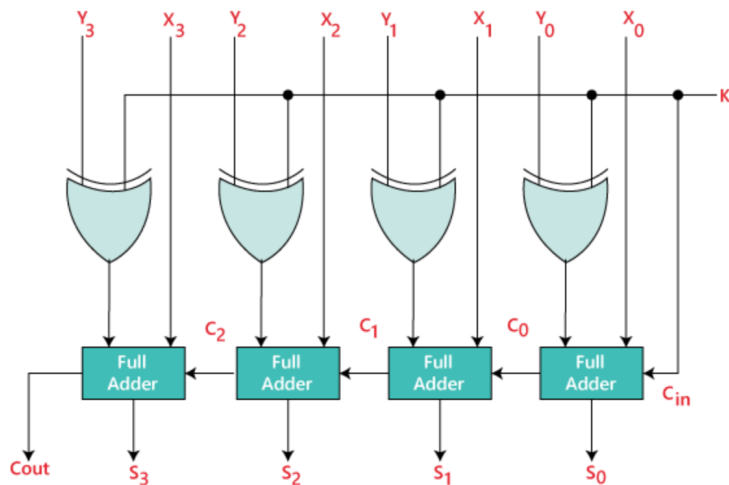


Figure 5.8: Binary Adder-Subtractor Circuit

In the above 5.8, the control lines of the first Full-Adder is directly coming as its input(input carry C_0). The X_0 is the least significant bit of A, which is directly inputted in the Full-Adder. The result produced by performing the XOR operation of Y_0 and K is the third input of the Binary Adder-Subtractor. The sum/difference(S_0) and carry(C_0) are the two outputs produced from the First Full-adder.

When the value of K is set to true or 1, the $Y_0 \oplus K$ produce the complement of Y_0 as the output. So the operation would be $X + Y_0$, which is the 2's complement subtraction of X and Y. It means when the value of K is 1; the subtraction operation is performed by the binary Adder-Subtractor.

In the same way, when the value of K is set to 0, the $Y_0 \oplus K$ produce Y_0 as the output. So the operation would be $X + Y_0$, which is the binary addition of X and Y . It means when the value of K is 0; the addition operation is performed by the binary Adder-Subtractor.

The carry/borrow C_0 is treated as the carry/borrow input for the second Full-Adder. The sum/difference S_0 defines the least significant bit of the sum/difference of numbers X and Y . Just like X_0 , the X_1 , X_2 , and X_3 are faded directly to the 2nd, 3rd, and 4th Full-Adder as an input. The outputs after performing the XOR operation of Y_1 , Y_2 , and Y_3 inputs with K are the third inputs for 2nd, 3rd, and 4th Full-Adder. The carry C_1 , C_2 are passed as the input to the Full-Adder. C_{out} is the output carry of the sum/difference. To form the final result, the S_1 , S_2 , S_3 are recorded with s_0 . We will use n number of Full-Adder to design the n -bit binary Adder-Subtractor.

Decimal or BCD Adder

The BCD-Adder is used in the computers and the calculators that perform arithmetic operation directly in the decimal number system. The BCD-Adder accepts the binary-coded form of decimal numbers. The Decimal-Adder requires a minimum of nine inputs and five outputs.

There is the following table used in designing of BCD-Adder.

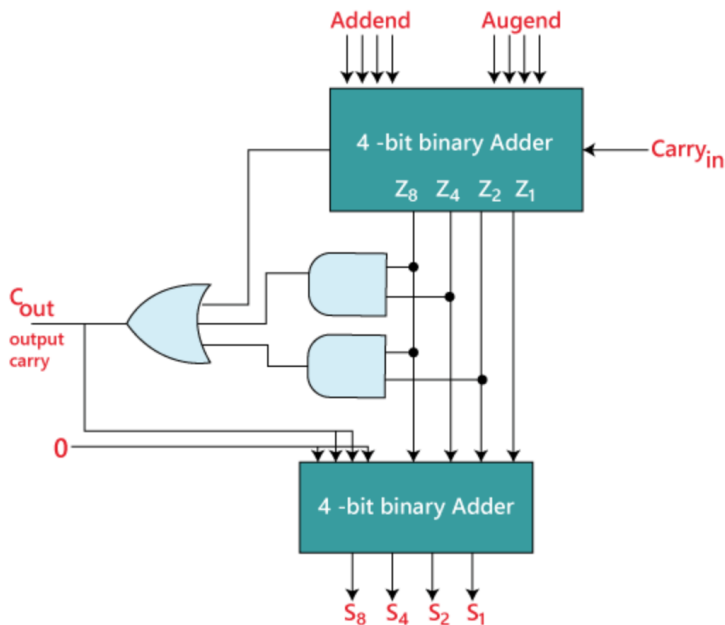
From the above table, it is clear that if the produced sum is between 1 to 9, the Binary and the BCD code is the same. But for 10 to 19 decimal numbers, both the codes are different. In the above table, the binary sum combinations from 10 to 19 give invalid BCD. There are the following points that help the circuit to identify the invalid BCD.

It is obvious from the table that a correction is needed when the

Binary Sum					S A M E C O D E	BCD Sum					Decimal	
K	Z ₈	Z ₄	Z ₂	Z ₁		C	S ₈	S ₄	S ₂	S ₁		
0	0	0	0	0		0	0	0	0	0		0
0	0	0	0	0		0	0	0	0	1		1
0	0	0	1	0		0	0	0	1	0		2
.	.	.	.	.		.	.	.	.	.		.
.	.	.	.	.		.	.	.	.	.		.
.	.	.	.	.		.	.	.	.	.		.
.	.	.	.	.		.	.	.	.	.		.
0	1	0	0	0		0	1	0	0	0		8
0	1	0	0	1	0	1	0	0	1	9		
10 to 19 Binary and BCD codes are not the same												
0	1	0	1	0	1	0	0	0	0	10		
0	1	0	1	1	1	0	0	0	1	11		
0	1	1	0	0	1	0	0	1	0	12		
0	1	1	0	1	1	0	0	1	1	13		
0	1	1	1	0	1	0	1	0	0	14		
0	1	1	1	1	1	0	1	0	1	15		
1	0	0	0	0	1	0	1	1	0	16		
1	0	0	0	1	1	0	1	1	1	17		
1	0	0	1	0	1	1	0	0	0	18		
1	0	0	1	1	1	1	0	0	1	19		

Figure 5.9: BCD-Adder Table

'Binary Sum' has an output carry $K = 1$. The other six combinations from 10 to 15 need correction in which the bit on the Z_8 position is 1. In the Binary sum of 8 and 9, the bit on the Z_8 position is also 1. So, the second step fails, and we need to modify it. To distinguish these two numbers, we specify that the bit on the Z_4 or Z_2 position also needs to be 1 with the bit of Z_8 . The condition for a correction and an output carry can be expressed by the Boolean function: $C = K + Z_8.Z_4 + Z_8.Z_2$. Once the circuit found the invalid BCD, the circuit adds the binary number of 6 into the invalid BCD code to make it valid.

Figure 5.10: *Decimal-to-BCD Block design*

In the above diagram 5.10, We take a 4-bit Binary-Adder, which takes addend and augend bits as an input with an input carry 'Carry in'. The Binary-Adder produces five outputs, i.e., Z₈, Z₄, Z₂, Z₁, and an output carry K. With the help of the output carry K and Z₈, Z₄, Z₂, Z₁ outputs, the logical circuit is designed to identify the C_{out}. The Z₈, Z₄, Z₂, and Z₁ outputs of the binary adder are passed into the 2nd 4-bit binary adder as an Augend. The addend bit of the 2nd 4-bit binary adder is designed in

such a way that the 1st and the 4th bit of the addend number are 0 and the 2nd and the 3rd bit are the same as C_{out} . When the value of C_{out} is 0, the addend number will be 0000, which produce the same result as the 1st 4-bit binary number. But when the value of the C_{out} is 1, the addend bit will be 0110, i.e., 6, which adds with the augend to get the valid BCD number.

5.3 Decoder and Encoder

5.3.1 Decoder:

A combinational circuit that converts binary information into 2^N output lines is called a Decoder. The binary data is provided through N input lines, and the corresponding output lines generate a 2^N -bit code representing the given binary information. Simply put, a Decoder performs the inverse function of an Encoder. To maintain simplicity, only one input line is active at any given time. The resulting 2^N -bit output code corresponds directly to the input binary data.

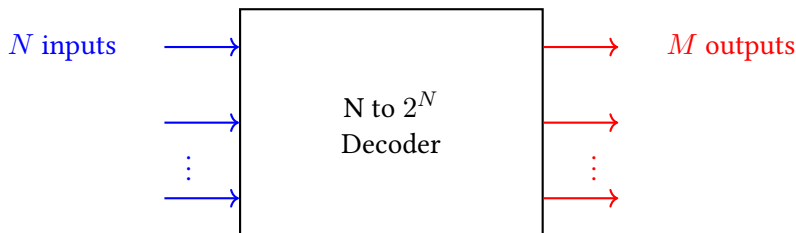


Figure 5.11: Decoder Block Diagram

There are various types of decoders which are as follows:

2 to 4 line decoder:

A 2-to-4 line decoder consists of three inputs: A_0 , A_1 , and an enable signal E , along with four outputs: Y_0 , Y_1 , Y_2 , and Y_3 . When the enable signal E is set to 1, only one of the four outputs will be activated (set to 1) for each unique combination of the input values. The block diagram and truth table of the 2-to-4 line decoder are presented below.

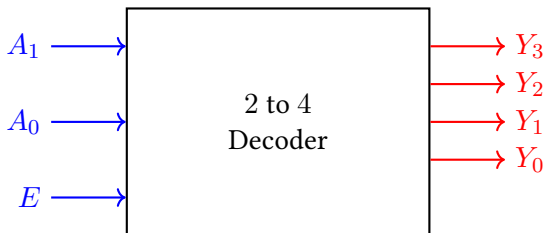


Figure 5.12: 2 to 4 Line Decoder Block Diagram

Truth Table:

Enable (E)	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Table 5.1: Truth Table with Enable, Inputs, and Outputs

The logical expression of the term Y_0 , Y_1 , Y_2 , and Y_3 is as follows:

$$Y_3 = E \cdot A_1 \cdot A_0$$

$$Y_2 = E \cdot A_1 \cdot \overline{A_0}$$

$$Y_1 = E \cdot \overline{A_1} \cdot A_0$$

$$Y_0 = E \cdot \overline{A_1} \cdot \overline{A_0}$$

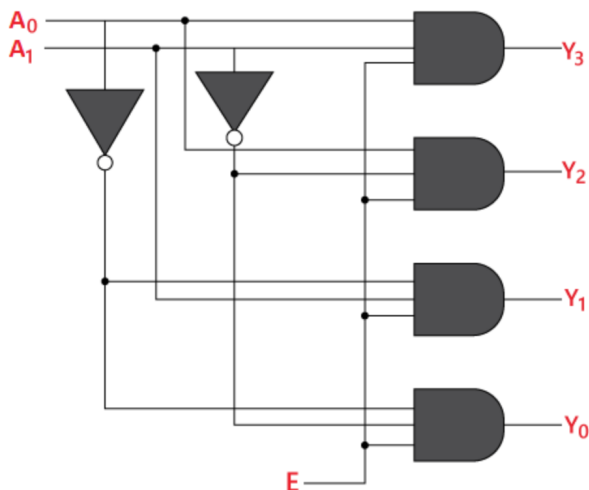


Figure 5.13: 2 to 4 Decoder Gates Design

3 to 8 line decoder:

The 3-to-8 line decoder, also referred to as a Binary-to-Octal Decoder, consists of three input lines A_0 , A_1 , and A_2 , along with eight output lines Y_0 , Y_1 , Y_2 , Y_3 , Y_4 , Y_5 , Y_6 , and Y_7 . This circuit also includes an enable input E . Similar to the 2-to-4 line decoder, when the enable signal E is set to 1, only one of the eight outputs will be activated (set to 1) based on the combination of input values. The block diagram and the truth table of the 3-to-8 line decoder are presented below.

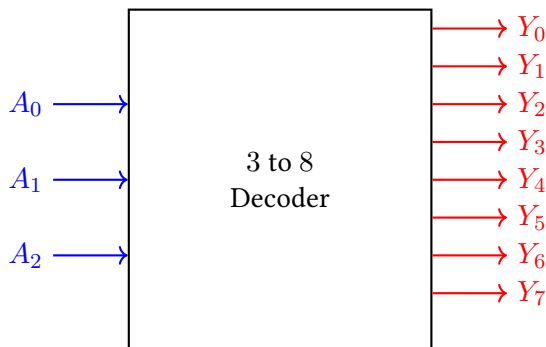


Figure 5.14: 3 to 8 Line Decoder Block Diagram

Enable	INPUTS			Outputs							
E	A ₂	A ₁	A ₀	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

Table 5.2: Enable and Input-to-Output Truth Table

The logical expression of the term Y_0 , Y_1 , Y_2 , Y_3 , Y_4 , Y_5 , Y_6 , and Y_7 is

as follows:

$$Y_0 = \overline{A_0} \cdot \overline{A_1} \cdot \overline{A_2}$$

$$Y_1 = A_0 \cdot \overline{A_1} \cdot \overline{A_2}$$

$$Y_2 = \overline{A_0} \cdot A_1 \cdot \overline{A_2}$$

$$Y_3 = A_0 \cdot A_1 \cdot \overline{A_2}$$

$$Y_4 = \overline{A_0} \cdot \overline{A_1} \cdot A_2$$

$$Y_5 = A_0 \cdot \overline{A_1} \cdot A_2$$

$$Y_6 = \overline{A_0} \cdot A_1 \cdot A_2$$

$$Y_7 = A_0 \cdot A_1 \cdot A_2$$

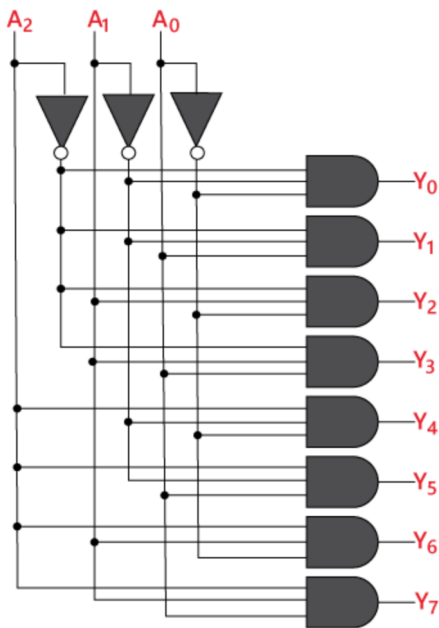


Figure 5.15: 3 to 8 Line Decoder Gates Design

4 to 16 line Decoder

In the 4 to 16 line decoder, there is a total of 16 outputs, i.e., $Y_0, Y_1, Y_2, \dots, Y_{16}$ and four inputs, i.e., A_0, A_1, A_2 , and A_3 . The 3 to 16 line decoder can be constructed using either 2 to 4 decoder or 3 to 8 decoder.

Required number of lower order decoders = m_2/m_1

$m_1 = 8$ and $m_2 = 16$

Required number of 3 to 8 decoders = $\frac{16}{8} = 2$

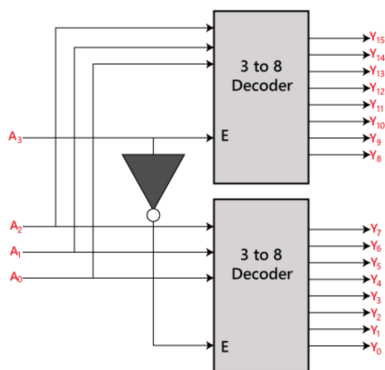


Figure 5.16: 4 to 16 Line Decoder Block Diagram

INPUTS				OUTPUTS															
A ₃	A ₂	A ₁	A ₀	Y ₁₅	Y ₁₄	Y ₁₃	Y ₁₂	Y ₁₁	Y ₁₀	Y ₉	Y ₈	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 5.17: 4 to 16 Line Encoder Truth Table

The logical expression of the term $A_0, A_1, A_2, \dots$,

$$Y_0 = \overline{A_0} \cdot \overline{A_1} \cdot \overline{A_2} \cdot \overline{A_3}$$

$$Y_1 = \overline{A_0} \cdot \overline{A_1} \cdot \overline{A_2} \cdot A_3$$

$$Y_2 = \overline{A_0} \cdot \overline{A_1} \cdot A_2 \cdot \overline{A_3}$$

$$Y_3 = \overline{A_0} \cdot \overline{A_1} \cdot A_2 \cdot A_3$$

$$Y_4 = \overline{A_0} \cdot A_1 \cdot \overline{A_2} \cdot \overline{A_3}$$

$$Y_5 = \overline{A_0} \cdot A_1 \cdot \overline{A_2} \cdot A_3$$

$$Y_6 = \overline{A_0} \cdot A_1 \cdot A_2 \cdot \overline{A_3}$$

$$Y_7 = \overline{A_0} \cdot A_1 \cdot A_2 \cdot A_3$$

$$Y_8 = A_0 \cdot \overline{A_1} \cdot \overline{A_2} \cdot \overline{A_3}$$

$$Y_9 = A_0 \cdot \overline{A_1} \cdot \overline{A_2} \cdot A_3$$

$$Y_{10} = A_0 \cdot \overline{A_1} \cdot A_2 \cdot \overline{A_3}$$

$$Y_{11} = A_0 \cdot \overline{A_1} \cdot A_2 \cdot A_3$$

$$Y_{12} = A_0 \cdot A_1 \cdot \overline{A_2} \cdot \overline{A_3}$$

$$Y_{13} = A_0 \cdot A_1 \cdot \overline{A_2} \cdot A_3$$

$$Y_{14} = A_0 \cdot A_1 \cdot A_2 \cdot \overline{A_3}$$

$$Y_{15} = A_0 \cdot A_1 \cdot A_2 \cdot A_3$$

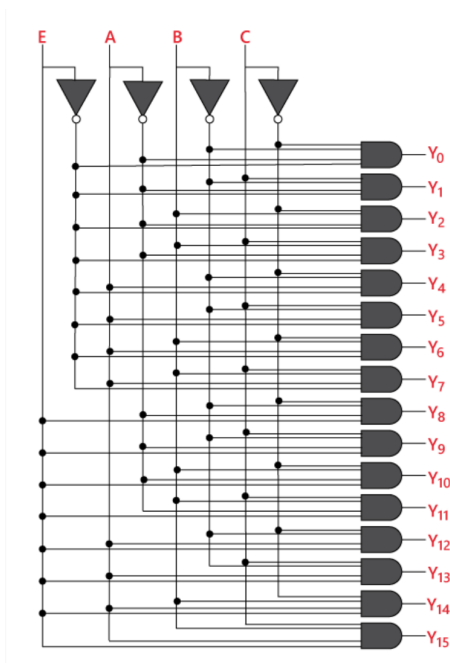


Figure 5.18: 4 to 16 Line Encoder Gates Design

5.3.2 Encoder:

The combinational circuits that change the binary information into N output lines are known as Encoders. The binary information is passed in the form of 2^N input lines. The output lines define the N -bit code for the binary information. In simple words, the Encoder performs the reverse operation of the Decoder. At a time, only one input line is activated for

simplicity. The produced N -bit output code is equivalent to the binary information.

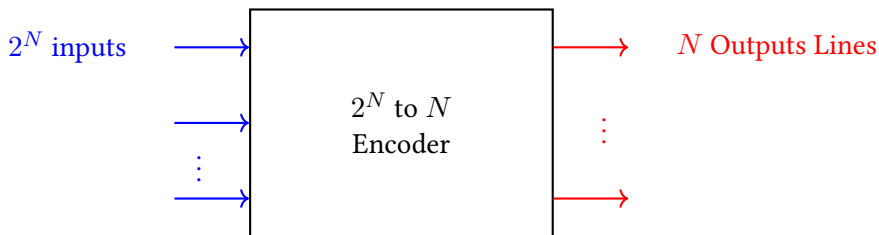


Figure 5.19: Encoder Block Diagram

There are various types of encoders which are as follows:

4 to 2 line Encoder:

In 4 to 2 line encoder, there are total of four inputs, i.e., Y_0 , Y_1 , Y_2 , and Y_3 , and two outputs, i.e., A_0 and A_1 . In 4-input lines, one input-line is set to true at a time to get the respective binary code in the output side. Below are the block diagram and the truth table of the 4 to 2 line encoder.

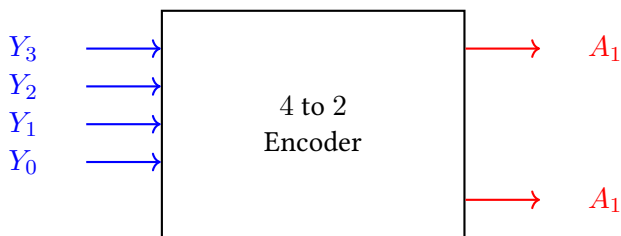


Figure 5.20: 4 to 2 Encoder Block Diagram

INPUTS				OUTPUTS	
Y_3	Y_2	Y_1	Y_0	A_1	A_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

Table 5.3: Truth table for 4 to 2 Encoder inputs and outputs

The logical expression of the term A_0 and A_1 is as follows:

$$A_1 = Y_3 + Y_2$$

$$A_0 = Y_3 + Y_1$$

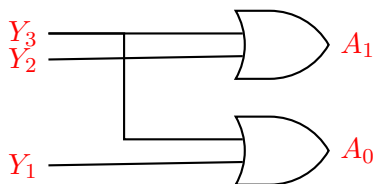


Figure 5.21: 4 to 2 Encoder Logic Gates Design

8 to 3 line Encoder:

The 8 to 3 line Encoder is also known as Octal to Binary Encoder. In 8 to 3 line encoder, there is a total of eight inputs, i.e., Y_0 , Y_1 , Y_2 , Y_3 , Y_4 , Y_5 , Y_6 , and Y_7 and three outputs, i.e., A_0 , A_1 , and A_2 . In 8-input lines, one input-line is set to true at a time to get the respective binary code in the output side. Below are the block diagram and the truth table of the 8 to 3 line encoder.

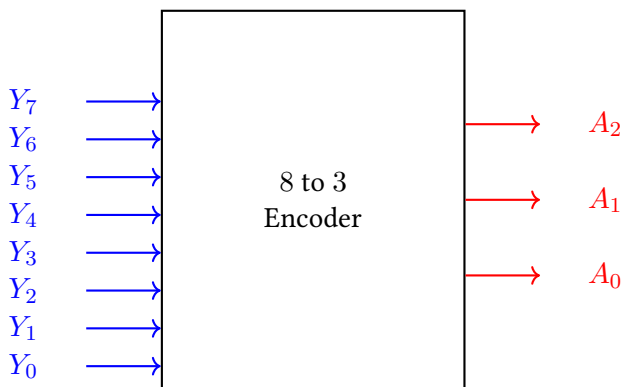


Figure 5.22: 8 to 3 Encoder Block Diagram

Truth Table

INPUTS								OUTPUTS		
Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Table 5.4: Truth Table

Logical Expressions

$$A_2 = Y_4 + Y_5 + Y_6 + Y_7$$

$$A_1 = Y_2 + Y_3 + Y_6 + Y_7$$

$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$

Logical circuit of the above expressions is given below:

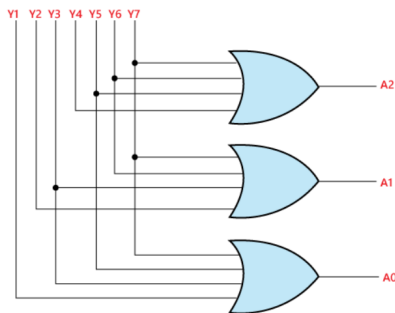


Figure 5.23: 8 to 3 Line Encoder Gates Design

Decimal to BCD Encoder

The Octal to Binary Encoder is also known as **10 to 4 line Encoder**. In a 10 to 4 line encoder, there are a total of ten inputs, i.e., $Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, Y_7, Y_8$, and Y_9 and four outputs, i.e., A_0, A_1, A_2 , and A_3 . In 10-input lines, one input line is set to true at a time to get the respective **BCD code** in the output side.

The block diagram and the truth table of the decimal to BCD encoder are given below.

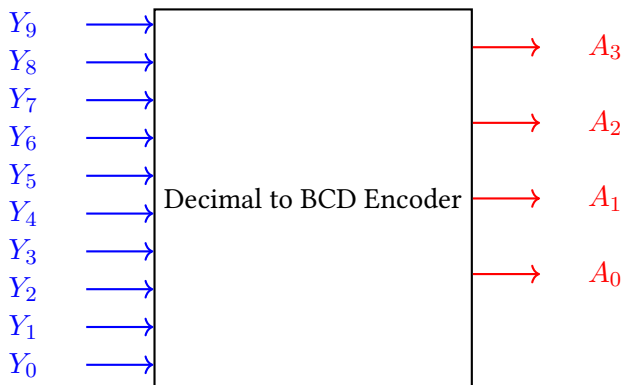


Figure 5.24: *Decimal to BCD Encoder Block Diagram*

Truth Table

INPUTS										OUTPUTS			
Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_3	A_2	A_1	A_0
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	1	0	0	1

Table 5.5: Truth Table for Decimal to BCD Encoder

Logical Expressions

$$A_3 = Y_9 + Y_8$$

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_0 = Y_9 + Y_7 + Y_5 + Y_3 + Y_1$$

Logical circuit of the above expressions is given below:

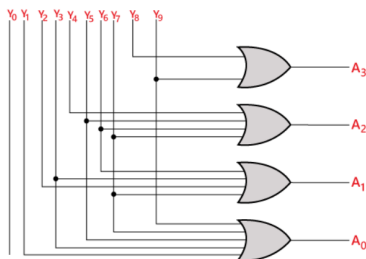


Figure 5.25: Decimal to BCD Line Encoder Gates Design

5.4 Multiplexer And De-Multiplexer

5.4.1 Multiplexer

A multiplexer is a combinational circuit that consists of 2^n input lines and a single output line. Essentially, it is a circuit that takes multiple inputs but produces a single output. The binary data from the input lines is transmitted to the output line based on the values of the selection lines, which determine which input is routed to the output.

Unlike encoders and decoders, a multiplexer has n selection lines and 2^n input lines, leading to a total of 2^n possible input combinations. A multiplexer is commonly referred to as a **Mux**.

There are different types of multiplexers, which include:

2×1 Multiplexer

A 2×1 multiplexer consists of two input lines, A_0 and A_1 , a single selection line, S_0 , and one output line, Y . Depending on the value of the selection line S_0 , one of the two input lines is directed to the output. The

block diagram and truth table of the 2×1 multiplexer are presented below.

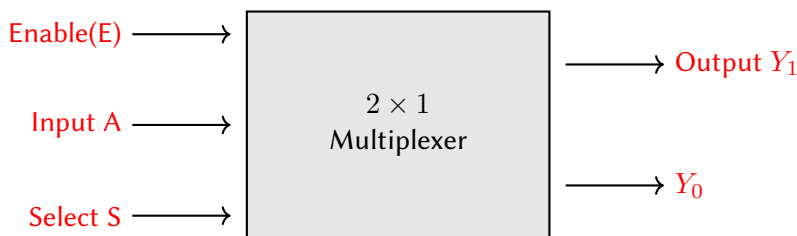


Figure 5.26: 2×1 Multiplexer

Truth Table

Inputs (S_0)	Output (Y)
0	A_0
1	A_1

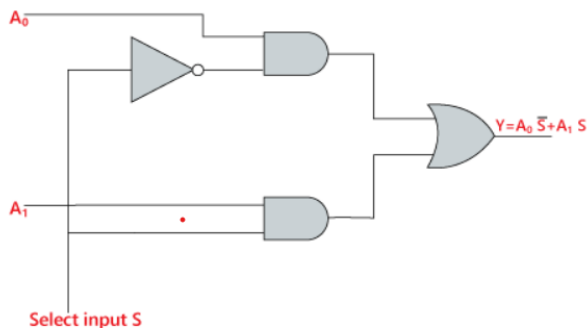
Table 5.6: Truth Table for 2×1 Multiplexer

Logical Expression

The logical expression of the term Y is as follows:

$$Y = S'_0 \cdot A_0 + S_0 \cdot A_1$$

Logical circuit of the above expression is given below:

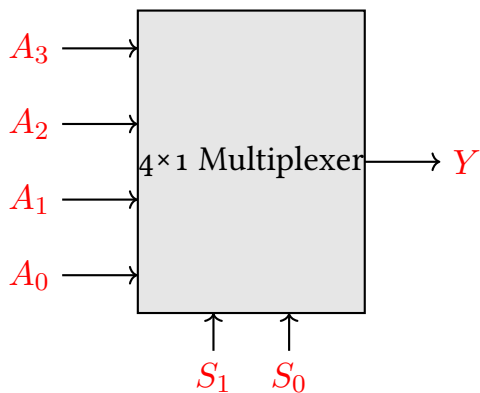
Figure 5.27: 2×1 Multiplexer

4×1 Multiplexer

A 4×1 multiplexer consists of four input lines: A_0 , A_1 , A_2 , and A_3 , two selection lines: S_0 and S_1 , and a single output line, Y . The selection lines S_0 and S_1 determine which one of the four input lines is routed to the output.

The block diagram and truth table of the 4×1 multiplexer are provided below.

Block Diagram



Truth Table

INPUTS		OUTPUTS
S_1	S_0	Y
0	0	A_0
0	1	A_1
1	0	A_2
1	1	A_3

Table 5.7: Truth Table for 4×1 Multiplexer

Logical Expression

$$Y = S_1' S_0' A_0 + S_1' S_0 A_1 + S_1 S_0' A_2 + S_1 S_0 A_3$$

Logical circuit of the above expression is given below:

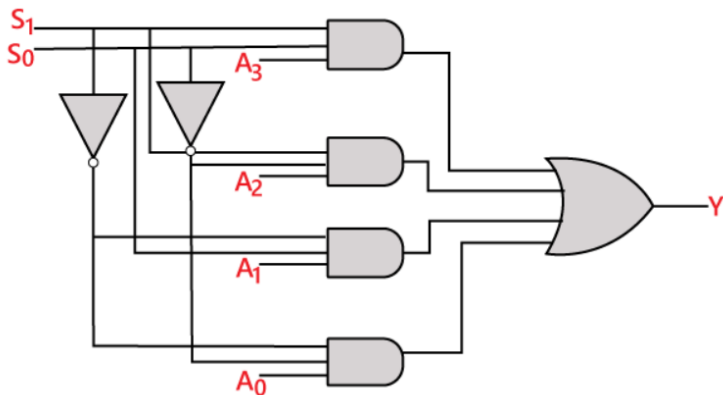


Figure 5.28: 4×1 Multiplexer

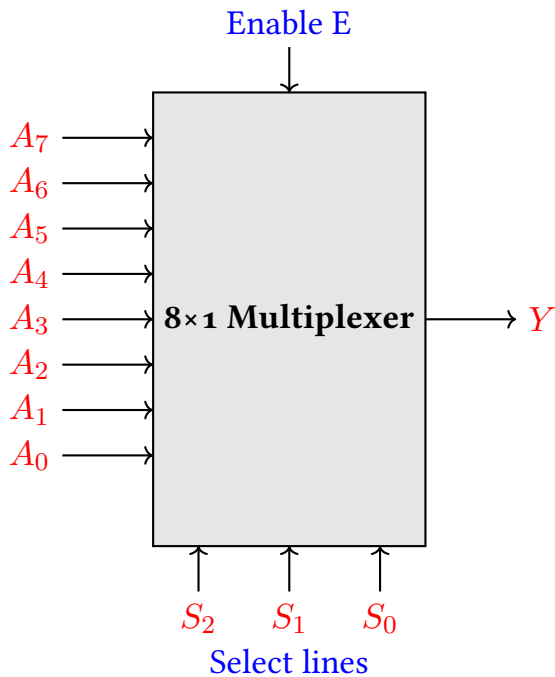
8×1 Multiplexer

An 8×1 multiplexer consists of eight input lines: $A_0, A_1, A_2, A_3, A_4, A_5, A_6$, and A_7 , three selection lines: S_0, S_1 , and S_2 , and a single output line, Y .

Based on the combination of values applied to the selection lines S_0, S_1 , and S_2 , one of the eight input lines is directed to the output.

The block diagram and truth table of the 8×1 multiplexer are provided below.

section*Block Diagram



Truth Table

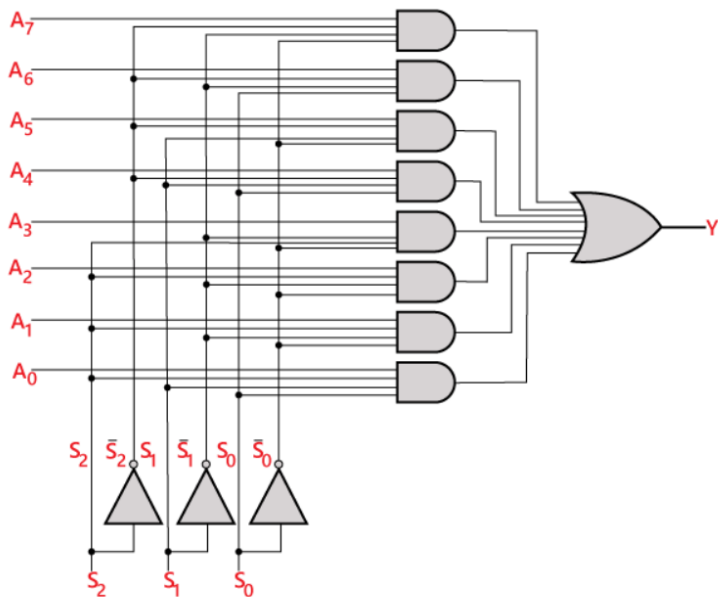
INPUTS			OUTPUTS
S_2	S_1	S_0	Y
0	0	0	A_0
0	0	1	A_1
0	1	0	A_2
0	1	1	A_3
1	0	0	A_4
1	0	1	A_5
1	1	0	A_6
1	1	1	A_7

Table 5.8: Truth Table for 8×1 Multiplexer

Logical Expression

$$Y = S'_0 S'_1 S'_2 A_0 + S_0 S'_1 S'_2 A_1 + S'_0 S_1 S'_2 A_2 + S_0 S_1 S'_2 A_3 + S'_0 S'_1 S_2 A_4 + S_0 S'_1 S_2 A_5 + S'_0 S_1 S_2 A_6 + S_0 S_1 S_2 A_7$$

Logical circuit of the above expression is given below:

Figure 5.29: 8×1 Multiplexer

8×1 Multiplexer Using 4×1 and 2×1 Multiplexers

An 8×1 multiplexer can be implemented using smaller multiplexers. Specifically, it can be constructed using two 4×1 multiplexers and one 2×1 multiplexer. A 4×1 multiplexer consists of two selection lines, four input lines, and a single output. Meanwhile, a 2×1 multiplexer has only one selection line.

To accommodate all eight data inputs, two 4×1 multiplexers are required. Each of these multiplexers generates an output, which is then processed by a 2×1 multiplexer to produce the final output. The block diagram illustrating the implementation of an 8×1 multiplexer using 4×1 and 2×1 multiplexers is presented below.

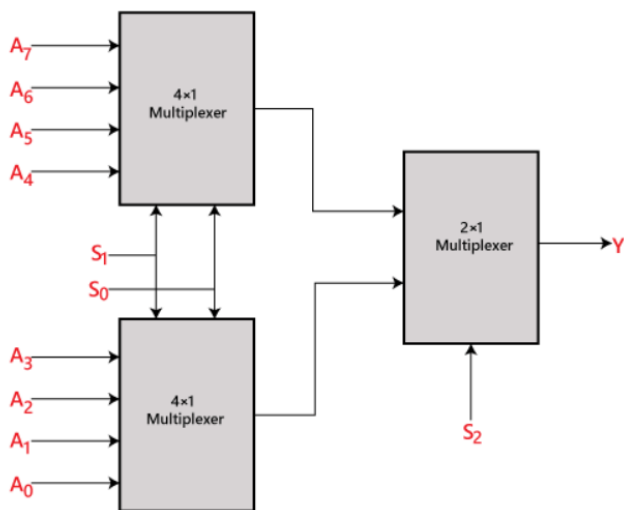


Figure 5.30: 8×1 Multiplexer

5.4.2 De-Multiplexer:

A De-multiplexer is a combinational circuit that consists of a single input line and 2^N output lines. Essentially, it is a single-input and multiple-output combinational circuit. The data is received through the single input line and then routed to one of the output lines based on the values of the selection lines. A De-multiplexer functions as the inverse of a multiplexer.

Unlike encoders and decoders, a De-multiplexer has n selection lines and 2^n output lines, resulting in 2^n possible output combinations. It is commonly referred to as a **De-mux**.

There are different types of De-multiplexers, which include:

1×2 De-multiplexer:

A 1×2 De-multiplexer consists of a single input A , one selection line S_0 , and two output lines: Y_0 and Y_1 . Based on the value of the selection line S_0 , the input signal is routed to one of the two outputs.

The block diagram and truth table of the 1×2 De-multiplexer are provided below.

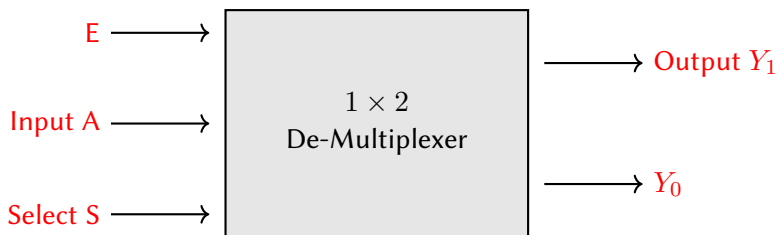


Figure 5.31: 1×2 De-Multiplexer

INPUTS		Output	
S_0		Y_1	Y_0
0		0	A
1		A	0

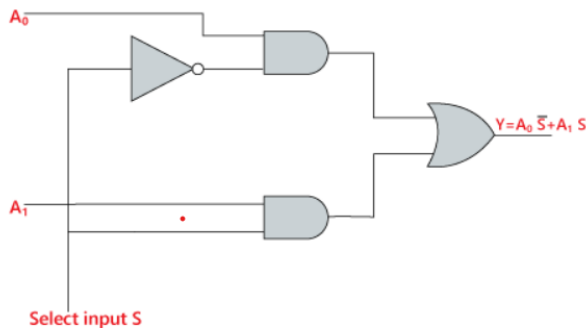
Table 5.9: Truth Table for De-Multiplexer

The logical expression of the term Y is as follows:

$$Y_0 = \overline{S_0} \cdot A$$

$$Y_1 = S_0 \cdot A$$

Logical circuit of the above expressions is given below:

Figure 5.32: 2×1 Multiplexer

1×4 De-multiplexer

A 1×4 De-multiplexer consists of a single input A , two selection lines S_0 and S_1 , and four output lines: Y_0 , Y_1 , Y_2 , and Y_3 . Depending on the combination of values applied to the selection lines S_0 and S_1 , the input A is directed to one of the four outputs.

The block diagram and truth table of the 1×4 De-multiplexer are presented below.

Block Diagram

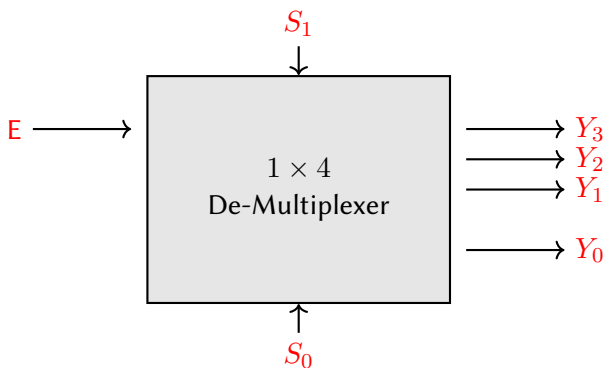


Figure 5.33: 1×4 De-Multiplexer

Truth Table

S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	A
0	1	0	0	A	0
1	0	0	A	0	0
1	1	A	0	0	0

Table 5.10: *Truth Table for 1×4 De-multiplexer*

The logical expressions for the terms Y are as follows:

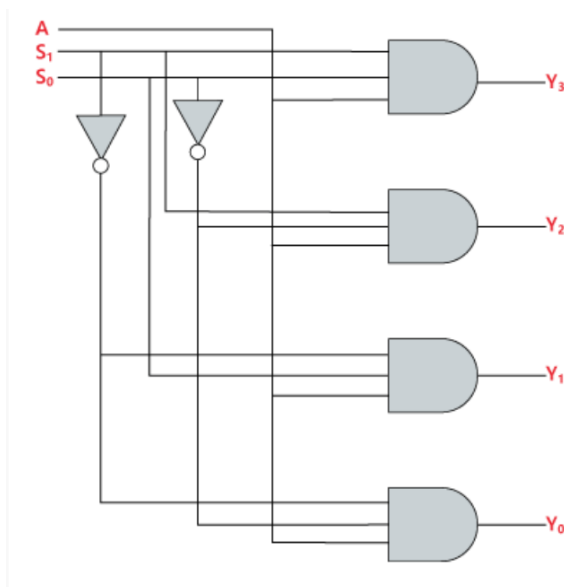
$$Y_0 = \overline{S_1} \cdot \overline{S_0} \cdot A$$

$$Y_1 = \overline{S_1} \cdot S_0 \cdot A$$

$$Y_2 = S_1 \cdot \overline{S_0} \cdot A$$

$$Y_3 = S_1 \cdot S_0 \cdot A$$

Logical circuit of the above expressions is given below:

Figure 5.34: Logical Circuit of 1×4 De-Multiplexer

1×8 De-multiplexer

A 1×8 De-multiplexer consists of a single input A , three selection lines S_0 , S_1 , and S_2 , and eight output lines: Y_0 , Y_1 , Y_2 , Y_3 , Y_4 , Y_5 , Y_6 , and Y_7 . Depending on the combination of values applied to the selection lines S_0 , S_1 , and S_2 , the input signal is routed to one of the eight outputs.

The block diagram and truth table of the 1×8 De-multiplexer are presented below.

Block Diagram

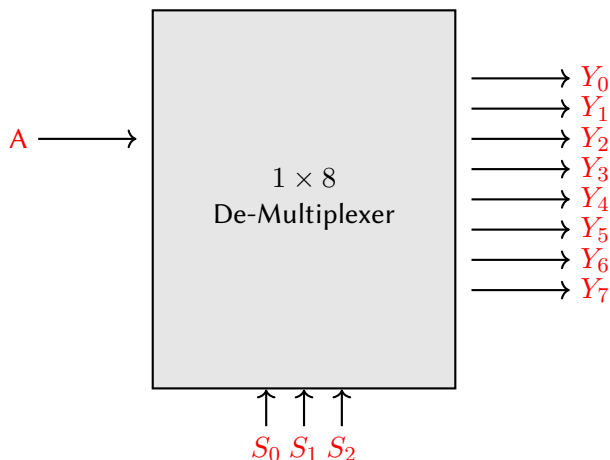


Figure 5.35: 1 × 8 De-Multiplexer

INPUTS			OUTPUT							
S ₂	S ₁	S ₀	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	0	0	0	A
0	0	1	0	0	0	0	0	0	A	0
0	1	0	0	0	0	0	0	A	0	0
0	1	1	0	0	0	0	A	0	0	0
1	0	0	0	0	0	A	0	0	0	0
1	0	1	0	0	A	0	0	0	0	0
1	1	0	0	A	0	0	0	0	0	0
1	1	1	A	0	0	0	0	0	0	0

Table 5.11: Truth Table for 1×8 De-multiplexer

Logical Expressions

The logical expressions for the terms Y are as follows:

$$Y_0 = \overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0} \cdot A$$

$$Y_1 = \overline{S_2} \cdot \overline{S_1} \cdot S_0 \cdot A$$

$$Y_2 = \overline{S_2} \cdot S_1 \cdot \overline{S_0} \cdot A$$

$$Y_3 = \overline{S_2} \cdot S_1 \cdot S_0 \cdot A$$

$$Y_4 = S_2 \cdot \overline{S_1} \cdot \overline{S_0} \cdot A$$

$$Y_5 = S_2 \cdot \overline{S_1} \cdot S_0 \cdot A$$

$$Y_6 = S_2 \cdot S_1 \cdot \overline{S_0} \cdot A$$

$$Y_7 = S_2 \cdot S_1 \cdot S_0 \cdot A$$

Logical Circuit

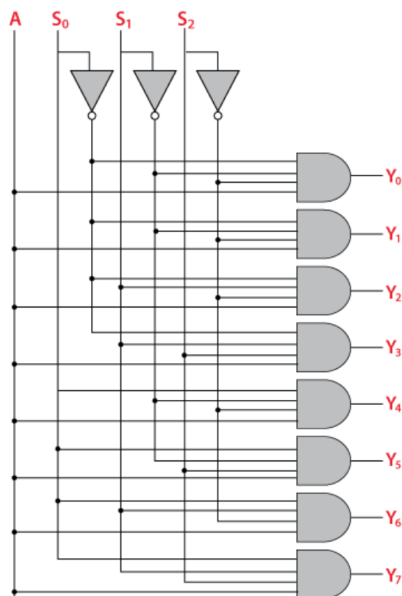
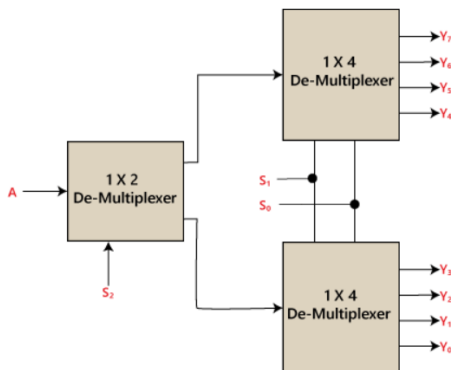


Figure 5.36: Logical Circuit for 1 × 8 de-multiplexer

1×8 De-multiplexer using 1×4 and 1×2 De-multiplexer

A 1×8 De-multiplexer can be constructed using lower-order De-multiplexers. Specifically, it can be implemented using two 1×4 De-multiplexers and one 1×2 De-multiplexer. The 1×4 De-multiplexer consists of two selection lines, four output lines, and a single input. The 1×2 De-multiplexer, on the other hand, has only one selection line.

To obtain eight output lines, two 1×4 De-multiplexers are required. The 1×2 De-multiplexer generates two outputs, which are then used as inputs for the two 1×4 De-multiplexers to produce the final set of outputs. The block diagram illustrating the implementation of a 1×8 De-multiplexer using 1×4 and 1×2 De-multiplexers is provided below.

Figure 5.37: 1×8 de-multiplexer

5.5 Exercises

- For the full-adder determine the outputs for each of the following inputs:
 - $A = 0, B = 1, C_{in} = 0$
 - $A = 1, B = 0, C_{in} = 1$
 - $A = 0, B = 0, C_{in} = 0$
- What are the half-adder inputs that will produce the following outputs:
 - $\Sigma = 0, C_{out} = 0$

b) $\Sigma = 1, C_{\text{out}} = 0$

c) $\Sigma = 0, C_{\text{out}} = 1$

3. Determine the outputs of a full-adder for each of the following inputs:

a) $A = 1, B = 0, C_{\text{in}} = 0$

b) $A = 0, B = 0, C_{\text{in}} = 1$

c) $A = 0, B = 1, C_{\text{in}} = 1$

d) $A = 1, B = 1, C_{\text{in}} = 1$

4. Show the decoding logic for each of the following codes if an active-HIGH (1) output is required:

a) 1101

b) 1000

c) 000101

d) 1110110

e) 101010

f) 111110

g) 11011

h) 11100

5. Using a decoder and external gates, design the combinational circuit defined by the following three Boolean functions:

(a)

$$F_1 = x'y'z' + xz$$

$$F_2 = xy'z' + x'y$$

$$F_3 = x'y'z' + xy$$

(b)

$$F_1 = (y' + x)z$$

$$F_2 = y'z' + x'y + yz'$$

$$F_3 = (x + y)z$$

6. For the decimal-to-BCD encoder logic, assume that the 9 input and the 3 inputs are both HIGH. What is the output code? Is it a valid BCD (8421) code?

7. Implement the following Boolean function with a multiplexer :

a) $F(A, B, C, D) = \Sigma(0, 2, 5, 8, 10, 14)$

b) $F(A, B, C, D) = \Pi(2, 6, 11)$

8. Implement a full adder with two 4×1 multiplexers.

An 8×1 multiplexer has inputs A , B , and C connected to the selection inputs S_2 , S_1 , and S_0 , respectively. The data inputs I_0 through I_7 are as follows:

a) $I_0 = I_2 = I_7 = 0$; $I_3 = 1$; $I_5 = 1$; $I_6 = A'$; $I_4 = D$; $I_1 = D'$

$$\text{b) } I_1 = I_2 = 0; I_3 = I_7 = 1; I_4 = I_5 = D; I_6 = D'$$

Determine the Boolean function that the multiplexer implements.

9. Implement the following Boolean function with a 4×1 multiplexer and external gates:

$$\text{a) } F(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$$

$$\text{b) } F(A, B, C, D) = \Sigma(1, 2, 5, 7, 8, 10, 13, 15)$$

Connect inputs A and B to the selection lines. The input requirements for the four data lines will be a function of variables C and D . These values are obtained by expressing F as a function of C and D for each of the four cases when $AB = 00, 01, 10$, and 11 . These functions may have to be implemented with external gates.

Chapter 6

Sequential Circuits

Introduction

In our previous sections, we learned about combinational circuits and their working. The combinational circuits have a set of outputs, which depend only on the present combination of inputs. Below is the block diagram of the synchronous logic circuit.

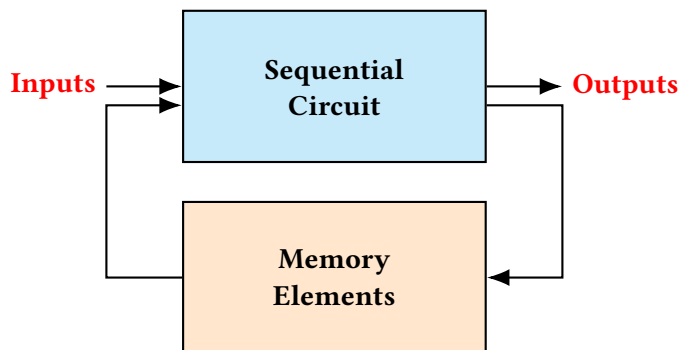


Figure 6.1: *Sequential Circuit Block Diagram*

6.1 Sequential Circuits

The sequential circuit is a special type of circuit that has a series of inputs and outputs. The outputs of the sequential circuits depend on both the combination of present inputs and previous outputs. The previous output is treated as the present state. So, the sequential circuit contains the combinational circuit and its memory storage elements. A sequential circuit doesn't need to always contain a combinational circuit. Thus, the sequential circuit can contain only the memory element.

Difference between Combinational Circuits and Sequential Circuits

Combinational Circuits	Sequential Circuits
The outputs of the combinational circuit depend only on the present inputs.	The outputs of the sequential circuits depend on both present inputs and the present state (previous output).
The feedback path is not present in the combinational circuit.	The feedback path is present in sequential circuits.
In combinational circuits, memory elements are not required.	In sequential circuits, memory elements play an important role and are required.
The clock signal is not required for combinational circuits.	The clock signal is required for sequential circuits.
The combinational circuit is simple to design.	It is not simple to design a sequential circuit.

Table 6.1: *Differences between Combinational Circuits and Sequential Circuits*

Types of Sequential Circuits

Asynchronous Sequential Circuits

The clock signals are not used by the asynchronous sequential circuits. The asynchronous circuit is operated through the pulses. So, changes in the input can change the state of the circuit. The asynchronous circuits

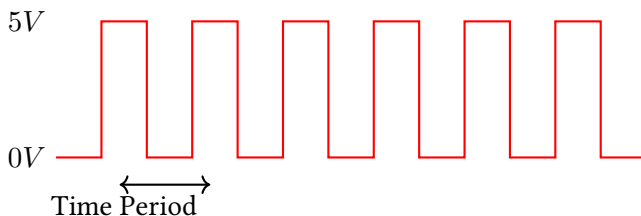


Figure 6.2: Clock Signal Diagram

do not use clock pulses. The internal state is changed when the input variable is changed. The un-clocked flip-flops or time-delayed elements are the memory elements of asynchronous sequential circuits. The asynchronous sequential circuit is similar to the combinational circuits with feedback.

Clock Signal and Triggering

Clock Signal

A clock signal is a periodic signal in which **ON time** and **OFF time** need not be the same. When the **ON time** and **OFF time** of the clock signal are the same, a square wave is used to represent the clock signal. Below is a diagram which represents the clock signal:

A clock signal is considered as the **square wave**. Sometimes, the signal stays at logic, either high (5V) or low (0V), for an equal amount of time. It repeats with a certain **time period**, which will be equal to twice the 'ON time' or 'OFF time'.

Types of Triggering

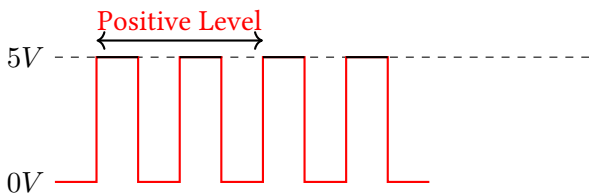
These are two types of triggering in sequential circuits:

Level Triggering

The logic **High** and logic **Low** are the two levels in the clock signal. In level triggering, when the clock pulse is at a particular level, only then the circuit is activated. There are the following types of level triggering:

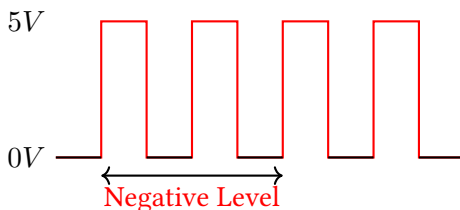
Positive Level Triggering

In a **positive level triggering**, the signal with Logic High occurs. So, in this triggering, the circuit is operated with such type of clock signal. Below is the diagram of positive level triggering:



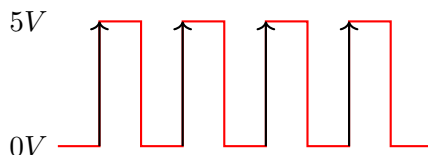
Negative Level Triggering

In **negative level triggering**, the signal with Logic Low occurs. So, in this triggering, the circuit is operated with such type of clock signal. Below is the diagram of Negative Level Triggering:



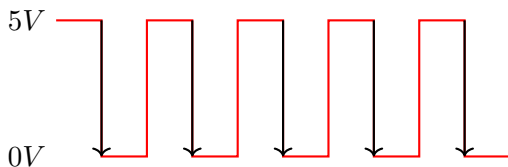
Positive Edge Triggering

The transition from **Logic Low to Logic High** occurs in the clock signal of positive edge triggering. So, in positive edge triggering, the circuit is operated with such type of clock signal. Below is the diagram of Positive Edge Triggering:



Negative Edge Triggering

The transition from **Logic High to Logic Low** occurs in the clock signal of negative edge triggering. So, in negative edge triggering, the circuit is operated with such type of clock signal. The diagram of negative edge triggering is given below:



6.2 Flip-Flops: SR, JK, D, T

Basics of Flip Flop

A circuit that has two stable states is treated as a **flip flop**. These stable states are used to store binary data that can be changed by applying varying inputs. Flip flops are the fundamental building blocks of the digital system. Flip flops and latches are examples of data storage elements. In the sequential logical circuit, the flip flop is the basic storage element.

The latches and flip flops are the basic storage elements but differ in working. There are the following types of flip flops:

SR Flip Flop

The **S-R flip flop** is the most common flip flop used in the digital system. In an SR flip flop:

- When the set input **S** is true, the output Y will be **high**, and Y' will be **low**.
- It is required that the wiring of the circuit is maintained when the outputs are established.
- The wiring must be maintained until the set or reset input goes **high**, or the power is shut down.

The SR flip flop is a 1-bit memory bistable device having two inputs, i.e., **SET** and **RESET**. The SET input (**S**) sets the device or produces the output 1, and the RESET input (**R**) resets the device or produces the output 0. The SET and RESET inputs are labeled as **S** and **R**, respectively.

The SR flip flop stands for "**Set-Reset**" flip flop. The reset input is used to bring the flip flop back to its original state from the current state with an output **Q**. This output depends on the set and reset conditions, which are either at the logic level "0" or "1".

The NAND gate SR flip flop is a basic flip flop that provides feedback from both of its outputs back to its opposing input. This circuit is used to store a single data bit in the memory circuit. Hence, the SR flip flop has three inputs: **S**, **R**, and the current output **Q**. This output **Q** relates to the current history or state. The term "**flip-flop**" relates to the actual operation of the device, as it can be "**flipped**" to a logic set state or "**flopped**" back to the opposing logic reset state.

The NAND Gate SR Flip-Flop

We can implement the set-reset flip flop by connecting two cross-coupled 2-input NAND gates together. In the SR flip flop circuit, feedback is connected from each output to one of the inputs of the other NAND gate. Thus, the device has two inputs: **Set (S)** and **Reset (R)** with two outputs **Q** and $\overline{Q}$. Below is the block diagram and circuit diagram of the SR flip flop.



Figure 6.3: SR-Flip Flop Block Diagram

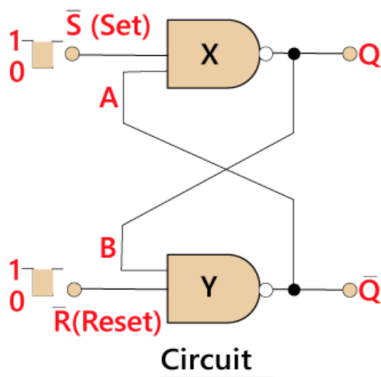
Circuit Diagram:

Figure 6.4: NAND Gate SR-Flip Flop Diagram

Set State

In the circuit diagram, when the input R is set to **false** (0) and S is set to **true** (1), the NAND gate Y receives 0 as one of its inputs, producing the output $\overline{Q} = 1$. This value of $\overline{Q}$ is provided as input A to the NAND gate X . Since both inputs to NAND gate X are now 1 ($S = A = 1$), the output becomes $Q = 0$.

If the input R is then changed to 1 while S remains 1, the inputs to NAND gate Y are $R = 1$ and $B = 0$. As one of the inputs is 0, the output $\overline{Q}$ remains 1. Therefore, the flip-flop circuit is **set** or latched with $Q = 0$ and $\overline{Q} = 1$.

Reset State

In the reset state, the output $\overline{Q} = 0$ and $Q = 1$. This state occurs when $R = 1$ and $S = 0$. Here, one of the inputs to NAND gate X is 0, resulting in the output $Q = 1$. The output Q is then fed as input B to NAND gate Y . With both inputs to NAND gate Y being 1, the output $\overline{Q}$ becomes 0.

If the input S remains 0 while R is held at 1, the output $\overline{Q}$ stays at 0, leaving the state unchanged. Thus, the flip-flop circuit is latched in the **reset state**, with the set/reset behaviors summarized in the following truth table.

SR Flip Flop Truth Table

The truth table for the SR flip-flop is shown below:

S	R	Q	$\overline{Q}$
0	0	Invalid	Invalid
0	1	0	1
1	0	1	0
1	1	No Change	No Change

Table 6.2: Truth Table for SR Flip Flop

From the above truth table, we can see that when set S and reset R inputs are set to 1, the outputs Q and $\overline{Q}$ will either be 1 or 0. These outputs depend on the previous input state of S or R . Therefore, when the inputs are 1, the output states remain unchanged.

The condition where both input states are set to 0 is treated as invalid and must be avoided.

JK Flip Flop

The **SR Flip Flop**, also known as the **Set-Reset Flip Flop**, has several advantages but is limited by the following drawbacks:

- When both the Set (S) and Reset (R) inputs are 0, this condition must always be avoided.
- If the Set or Reset inputs change state while the enable input is active (1), it can lead to incorrect latching behavior.

The **JK Flip Flop** addresses these two limitations of the SR Flip Flop.

The JK Flip Flop is one of the most widely used flip flops in digital circuits. It is a versatile flip flop with two inputs: J and K . In the SR Flip Flop, S and R denote Set and Reset, respectively. However, the labels J and K are chosen to distinguish the JK Flip Flop from other designs.

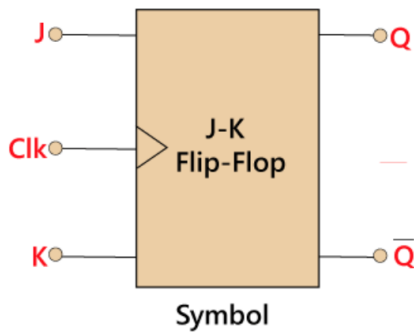
The JK Flip Flop operates similarly to the SR Flip Flop, with J and K replacing S and R , respectively. Unlike the SR Flip Flop, the JK Flip Flop does not encounter invalid states even when both J and K are set to 1.

Working of JK Flip Flop

The JK Flip Flop is essentially a gated SR Flip Flop with additional clock input circuitry. The clock input eliminates the invalid or undefined output condition that occurs when both inputs are set to 1. The JK Flip Flop operates with four possible input states: 1, 0, "no change," and "toggle." The symbol for the JK Flip Flop is similar to the SR Bistable Latch, with the addition of a clock input.

Block Diagram

The block diagram of the JK Flip Flop is shown below:

Figure 6.5: *JK Block Diagram*

Circuit Diagram

In the JK Flip Flop circuit, the S and R inputs of the SR Flip Flop are replaced by J and K inputs, respectively. Two 2-input AND gates are replaced by two 3-input NAND gates. The third input of each gate is connected to the outputs Q and Q' . The cross-coupling of the SR Flip Flop allows the invalid condition ($S = 1, R = 1$) to produce the "toggle action" as the two inputs are interlocked.

The circuit diagram of the JK Flip Flop is shown below:

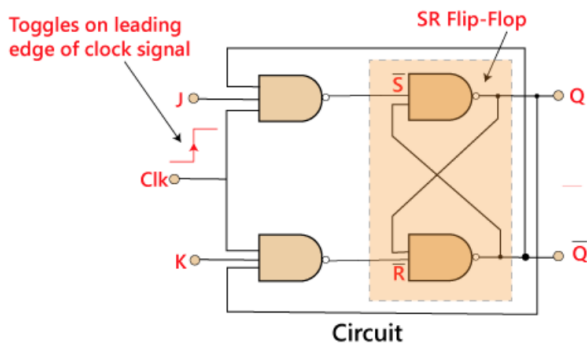
Circuit Diagram:

Figure 6.6: JK Flip Flop Gate Diagram

Truth Table of JK Flip Flop

When both inputs of the **JK Flip Flop** are set to 1 and the clock input receives a "High" pulse, the circuit alternates between the **SET** and **RESET** states. In this configuration, the JK Flip Flop functions as a **T-type toggle flip flop**. The corresponding truth table is provided below.

Same as for SR Latch	Clock	Input		Output		Description
	Clk	J	K	Q	Q'	
	X	0	0	1	0	Memory no change
	X	0	0	0	1	
	$\downarrow$	0	1	1	0	Reset $Q \gg 0$
	X	0	1	0	1	
	$\downarrow$	1	0	0	1	Set $Q \gg 1$
	X	1	0	1	0	
Toggle action	$\downarrow$	1	1	0	1	Toggle
	$\downarrow$	1	1	1	0	

When both inputs of the **JK flip flop** are set to 1 and the clock input receives a "High" pulse, the circuit toggles between the **SET** and **RESET** states. In this scenario, the JK flip flop behaves like a **T-type toggle flip flop**.

The JK flip flop is an enhanced version of the clocked SR flip flop. However, it is still prone to the "race" condition. This issue occurs when the output state Q changes before the clock input's timing pulse turns "Off." To prevent this, the duration of the timing pulse (T) must be kept short.

D Flip Flop

In the **SR NAND Gate Bistable circuit**, the undefined input condition where $SET = 0$ and $RESET = 0$ is considered invalid. This state has the following issues:

- Disrupts the feedback latching mechanism.
- Forces both outputs to be **1** simultaneously.

- Results in a loss of input control, as the first input to change to **1** determines the latch's state, leaving the other input at **0**.

To eliminate this problem, an **inverter** is introduced. The inverter is connected between the Set and Reset inputs, forming another type of flip flop called the **D flip flop**, also known as the **Delay flip flop**, **D-type Bistable**, or **D-type flip flop**.

The **D flip flop** is one of the most significant types of clocked flip flops. It ensures that the inputs (S and R) are never equal to **1** simultaneously. This flip flop is created using a gated SR flip flop with an inverter placed between the inputs, resulting in a single input labeled D (Data).

This single data input D replaces the "Set" input, while the complementary "Reset" input is generated using the inverter. Thus, the **D-type** or **D flip flop**, which is level-sensitive, is constructed from a level-sensitive SR flip flop.

The **D flip flop** is widely utilized in digital systems and is commonly used in **shift registers**, **counters**, and **input synchronization** applications.



Figure 6.7: *D-Flip Flop Block Diagram*

We know that the SR flip-flop requires two inputs, i.e., one to **SET** the output and another to **RESET** the output. By using an inverter, we can set and reset the outputs with only one input, as the two input signals complement each other. In the SR flip-flop, when both the inputs are 0, that state is no longer possible. This ambiguity is removed by the complement in the **D Flip-Flop**.

In the **D Flip-Flop**, the single input D is referred to as the **Data** input. When the data input is set to 1, the flip-flop enters the set state, and when it is set to 0, the flip-flop resets. However, this approach can be inefficient, as the output of the flip-flop would change with every pulse applied to the data input.

To address this, a **CLOCK** or **ENABLE** input is introduced to isolate the data input from the flip-flop's latching mechanism. When the clock input is **high**, the condition of the D input is transferred to the output Q . This operation forms the foundation of the sequential device known as the **D Flip-Flop**.

When the clock input is 1, the **SET** and **RESET** inputs of the flip-flop are both held at 1, preventing any changes in the state. The flip-flop retains the data present at its output until the next clock transition. Simply put, the output is **latched** at either 0 or 1, depending on the data input at the moment the clock signal is active.

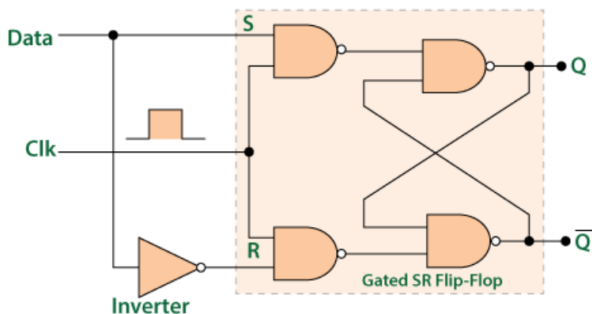


Figure 6.8: D Flip Flop Gates Diagram

Truth Table:

Clock	D	Q	$\overline{Q}$	Description
$\downarrow 0$	X	Q	$\overline{Q}$	Memory no change
$\uparrow 1$	0	0	1	Reset $Q \rightarrow 0$
$\uparrow 1$	1	1	0	Set $Q \rightarrow 1$

Table 6.3: Truth Table for D Flip Flop

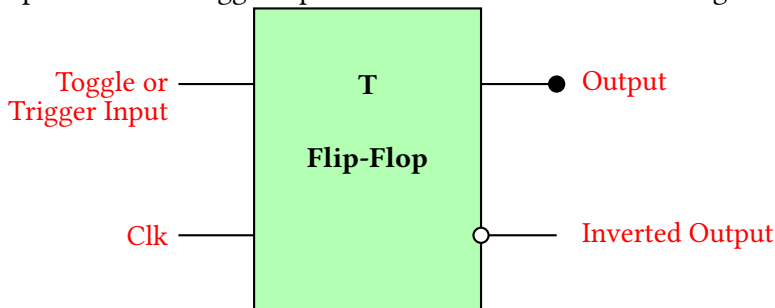
Symbols $\downarrow$ and $\uparrow$ indicates the direction of the clock pulse. D-type flip flop assumed these symbols as edge-triggers.

T Flip Flop

In a **T flip flop**, the "T" stands for **Toggle**. Unlike the SR Flip Flop, the T Flip Flop uses a single input, referred to as the "Toggle" or "Trigger" input, to eliminate the occurrence of intermediate states. This flip flop functions like a toggle switch, where the next output state is the complement of the current output state. This behavior is referred to as **toggling**.

The **T Flip Flop** can be constructed by modifying a **JK Flip Flop**. It has only one input, created by connecting both inputs of the JK Flip Flop together. This single input is labeled as T . Simply put, the **T Flip Flop** is a variation of the JK Flip Flop. In some cases, it is also referred to as a single-input JK Flip Flop.

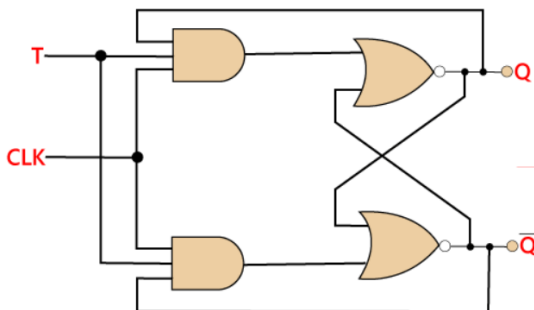
The block diagram of the **T Flip Flop** is provided below, where T represents the "Toggle input" and **CLK** denotes the clock signal input.



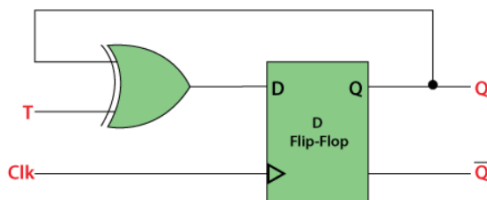
Construction

The "T Flip Flop" is designed by passing the AND gate's output as input to the NOR gate of the "SR Flip Flop". The inputs of the "AND" gates, the present output state Q , and its complement Q' are sent back to each AND gate. The toggle input is passed to the AND gates as input. These gates

are connected to the Clock (CLK) signal. In the "T Flip Flop", a pulse train of narrow triggers are passed as the toggle input, which changes the flip flop's output state. The circuit diagram of the "T Flip Flop" using "SR Flip Flop" is given below:

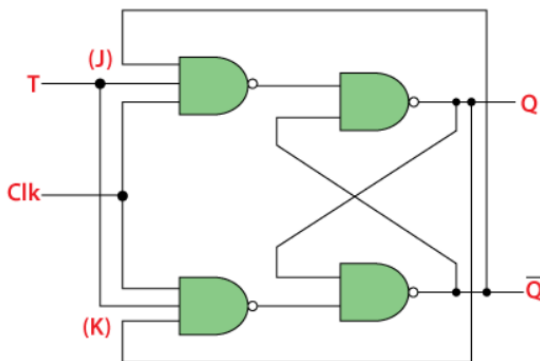


The "T Flip Flop" is formed using the "D Flip Flop". In D flip - flop, the output after performing the XOR operation of the T input with the output "QPREV" is passed as the D input. The logical circuit of the "T-Flip Flop" using the "D Flip Flop" is given below:



The simplest construction of a D Flip Flop is with JK Flip Flop. Both

the inputs of the "JK Flip Flop" are connected as a single input T. Below is the logical circuit of the T Flip Flop" which is formed from the "JK Flip Flop":



Truth Table:

T	Previous		Next	
	Q	Q'	Q	Q'
0	0	1	0	1
0	1	0	1	0
1	0	1	1	0
1	1	0	0	1

Table 6.4: Truth Table for T Flip-Flop

The upper NAND gate is enabled, and the lower NAND gate is disabled when the output Q is set to 0. make the flip flop in "set state($Q = 1$)", the trigger passes the S input in the flip flop.

The upper NAND gate is disabled, and the lower NAND gate is enabled when the output Q is set to 1. The trigger passes the R input in the flip flop to make the flip flop in the reset state($Q = 0$).

Operations of T Flip-Flop

When the **T input** is set to 0, the next state of the T Flip-Flop remains the same as the current state:

- If the toggle input is 0 and the current state is 0, the next state will also be 0.

- If the toggle input is 0 and the current state is 1, the next state will also be 1.

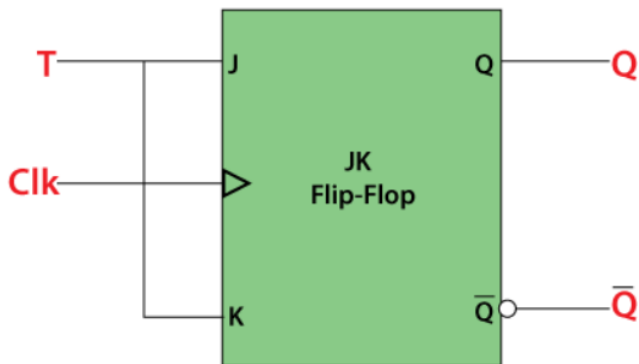
When the **T input** is set to 1, the next state of the flip-flop becomes the opposite of the current state:

- If the toggle input is 1 and the current state is 0, the next state will change to 1.
- If the toggle input is 1 and the current state is 1, the next state will change to 0.

The T Flip-Flop toggles its state when the set and reset inputs are alternately triggered by an incoming signal. It requires two trigger pulses to complete one full cycle of its output waveform. As a result, the frequency of the output signal is half that of the input frequency, making the T Flip-Flop function as a **Frequency Divider Circuit**.

In the T Flip-Flop, the state after a trigger pulse depends on the definition of the previous state, which is a significant drawback.

The T Flip-Flop can be constructed from **JK Flip-Flop**, **SR Flip-Flop**, or **D Flip-Flop**, as it is not available in integrated circuit (IC) form. The block diagram of a T Flip-Flop using a JK Flip-Flop is shown below.



6.3 Latches in Digital Electronics

A latch is an asynchronous sequential circuit in which the output instantly responds to any changes in the applied input. It functions as a fundamental memory element in a digital system, storing a single bit of information. This chapter provides an in-depth explanation of latches in digital electronics, covering their various types and applications.

In this chapter, we will explain in detail about latches in digital electronics along with their types and applications.

What is a Latch?

In digital electronics, a latch is an asynchronous sequential circuit capable of storing a single bit of information. It serves as a fundamental memory element in digital circuits.

A latch has two stable states: set and reset. The set state corresponds to logic 1, while the reset state is represented by logic 0. Because of these

two stable states, a latch is also referred to as a bistable multivibrator. Its state changes based on the applied input.

One key characteristic of latches is that they do not require a clock signal for synchronization, which is why they are classified as *asynchronous sequential circuits*.

Logic gates form the basic building blocks of latches. Since there is no synchronization or clock signal involved, latches respond immediately to input signal changes.

Characteristics of Latches

Here are some key characteristics of latches:

- Latches are capable of storing a single bit of digital information, represented as either logic 0 or logic 1. As a result, they primarily function as memory elements in digital circuits.
- They incorporate a feedback mechanism that enables them to retain their current state until a new input is provided.
- The behavior of latches is entirely determined by the applied inputs, meaning their output changes in response to modifications in the input signals.

Types of Latches

The following are the main types of latches that are used in digital circuits and systems:

- SR Latch
- JK Latch

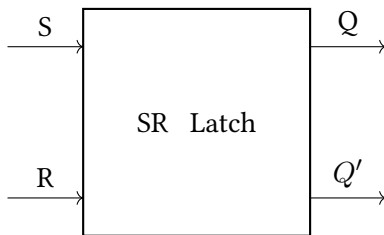
- D Latch
- T Latch

Let us now discuss each type of latch in detail.

SR Latch

The **SR latch** is a type of latch with two input lines labeled as **S** and **R**, where **S** stands for the *Set* input and **R** denotes the *Reset* input. For this reason, it is also referred to as a **Set-Reset Latch**.

This latch has two stable states: the *Set state* (*S*) and the *Reset state* (*R*). The block diagram of the SR latch is illustrated in the following figure.



For an **SR latch**, when the **S** input is activated, the output *Q* is set to 1, while *Q'* becomes 0. Conversely, when the *R* input is activated, *Q* is set to 0 and *Q'* becomes 1. If both *S* and *R* inputs are high simultaneously, the latch enters an **undefined or forbidden state**.

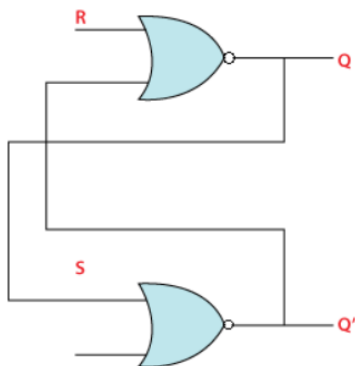
The complete functionality of the SR latch for various input conditions is summarized in the truth table below.

S	R	Q	Q'	Comment
0	0	Q	Q'	No change
0	1	0	1	Reset state
1	0	1	0	Set state
1	1	X	X	Forbidden state

Table 6.5: Truth table for the SR Latch

From this truth table, it is clear that the problem of forbidden state is addressed by implementing the toggle state.

The SR latch can be implemented by connecting two NOR gates in a cross-coupled manner as shown in the following figure.



JK Latch

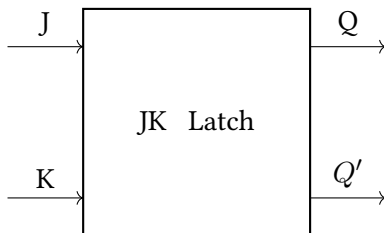
The **JK latch** is another type of latch that consists of two inputs, labeled **J** and **K**. Here, the **J** input functions similarly to the **S** input, while the **K**

input corresponds to the **R** input in an **SR latch**.

Although the JK latch operates similarly to the SR latch, it eliminates the **forbidden state**. Instead, when both **J** and **K** inputs are **1**, the latch enters a **toggle state**, causing the outputs Q and Q' to switch their states.

Thus, the **JK latch** is specifically designed to address the **forbidden state** issue found in the **SR latch**.

The block diagram of the JK latch is illustrated in the following figure:



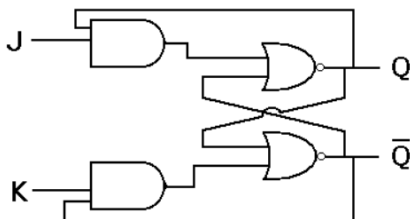
The truth table given below describes the operation of the JK latch for different input combinations:

J	K	Q	Q'	Comment
0	0	Q	Q'	No change
0	1	0	1	Reset state
1	0	1	0	Set state
1	1	Q'	Q	Toggle state

Table 6.6: Truth table for the JK Latch

From this truth table, it is clear that the problem of forbidden state is addressed by implementing the toggle state.

The logic circuit of the JK latch consists of a combination of two NOR gates and two AND gates as shown in the following figure.

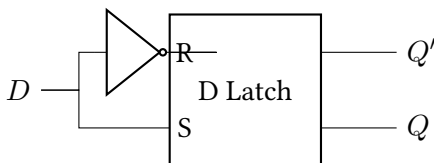


D Latch

The **D Latch**, also referred to as a **Data latch** or **transparent latch**, is a type of **bistable multivibrator** with two input signals: **D (Data) input** and **E (Enable) input**.

The output Q of the **D latch** follows the input applied at the D input line as long as the E input remains **high**. However, when the E input transitions to **low**, the output retains its current state until a new input is applied to the D input.

The block diagram of the **D latch** is presented in the following figure.

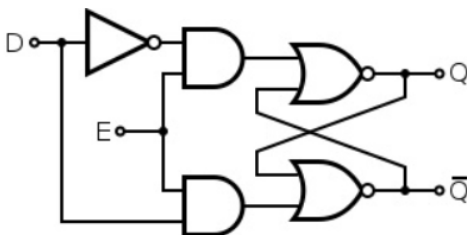


The truth table given below explains the operation of the D latch:

D	E	Q	$\bar{Q}$	Comment
0	0	Q	$\bar{Q}$	No change
0	1	0	1	Reset state
1	0	Q	$\bar{Q}$	No change
1	1	1	0	Set state

Table 6.7: Truth table for the D Latch

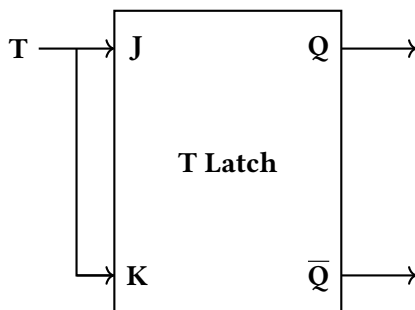
The logic circuit diagram of the D latch is depicted in the following figure



T Latch

The **T latch** is a type of latch that **toggles** its output state (Q) whenever a logic 1 is applied to its input. For this reason, it is also called a **toggle latch**.

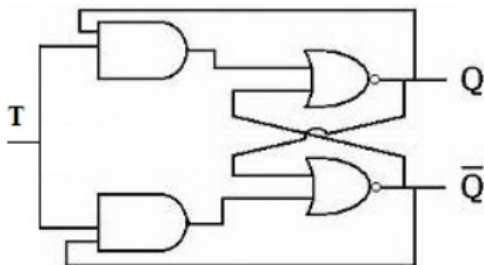
The **T latch** is constructed by connecting the **J** and **K** inputs of a **JK latch** together, as illustrated in the following block diagram.

Figure 6.9: *T Latch Block Diagram*

The truth table describing the operation of the T latch is shown below:
The logic circuit diagram of the T latch is shown in the following figure:

T	Q (Present State)	$\bar{Q}$	Q (Next State)	$\bar{Q}$
0	0	1	0	1
0	1	0	1	0
1	0	1	1	0
1	1	0	0	1

Table 6.8: *Truth table for the T Latch*



Applications of Latches

Latches have numerous applications in the field of **digital electronics**. As the most basic storage components, they are primarily used to store a **single bit of information** in digital systems.

Some of the common applications of latches include:

- Serving as **1-bit memory elements** in digital circuits.
- Contributing to the design of **digital registers**, which are used for data storage and processing in **microprocessors** and **micro controllers**.
- Playing a key role in the development of **flip-flops**, which are essentially synchronized versions of latches.
- Being utilized in **communication systems** for **temporary data storage** or **buffering purposes**.

Exercises

Problem 1

What will be the output of an **SR flip-flop** if:

- $S = 1$, $R = 0$, and the current state $Q = 0$?

Problem 2

Why is the condition $S = 1$ and $R = 1$ considered invalid in an **SR flip-flop**?

Problems on JK Flip-Flop

Problem 3

In a **JK flip-flop**, if $J = 0$, $K = 1$, and the clock input is active (High), what will be the output Q if the current state is $Q = 1$?

Problem 4

What happens when both inputs J and K are set to 1 in a **JK flip-flop**?

Problems on T Flip-Flop

Problem 5

If a **T flip-flop** has an initial state $Q = 0$ and the toggle input $T = 1$, what will the state of Q be after one clock pulse?

Problems on D Flip-Flop

Problem 6

In a **D flip-flop**, if the data input $D = 1$ and the clock input is active, what will be the output Q ?

Bibliography

- [1] M. Morris Mano and Michael D. Ciletti, *Digital Design*, 6th ed., Pearson, 2017.
- [2] Ronald J. Tocci, Neal S. Widmer, and Gregory L. Moss, *Digital Systems: Principles and Applications*, 12th ed., Pearson, 2016.
- [3] Randy H. Katz and Gaetano Borriello, *Contemporary Logic Design*, 2nd ed., Pearson, 2020.
- [4] Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed., McGraw-Hill Education, 2012.
- [5] John F. Wakerly, *Digital Design: Principles and Practices*, 5th ed., Pearson, 2017.
- [6] Thomas L. Floyd, *Digital Fundamentals*, 11th ed., Pearson, 2014.
- [7] Charles H. Roth and Larry L. Kinney, *Fundamentals of Logic Design*, 7th ed., Cengage Learning, 2013.
- [8] Parag K. Lala, *Digital Logic and Computer Design*, 1st ed., Pearson, 2010.

- [9] Donald D. Givone, *Digital Principles and Design*, 1st ed., McGraw-Hill Education, 2015.
- [10] Hung T. Nguyen and Elbert A. Walker, *Digital Logic Design*, 1st ed., CRC Press, 2016.